Universidade do Estado do Rio de Janeiro
Instituto de Química
Programa de Pós-Graduação em Engenharia Química
Prof. Aderval S. Luna, D.Sc. - Professor Titular
Igor C. A. Lima, D.Sc. - Pós-Doutorando

Planejamento e Otimização de Experimentos: Conceitos fundamentais

Última atualização: 20 de setembro de 2022, 08:55:13

Sumário

Emprego do programa R e sua interface Rstudio **7**

Criação de vetor de dados . . . 7

Operação de soma no vetor . . . 7

Medidas estatísticas descritivas . . . 7

Média aritmética simples . . . 7

Variância amostral . . . 7

Desvio-padrão amostral . . . 8

Dados Faltantes: como lidar? . . . 8

Funções criadas pelo usuário: uma pitada de programação no R . . . 8

Vetores: revisitando . . . 9

Matrizes . . . 10

Transposição de matriz: . . . 11

Localização de elementos na matriz . . . 11

Fatores . . . 12

Listas . . . 12

Data frames . . . 13

Extraindo dados de uma data frame . . . 14

Gráficos **16**

Histograma . . . 17

Gráfico de Barras . . . 19

Gráfico de Pizza . . . 20

Gráfico de Pontos . . . 21

Revendo o gráfico de barras . . . 22

Gráfico de dispersão . . . 24

Matriz do diagrama de dispersão . . . 26

Diagrama de Caixa . . . 27

Gráficos elegantes: ggplot2 . . . 28

Histograma . . . 28

Gráfico de barras . . . 29

Gráfico de Pontos . . . 30

Matriz do diagrama de dispersão . . . 31

Diagrama de caixa . . . 33
Medidas de Centralidade . . . 35
Média aritmética: . . . 35
Emprego da função with . . . 35
Média aparada ou truncada (trimmed mean). . . . 37
Mediana . . . 37
Moda . . . 38
Medidas de Dispersão . . . 38
Variância Amostral usando a função var() . . . 38
Desvio-padrão . . . 39
Padronização . . . 40
Percentil . . . 41
Função summary . . . 41
Quadro resumido . . . 41
Momentos em Estatística Descritiva . . . 43
Definição de assimetria . . . 44
Curtose . . . 46
Frequência dos dados . . . 46
Variáveis nominais . . . 46
Variáveis numéricas . . . 47
Frequência cumulativa . . . 48
Variáveis numéricas: stem (diagrama de ramo e folhas) . . . 52
Resumo de uma data frame . . . 52

Distribuições de probabilidades **53**
Distribuição Normal . . . 53
Função de densidade cumulativa . . . 56
Quantis de distribuição normal . . . 59
Quantis . . . 60
Distribuição Binomial . . . 62
Influência do tamanho da amostra na Dist. Binomial . . . 64
Distribuição de Poisson . . . 66
Distribuição Exponencial . . . 68
Influência do tamanho da amostra . . . 68

Amostragem Aleatória **70**

Funções da distribuição normal no R 70

Estimação e Intervalo de Confiança **73**

Simulando o teorema do limite central 73

Previsões usando o teorema do limite central 75

Intervalo de confiança para a média usando a estatística z 76

Intervalo de confiança para a média usando a estatística t 76

Teste de hipóteses para uma amostra **78**

Teste de hipóteses usando a estatística Z 78

Teste de hipóteses usando a estatística t 78

Trabalhando com a Distribuição t 79

Teste da Variância 83

Trabalhando com Distribuição qui-quadrado 84

Teste de hipóteses para duas amostras **88**

Teste Z 88

Teste de hipóteses para duas amostras com a estatística t 89

Variâncias iguais 89

Variâncias diferentes 89

Teste t para amostras dependentes ou pareadas 89

Teste de igualdade de variâncias 89

Trabalhando com a distribuição F 93

Visualizando a distribuição F 94

Análise da Variância (ANOVA One-way) **96**

Procedimento de Tukey-Kramer de comparações múltiplas 99

Homogeneidade da variância 101

Teste de Levene 101

Teste de Bartlett 101

ANOVA de dois fatores (ANOVA Two-way) **103**

Modelos Empíricos: Regressão linear simples **109**
Modelo de regressão linear simples . . . 109
Equação da linha de previsão . . . 109
Coeficiente de determinação . . . 109
Coeficiente de determinação ajustado . . . 109
Erro-padrão dos coeficientes angular e linear . . . 110
Erro-padrão residual . . . 110
Análise dos resíduos . . . 110
Resíduo padronizado . . . 110
Resíduos estudentizados . . . 111
Pontos de alavancagem . . . 111
Análise de resíduos: Teste de Breusch-Pagan (heterocedasticidade) . . . 111
Exemplos . . . 112

Modelos Empíricos: Regressão Linear Múltipla (MLR) **118**
Modelo de regressão com k variáveis independentes . . . 118
Equação da linha de previsão . . . 118
Métricas . . . 118
Raiz do Erro Quadrático Médio (RMSE) . . . 118
Erro Quadrático Médio (MSE) . . . 118
Erro Absoluto Médio (MAE) . . . 118
Multicolineridade: Tolerância e Fator de Inflação da Variância . . . 118
Seleção de variáveis . . . 119
Cp de Mallows . . . 119
Critério de Informação de Akaike . . . 119
Análise de resíduos: pontos influentes . . . 119
Difference in Fits, DFFITS . . . 119
Difference in Betas, DFBETAS . . . 119
Distância de Cook . . . 120
Regressão por passos (Stepwise) . . . 132
Regressão por passos para trás (Backward Regressrion) . . . 135

PPGEQ/UERJ

PPGEQ/UERJ

Emprego do programa R e sua interface Rstudio

Criação de vetor de dados

A linguagem `R` é orientada a objetos. Isso significa que dentro desse ambiente, escalares, vetores, matrizes são objetos.

Assim como a simbologia matemática de funções, na linguagem `R` precisamos indicar o que um objeto receberá. A atribuição "de algo" para "alguma coisa" é feita com os símbolos menor, $<$, seguido pelo símbolo de menos, $-$, da seguinte forma: `<-` ou então usando o sinal de igualdade, `=`.

Na literatura, o uso de `<-` é feito na atribuição de dados/funções à objetos, enquanto que o símbolo `=` é usado dentro dos argumentos das funções.

```
# Criação do vetor x:
x <- c(3, 4, 5)
# Digite x e aperte Enter para ver o resultado:
x
```

```
## [1] 3 4 5
```

Operação de soma no vetor

```
# Soma dos valores do vetor, usando função nativa:
sum(x)
```

```
## [1] 12
```

Medidas estatísticas descritivas

Ao instalar o software R, ele vem com o pacote `stats`, que permite o cálculo de algumas medidas descritivas.

Média aritmética simples

```
# Média aritmética:
mean(x)
```

```
## [1] 4
```

Variância amostral

PPGEQ/UERJ

```r
# variância amostral:
var(x)
```

```
## [1] 1
```

Desvio-padrão amostral

```r
# Desvio-padrão amostral:
sd(x)
```

```
## [1] 1
```

Dados Faltantes: como lidar?

Considere os seguintes dados, referentes à capacidade de bagagem de veículos em metros cúbicos:

$$C = \{14, 13, 14, 13, 16, NA, NA, 20, NA\}$$

```r
# Dados de capacidade de bagagem:
cap <- c(14, 13, 14, 13, 16, NA, NA, 20, NA)
# O que ocorrerá se a média for executada?
mean(cap)
```

```
## [1] NA
```

A operação correta é feita removendo os falores faltantes,`NA`, na hora da operação, através do argumento `na.rm=TRUE`:

```r
mean(cap, na.rm = TRUE)
```

```
## [1] 15
```

É possível verificar se um vetor apresenta, ou não, valores faltantes, através da função `is.na`.

```r
# Função is.na(objeto)
is.na(cap)
```

```
## [1] FALSE FALSE FALSE FALSE FALSE  TRUE  TRUE FALSE  TRUE
```

Funções criadas pelo usuário: uma pitada de programação no R

A construção de uma função no R segue a seguinte estrutura:

PPGEO/UERJ

```r
# Exemplo genérico
myfunction <- function(argument1, argument2, ...){
  + statments
  + return(object)
}
```

Suponha que desejamos construir uma função de soma dos quadrados de três números. No `R`, temos:

```r
# Construção da função:
sumofsquares <- function(x, y, z){
  sumsq <- sum(c(x^2, y^2, z^2))
  return(sumsq)
  }
# Usando a função construída:
sumofsquares(3, 4, 5)
```

```
## [1] 50
```

Vetores: revisitando

Um vetor é uma uma estrutura de dados básica do R, que permite armazenar um conjunto de valores numéricos ou de caracteres sob um mesmo nome. Esses valores podem ser acessados individualmente através de índices.

Na criação de sequências, podemos usar tanto o operador `:` (que define somente o início e o final, com incremento unitário) quanto usar a função `seq`, que precisa, no mínimo, de três argumentos: `from=` (valor inicial), `to=` (valor final) e `by=` (incremento da sequência).

```r
# Vetores numéricos
# Criação de um vetor de números de 10 a 30
y1 <- 10:30
y1
```

```
##  [1] 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
```

```r
# Criação de um vetor de números de 10 a 30, com definição do incremento:
y2 <- seq(10, 30, 1)
y2
```

```
##  [1] 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
```

PPGEQ/UERJ

```r
# Verificação
y1 == y2
```

```
##  [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [16] TRUE TRUE TRUE TRUE TRUE TRUE
```

```r
# Criação de uma sequência com incremento igual a 2:
w <- seq(10, 30, 2)
w
```

```
##  [1] 10 12 14 16 18 20 22 24 26 28 30
```

```r
# Criação de um vetor com valores repetidos:
trifecta <- c(6, 8, 2)
repeat_trifecta <- rep(trifecta, times = 4)
repeat_trifecta
```

```
##  [1] 6 8 2 6 8 2 6 8 2 6 8 2
```

Matrizes

As matrizes se caracterizam por um conjunto de vetores. Para criar uma matriz, pode-se utilizar a função `matrix()`, que possui dois argumentos: o primeiro composto por um vetor com os elementos da matriz e o segundo composto pela informação do número de linhas (`nrow`) ou de colunas (`ncol`) da matriz.

```r
# Criação de uma matriz numérica com 5 linhas e 4 colunas:
num_matrix <- seq(5, 100, 5)
dim(num_matrix) <- c(5, 4)
# Visualização da matriz construída:
num_matrix
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    5   30   55   80
## [2,]   10   35   60   85
## [3,]   15   40   65   90
## [4,]   20   45   70   95
## [5,]   25   50   75  100
```

Outra forma de construção de uma matriz é através da função `matrix`. Ela apresenta quatro argumentos:

- ▷ `data`: que é o vetor com os valores.

PPGEQ/UERJ

- ▷ `nrow`: número de linhas da matriz.
- ▷ `ncol`: número de colunas da matriz.
- ▷ `byrow`: função lógica que indica se o preenchimento será por linhas ou por colunas.

```
# Criação de uma matriz com a função matrix:
num_matrix <- matrix(
  seq(5, 100, 5),
  nrow = 5,
  ncol = 4,
  byrow = TRUE)
num_matrix
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    5   10   15   20
## [2,]   25   30   35   40
## [3,]   45   50   55   60
## [4,]   65   70   75   80
## [5,]   85   90   95  100
```

Transposição de matriz:

```
# Obtenção da matriz transposta:
mat_transp <- t(num_matrix)
mat_transp
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    5   25   45   65   85
## [2,]   10   30   50   70   90
## [3,]   15   35   55   75   95
## [4,]   20   40   60   80  100
```

Localização de elementos na matriz

Para identificar, por exemplo, qual o elemento que está na 5ª linha e 4ª coluna, na matriz representada pelo objeto criado `num_matrix`, fazemos da seguinte forma:

```
# localização do valor na 5ª linha e 4ª coluna:
num_matrix[5 , 4]
```

```
## [1] 100
```

PPGEQ/UERJ

Fatores

Suponha que estejamos fazendo uma pesquisa sobre a cor dos olhos das pessoas. Quando registramos a cor dos olhos, indicamos um número, conforme a tabela a seguir.

Tabela 1: Codificação de cor dos olhos.

Código	Cor
1	amber(âmbar)
2	blue (azul)
3	brown (castanho)
4	gray (acinzentado)
5	green (verde)
6	hazel (mel)

Podemos considerar que a cor do olho é um **fator** e cada cor o seu **nível**.

```
# Fator é o termo do R para uma variável nominal ou categórica
eye_color <- c(2, 2, 4, 1, 5, 5, 5, 6, 1, 3, 6, 3, 1, 4)
# Use a função factor() para transformar eye_color em um fator
feye_color <- factor(eye_color)
# Identificação dos níveis do fator:
levels(feye_color)
```

```
## [1] "1" "2" "3" "4" "5" "6"
```

```
# Atribuição de níveis ao fator
levels(feye_color) <- c("amber", "blue", "brown", "gray", "green", "hazel")
# Examinando-se os dados das cores dos olhos em termos de níveis de fatores
levels(feye_color)
```

```
## [1] "amber" "blue"  "brown" "gray"  "green" "hazel"
```

Listas

Uma lista é uma coleção de objetos que não são necessariamente do mesmo tipo. Suponha que você tenha coletado uma nota para a simpatia das pessoas.

```
# base de dados
empathy_score <- c(15, 21, 45, 32, 61, 74, 53, 92, 83, 22, 67, 55, 42, 44)
```

Combinando-se o vetor dos olhos na forma codificada, o vetor da cor dos olhos na forma de fator e vetor de nota para a simpatia das pessoas.

PPGEQ/UERJ

```r
# Combinação usando a função list:
eyes_and_empathy <- list(eyes_code = eye_color,
                         eyes = feye_color,
                         empathy = empathy_score)
# Verificação:
eyes_and_empathy
```

```
## $eyes_code
##  [1] 2 2 4 1 5 5 5 6 1 3 6 3 1 4
##
## $eyes
##  [1] blue  blue  gray  amber green green green hazel amber brown hazel brown
## [13] amber gray
## Levels: amber blue brown gray green hazel
##
## $empathy
##  [1] 15 21 45 32 61 74 53 92 83 22 67 55 42 44
```

Caso queira se referir a um componente da lista, digite o nome da lista, o cifrão e o número do componente

```r
# Especificação de um componente:
eyes_and_empathy$empathy
```

```
##  [1] 15 21 45 32 61 74 53 92 83 22 67 55 42 44
```

Caso queira uma nota específica, por exemplo, a quarta, fazemos:

```r
# Obtenção do 4º valor do nível empathy
eyes_and_empathy$empathy[4]
```

```
## [1] 32
```

Data frames

A função `data.frame()` trabalha com os vetores existentes para realizar a tarefa.

```r
# Criação do data frame
e <- data.frame(eye_color, feye_color, empathy_score)
# Visualização
e
```

PPGEQ/UERJ

```
##    eye_color feye_color empathy_score
## 1          2       blue            15
## 2          2       blue            21
## 3          4       gray            45
## 4          1      amber            32
## 5          5      green            61
## 6          5      green            74
## 7          5      green            53
## 8          6      hazel            92
## 9          1      amber            83
## 10         3      brown            22
## 11         6      hazel            67
## 12         3      brown            55
## 13         1      amber            42
## 14         4       gray            44
```

```r
# Qual a nota da empatia da sétima pessoa?
e[7 , 3]
```

```
## [1] 53
```

```r
# Quais as informações da sétima pessoa?
e[7 , ]
```

```
##   eye_color feye_color empathy_score
## 7         5      green            53
```

Extraindo dados de uma data frame

Suponha que você queira fazer uma comparação da média da nota da empatia com a cor dos olhos.

☞Tarefa inicial: extrair as notas de empatia para cada cor dos olhos e criar os vetores

```r
# Extração seletiva:
e.blue <- e$empathy_score[e$feye_color == "blue"]
e.green <- e$empathy_score[e$feye_color == "green"]
e.hazel <- e$empathy_score[e$feye_color == "hazel"]
# Criar um vetor de médias
e.averages <- c(mean(e.blue),
                mean(e.green),
                mean(e.hazel))
# Use a função length() para criar o vetor de notas de cada grupo
e.amounts <- c(length(e.blue),
               length(e.green),
               length(e.hazel))
```

PPGEQ/UERJ

```r
# Crie o vetor das cores
colors <- c("blue", "green", "hazel")
# Criação do data frame
e.averages.frame <- data.frame(color = colors,
                               average = e.averages,
                               n = e.amounts)
# Visualização
e.averages.frame
```

```
##   color  average n
## 1  blue 18.00000 2
## 2 green 62.66667 3
## 3 hazel 79.50000 2
```

Obs: O operador de sinal igual duplo (==) é um operador lógico ("se a for igual b"). A tabela a seguir contém um resumo das relações lógicas:

Tabela 2: relações lógicas no R:

Símbolo	Exemplo	Descrição
==	$a == b$	se a for *igual a* b
!=	$a != b$	se a for *diferente de* b
>=	$a >= b$	se a for *maior ou igual a* b
<=	$a <= b$	se a for *menor ou igual a* b
>	$a >= b$	se a for *maior que* b
<	$a <= b$	se a for *menor que* b

PPGEO/UERJ

Gráficos

Para este módulo, são necessários os pacotes {MASS}, {ggplot2}, {GGally}, {modeest}, {moments}, {BSDA}, {tigerstats}, {reshape2}, {EnvStats}, {dplyr} e {tidyr}. Use a sintaxe a seguir, para verificar se estão instalados e, caso não, sejam instalados.

```
# Lista com os pacotes necessários:
pacotes <- c("MASS", "ggplot2", "GGally", "modeest", "moments", "tigerstats",
             "BSDA", "reshape2", "EnvStats", "dplyr", "tidyr")
# Verificação e instalação:
if(!require(pacotes)){install.packages(pacotes)}
```

Uma alternativa mais fácil usando o `RStudio`, para instalação múltipla, é clicar na guia **Packages**, no botão **Install** e a seguir, indicar os pacotes, entre aspas, e separados por vírgula:

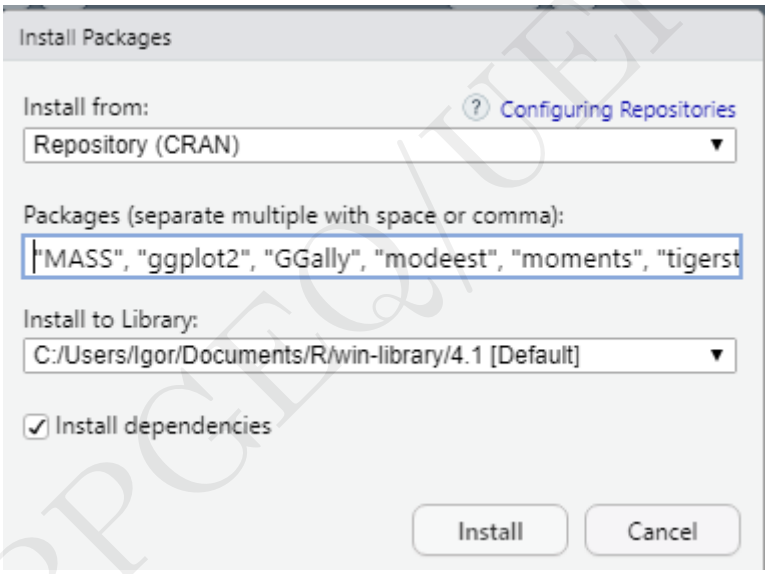


Figura 1: Instalação múltipla de pacotes, usando o RStudio.

Caso deseje deixar já carregados os pacotes, use a seguinte sintaxe:

```
# Ativação múltipla de pacotes:
lapply(c("MASS", "ggplot2", "GGally", "modeest", "moments", "tigerstats", "BSDA",
         "reshape2", "EnvStats", "dplyr", "tidyr"),
       fun = require,
       character.only = TRUE)
```

Independente da sua escolha, indicaremos sempre o pacote a ser carregado em cada atividade neste material didático.

Considere a base de dados **Cars93**, disponibilizada no pacote **MASS**, para os exemplos a seguir.

PPGEQ/UERJ

```r
# Requisição do pacote
library(MASS)
# Conjunto de dados disponível no pacote MASS
data(Cars93)
```

Informações sobre a base de dados, caso estejam disponíveis, são obtidas através do símbolo de interrogação, seguido do nome da base de dados.

```r
# Informações do data frame Cars93
?Cars93
```

Histograma

A função para construção de um histograma é a `hist()`. Ela apresenta diversos **argumentos**. A tabela a seguir apresenta alguns (mas não todos):

Tabela 3: Alguns argumentos da função `hist()`.

Argumento	Atividade
`xlab`	Rótulo do eixo x
`ylab`	Rótulo do eixo y
`xlim`	Limites do eixo x
`ylim`	Limites do eixo y
`main`	Título do gráfico
`probability`	Função para indicar a densidade de probabilidade
`col`	Cor da área do retângulo
`border`	Cor da borda do retângulo

A função `lines(density(x))`, combinada com o argumento `probability=TRUE`, gera uma linha com a densidade de probabilidade. O argumento `lty` modifica o tipo de linha.

```r
# Construir um histograma com a variável Price com a linha
# da função de densidade de probabilidade
hist(Cars93$Price,
     xlab = "Price (x $1,000)",
     xlim = c(0, 70),
     main = "Prices of 93 Cars Models of 1993 Cars",
     probability = TRUE,
     col = "slategray2")
lines(density(Cars93$Price),
      col = "red",
      lwd = 1.3,
      lty = "solid")
```

PPGEO/UERJ

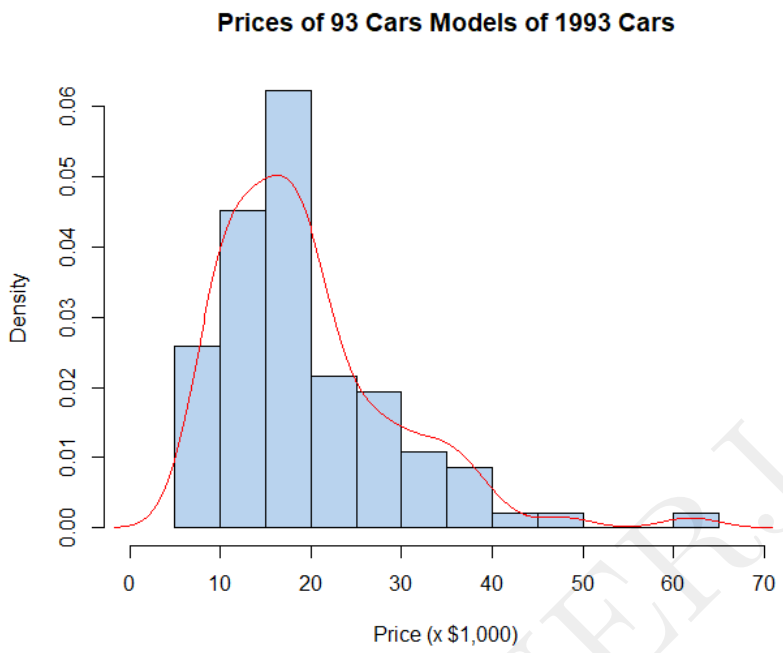


Figura 2: Histograma da variável Price.

Figura 3: Tipos de linhas

PPGEQ/UERJ

Gráfico de Barras

```
# Criar uma tabela extraindo os dados de Cars 93
table(Cars93$Type)
# Para gerar o gráfico de barras:
barplot(table(Cars93$Type))
```

Como qualquer gráfico no R, podemos edita-lo. Os limites nos eixos x e y são dados pelos argumentos `xlim` e `ylim`, respectivamente.

Para desenhar um eixo sólido, use `axis.lty = "solid"`.

Use `space = 0.05` para aumentar o espaçamento entre as barras.

```
# Função completa
barplot(table(Cars93$Type),
        ylim = c(0, 25),
        xlab = "Type",
        ylab = "Frequency",
        axis.lty = "solid",
        space = 0.05,
        col = cm.colors(6))
```

PPGEQ/UERJ

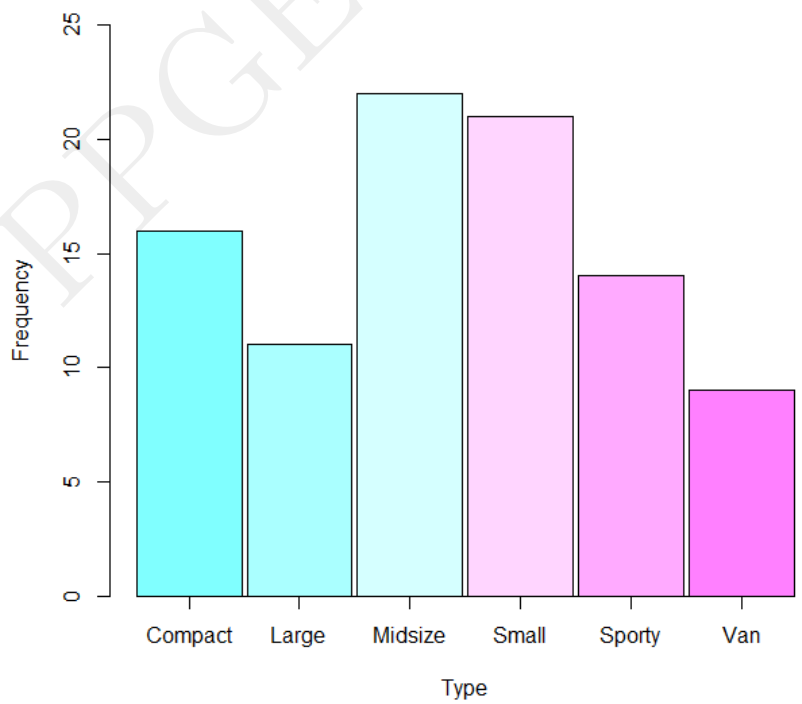


Figura 4: Gráfico de barras da variável Type.

O R apresenta algumas funções de cores contíguas: `heat.colors`, `cm.colors`, `terrain.colors`, `topo.colors` e `rainbow`.

Gráfico de Pizza

```
# Gráfico de pizza
pie(table(Cars93$Type),
    col= c("purple", "violetred1", "green3", "navy", "cyan", "thistle"),
    main= "Quantidade de carros")
```

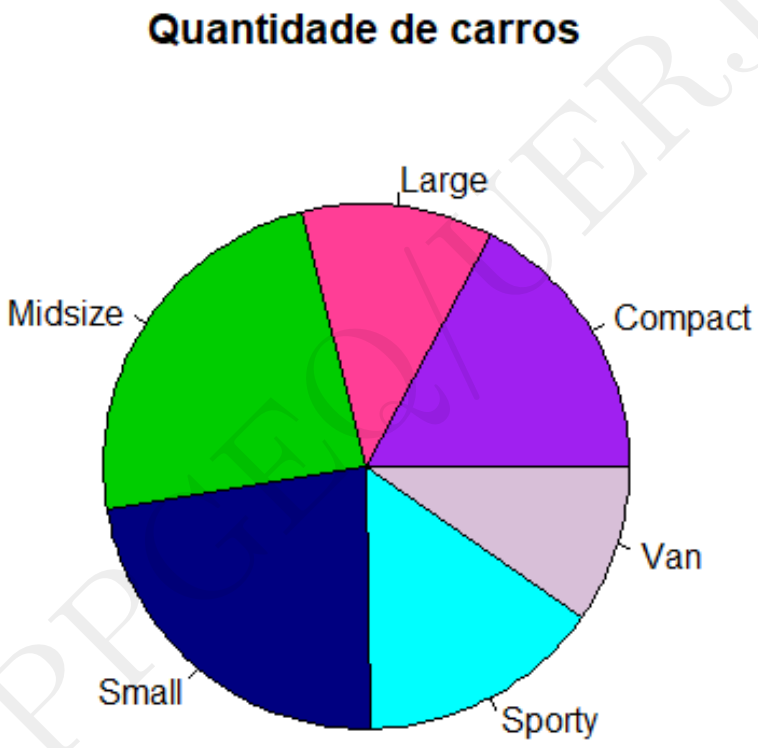


Figura 5: Gráfico de pizza da variável Type.

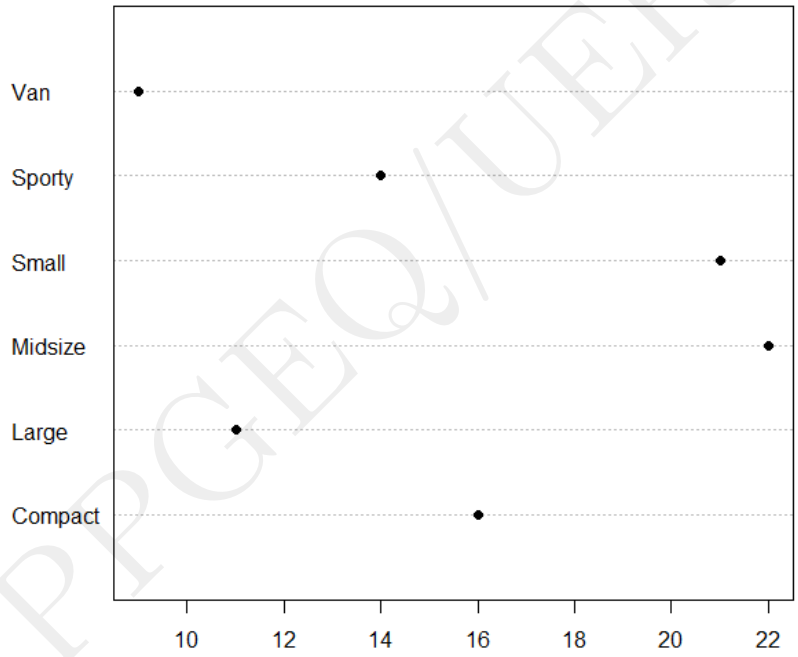
PPCEO/UERJ

Gráfico de Pontos

Para construir um gráfico de pontos no R, usando a função `dotchart(x, labels, arg1, arg2, ...)`. Os dois primeiros argumentos são vetores e os demais são argumentos operacionais.

Para criar os vetores necessários, transforma-se a tabela em um obejto `data.frame`.

```
# Transformação em objeto data.frame:
type.frame <- data.frame(table(Cars93$Type))
type.frame
# Para gerar o gráfico de pontos
dotchart(type.frame$Freq, type.frame$Var1)
# De forma opcional, pode-se usar o comando abaixo
dotchart(type.frame[,2], type.frame[,1])
```



PPGEQ/UERJ

Figura 6: Gráfico de pontos da variável Type.

Revendo o gráfico de barras

Tabela 4: Dados do Resumo Estatístico dos EUA 1990 - 1994.

Setor	1990	1991	1992	1993	1994
Commerce Satellites Delevered	1000	1300	1300	1100	1400
Satellites Services	800	1200	1500	1850	2350
Satellites Ground Equipment	860	1300	1400	1600	1970
Commercial Launches	570	380	450	465	580
Remote Sensing Data	155	190	210	250	300

```r
# Criar um vetor de valores
rev.values <- c(1000, 1300, 1300, 1100, 1400, 800, 1200, 1500, 1850, 2350, 860,
                1300, 1400, 1600, 1970, 570, 380, 450, 465, 580, 155, 190, 210,
                250, 300)
# Quantidade de dados no vetor:
length(rev.values)
```

```
## [1] 25
```

```r
# Transformar este vetor em matriz
space.rev <- matrix(rev.values,
                    nrow = 5,
                    byrow = TRUE)
# Visualização
space.rev
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,] 1000 1300 1300 1100 1400
## [2,]  800 1200 1500 1850 2350
## [3,]  860 1300 1400 1600 1970
## [4,]  570  380  450  465  580
## [5,]  155  190  210  250  300
```

```r
# Adicionar os nomes das linhas e colunas da matriz
colnames(space.rev) <- c("1990", "1991", "1992", "1993", "1994")
rownames(space.rev) <- c("Commerce Satellites Delivered", "Satellites Services",
                         "Satellite Ground Equipment", "Commercial Launches",
                         "Remote Sensing Data")
# Visualização
space.rev
```

PPGEQ/UERJ

```
##                                 1990 1991 1992 1993 1994
## Commerce Satellites Delivered 1000 1300 1300 1100 1400
## Satellites Services            800 1200 1500 1850 2350
## Satellite Ground Equipment     860 1300 1400 1600 1970
## Commercial Launches            570  380  450  465  580
## Remote Sensing Data            155  190  210  250  300
```

```r
# Vetor de cores para as barras
color.names <- c("black", "grey25", "grey50", "grey75", "white")
```

Uma vez preparada a base de dados, podemos facilmente criar o gráfico de barras:

```r
# Gráfico de Barra
barplot(space.rev,
        beside = TRUE,
        xlab = "Year", ylab = "Revenue (X $1,000)",
        col = color.names, ylim = c(0, 2500))
# Adicionar a legenda
legend(1, 2500,
       rownames(space.rev),
       cex = 1.0, fill = color.names, bty = "n")
```

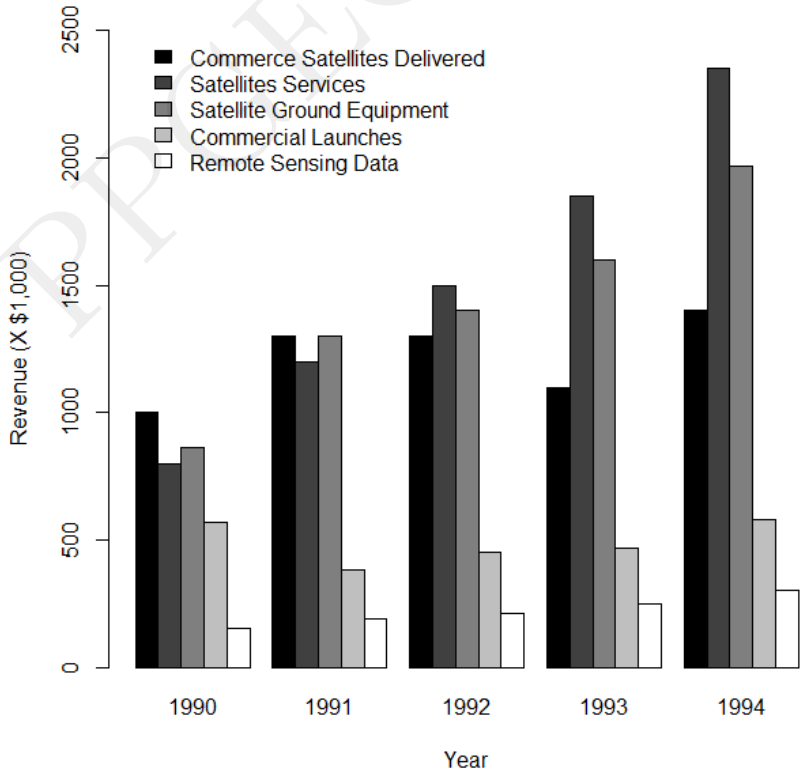


Figura 7: Gráfico de barras.

PPGEQ/UERJ

Gráfico de dispersão

Para o gráfico de dispersão, a função `plot()` tem, nos dois primeiros argumentos, `x =` e `y =`. Não há necessidade de digitá-los.

```
# Visualizar o relacionamento entre potência e MPG (milhas/galão)
# para dirigir na cidade
plot(Cars93$Horsepower, Cars93$MPG.city,
     xlab = "Horsepower",
     ylab = "MPG City",
     main = "MPG City versus Horsepower")
# Outra forma de escrever a função:
plot(Cars93$MPG.city ~ Cars93$Horsepower,
     xlab = "Horsepower",
     ylab = "MPG City",
     main = "MPG City versus Horsepower",
     pch = 16)
```

Para mudar o símbolo que representa os pontos usando o argumento `pch`. Valores de `pch`, de 0 a 15, correspondem a formas ocas. Já os valores de `pch` de 16 a 25 correspondem a formas preenchidas. O valor padrão é 1.

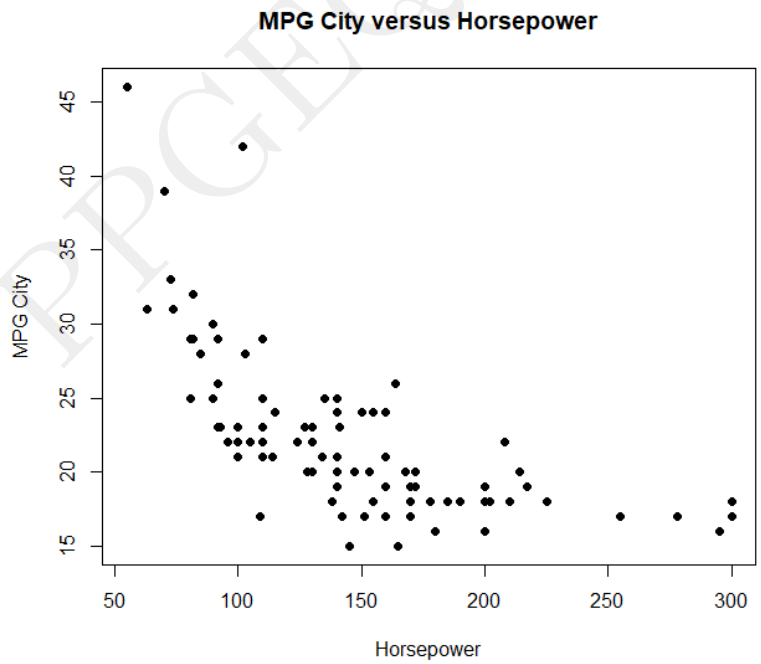


Figura 8: Diagrama de dispersão, das variáveis Horsepower e MPG.city.

Para ajudar a descobrir padrões nos dados, obrigue que o valor dos cilindros seja um caractere. Desta forma, aplique `as.character()` a `Cars93$Cylinder`:

PPGEQ/UERJ

```r
# Diagrama de dispersão:
plot(Cars93$Horsepower, Cars93$MPG.city,
     xlab = "Horsepower",
     ylab = "MPG City",
     main = "MPG City versus Horsepower",
     pch = as.character(Cars93$Cylinders))
```

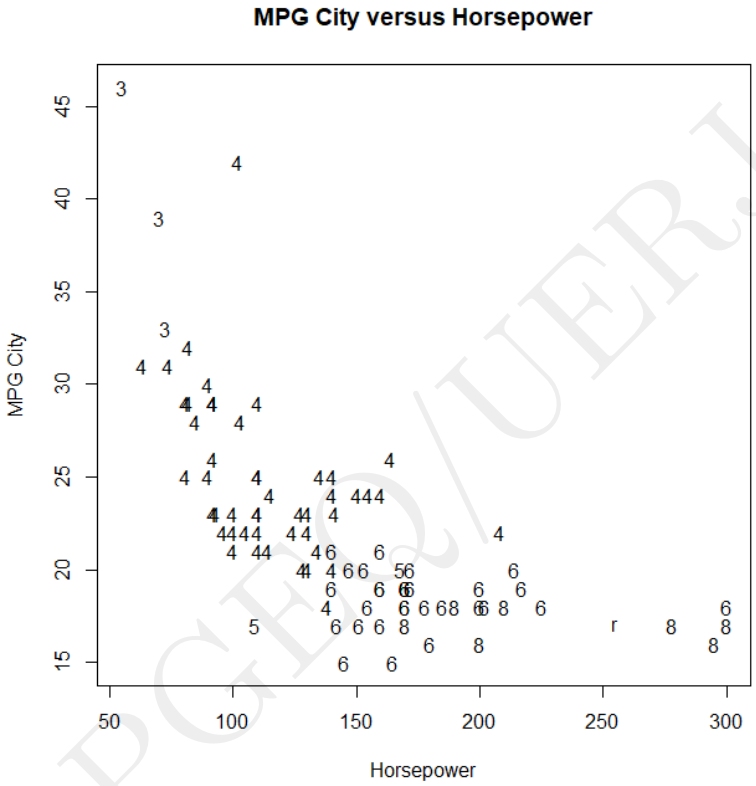


Figura 9: Diagrama de dispersão, das variáveis Horsepower e MPG.city, com identificação dos levels..

PPGEO/UERJ

Matriz do diagrama de dispersão

Por conveniência, cria-se um data frame que seja um subconjunto do data frame Cars93.

```
# Seleção da base de dados, com a função subset:
cars.subset <-subset(Cars93,
                     select = c(MPG.city, Price, Horsepower))
head(cars.subset)
```

```
##   MPG.city Price Horsepower
## 1       25  15.9        140
## 2       18  33.9        200
## 3       20  29.1        172
## 4       19  37.7        172
## 5       22  30.0        208
## 6       22  15.7        110
```

Após a preparação da base de dados, podemos gerar a matriz do diagrama de dispersão:

```
# Matriz do diagrama de dispersão
pairs(cars.subset, pch=16, col="blue", upper.panel = TRUE)
```

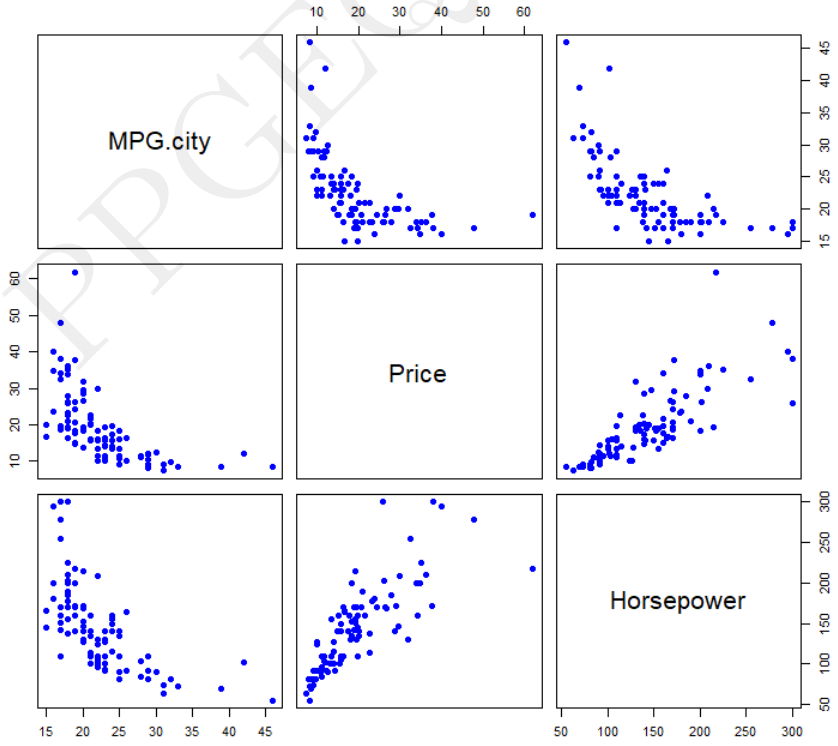


Figura 10: Matriz do diagrama de dispersão.

OBS.: Esta função, `pairs()` não é limitada a três variáveis.

PPGEQ/UERJ

Diagrama de Caixa

```r
# Gráfico de caixas
boxplot(Cars93$Horsepower ~ Cars93$Cylinders,
        xlab = "Cylinders",
        ylab = "Horsepower",
        col = heat.colors(6),
        border = "black",
        cex = 1.3)
```

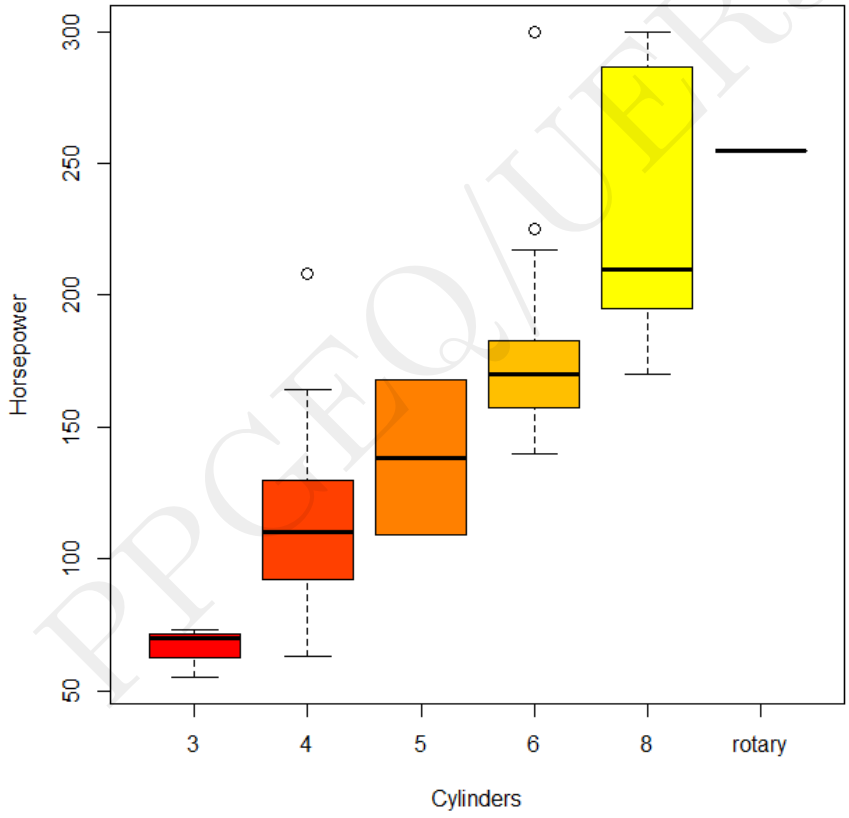


Figura 11: Gráfico de caixas.

PPGEQ/UERJ

Gráficos elegantes: ggplot2

Histograma

Utilizando o pacote `ggplot2`, apenas a sintaxe `ggplot(Cars93, aes(x = Price))` geraria apenas a figura com o background, mas sem o histograma. Ao adicionar a sintaxe `+ geom_histogram()` é gerado o histograma. A adição de `geom_histogram(binwidth=5, color = "black", fill = "white")` configura a estética do histograma. Para mudar os rótulos dos eixos e colocar um título no gráfico, colocamos `labs = "Price (x $1,000)", y="Frequency", title = "Price of 93 Models of 1993 Cars")`. Note que `gg` = gramática dos gráficos.

```
# Requisição do pacote:
library(ggplot2)
# Criação do histograma
ggplot(Cars93, aes(x = Price)) +
  geom_histogram(binwidth = 5,
                 color = "black",
                 fill = "white") +
  labs(x = "Price (x $1,000)",
       y="Frequency",
       title = "Price of 93 Models of 1993 Cars")
```

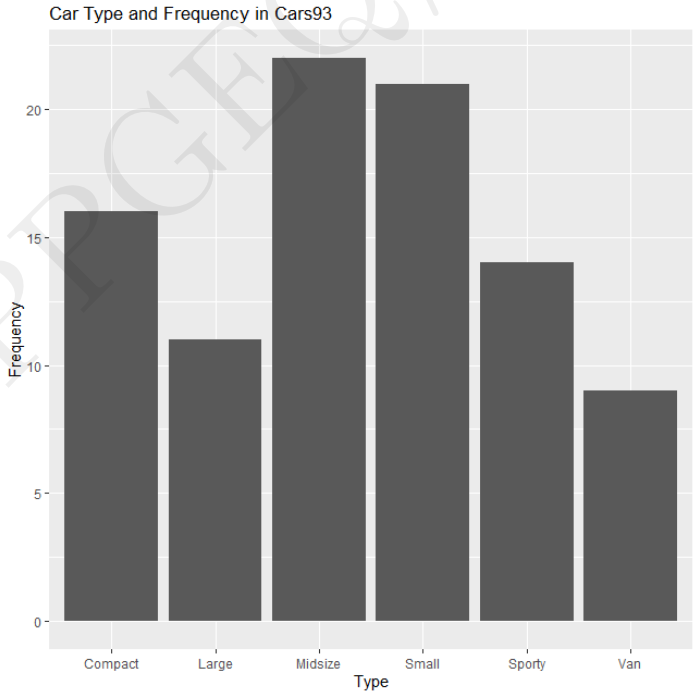


Figura 12: Histograma com ggplot2.

PPGEQ/UERJ

Gráfico de barras

```
# Criação do gráfico de barras
ggplot(Cars93, aes(x = Type)) +
  geom_bar() +
  labs(y="Frequency",
       title = "Car Type and Frequency in Cars93")
```

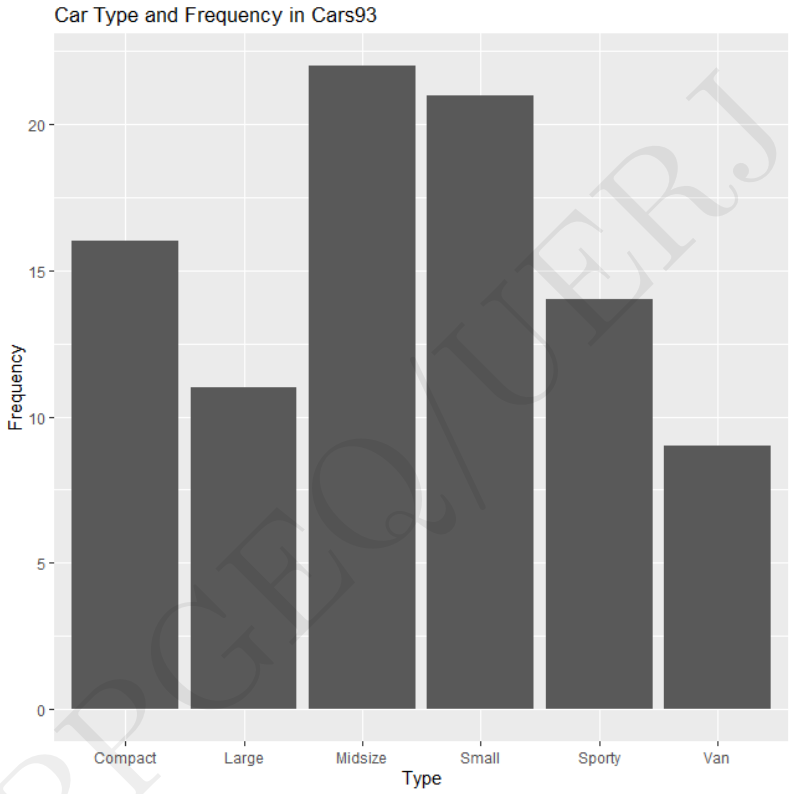


Figura 13: Gráfico de barras com ggplot2.

PPGEQ/UERJ

Gráfico de Pontos

```
# Criar um data frame
type.frame<-data.frame(table(Cars93$Type))
# Nomear as colunas
colnames(type.frame)<-c("Type","Frequency")
type.frame
# Criar o gráfico de pontos
ggplot(type.frame, aes(x = Frequency,
                       y = reorder(Type, Frequency))) +
  geom_point(size=4) +
  theme_bw() +
  theme(panel.grid.major.x = element_blank(),
        panel.grid.major.y = element_line(color = "black",
                                          linetype = "dotted")) +
  labs(y="Type")
```

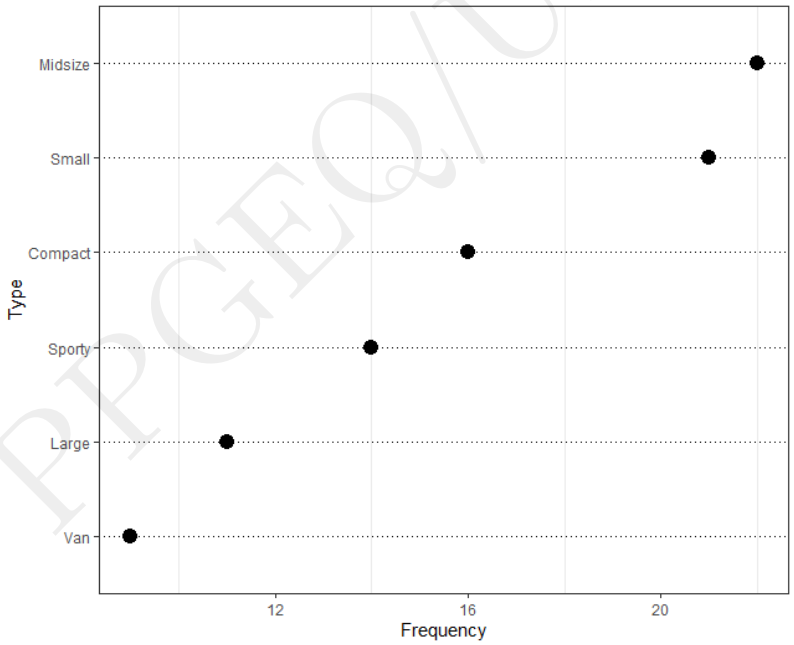


Figura 14: Gráfico de pontos com ggplot2.

PPGEQ/UERJ

Matriz do diagrama de dispersão

O pacote `GGally` permite a aplicação da Análise Exploratória de Dados (AED) no conjunto, gerando informações como a correlação linear e a significância estatística, além de gerar gráficos de de densidade, de dispersão, de barras.

```
# Requisição do pacote:
library(GGally)
# Criação de um subconjunto de Cars93
cars.subset <- subset(Cars93, select = c(MPG.city, Price, Horsepower))
# Criação da matriz do diagrama de dispersão (fig. 15)
ggpairs(cars.subset)
```

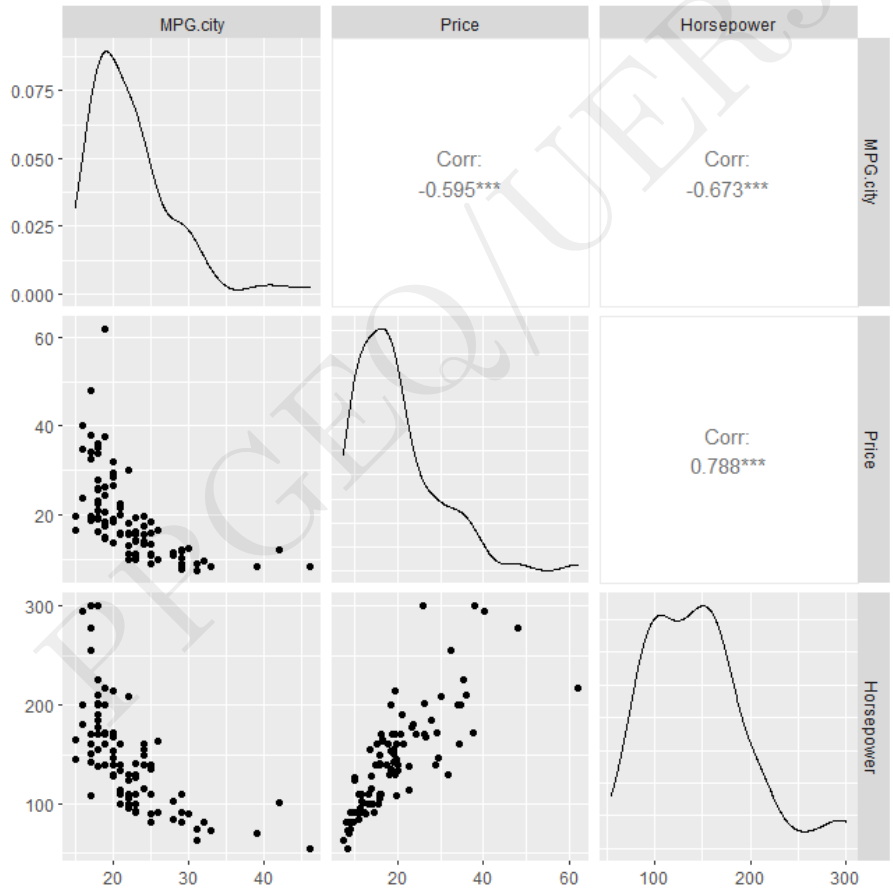


Figura 15: AED com GGally.

PPGEQ/UERJ

Note que, ao adicionar a variável **Cylinders** (uma variável categórica na estrutura de um objeto `factor`), é gerado também um histograma no gráfico de AED.

```
# Adição de Cylinders a cars.subset:
cars.subset <- subset(Cars93, select = c(MPG.city, Price, Horsepower, Cylinders))
# Gráfico AED  (fig. 16)
ggpairs(cars.subset)
```

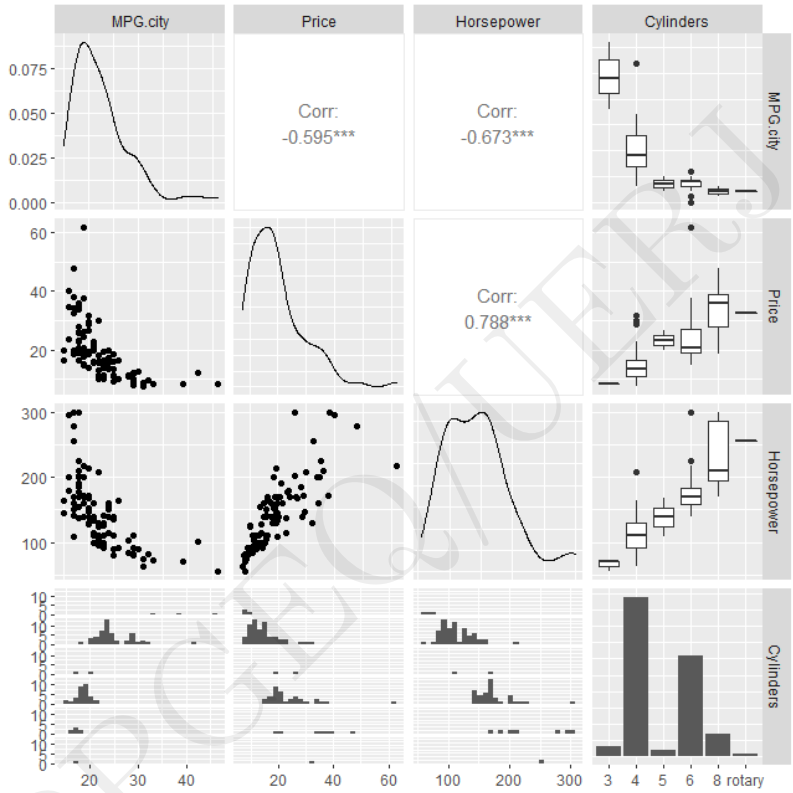


Figura 16: AED com GGally.

PPGEO/UERJ

Diagrama de caixa

```
# Gráfico de caixas, com o pacote ggplot2:
ggplot(Cars93, aes(x = Cylinders, y = Horsepower)) +
  geom_boxplot()
```

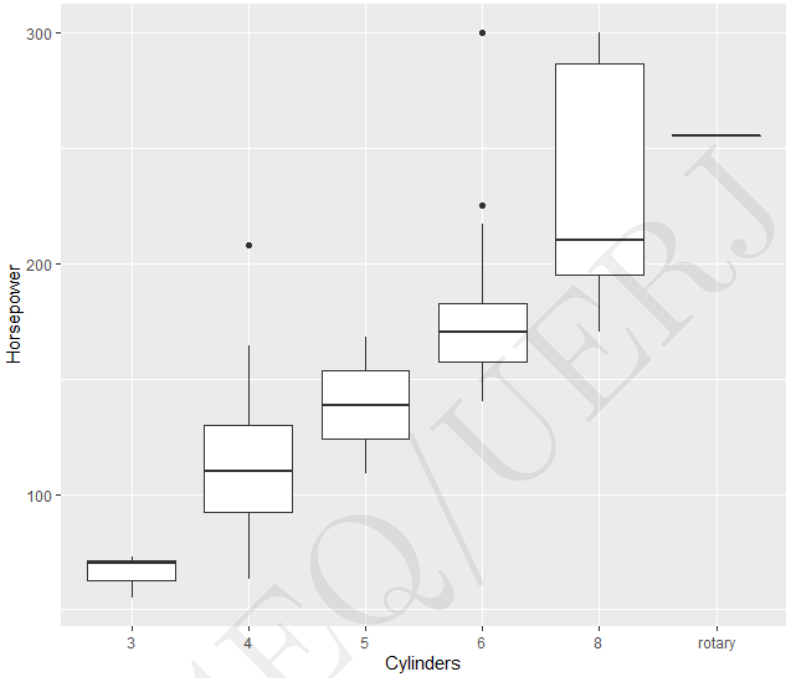


Figura 17: Gráfico de caixas com ggplot2.

PPGEQ/UERJ

```
# Gráfico de caixas, com distribuição dos dados (fig. 18):
ggplot(Cars93, aes(x = Cylinders, y = Horsepower)) +
  geom_boxplot() +
  geom_point()
```

```
# Problema: overplotting (fig. 19)
ggplot(Cars93, aes(x = Cylinders, y = Horsepower)) +
  geom_boxplot() +
  geom_point() +
  geom_jitter()
```

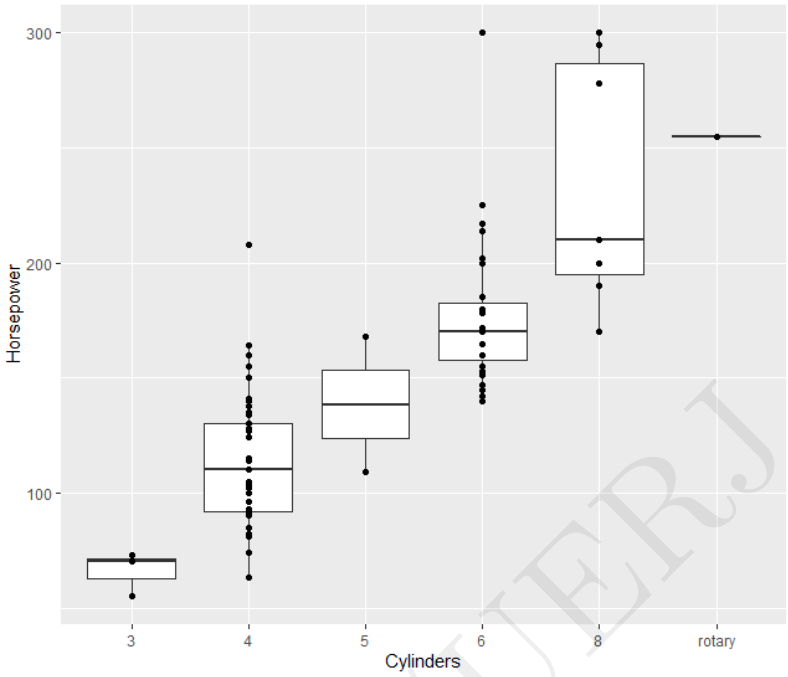


Figura 18: Gráfico de caixas com ggplot2: distribuição dos dados.

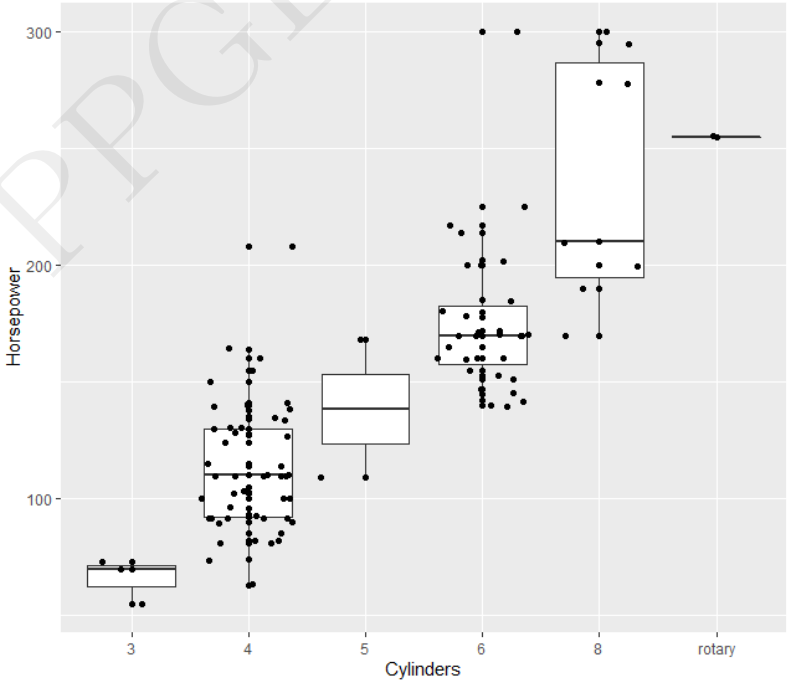


Figura 19: Gráfico de caixas com ggplot2: overplotting.

PPGEQ/UERJ

Medidas de Centralidade

Média aritmética:

```
# Criar um vetor das alturas das crianças
heights <- c(36, 42, 43, 37, 40, 45)
# função mean()
mean(heights)
```

Atividade: Calcular a média de uma variável em um data frame existente.

```
# Escolha de uma variável, na base de dados Cars93
Horsepower.USA <- Cars93$Horsepower[Cars93$Origin == "USA"]
# Média aritmética
mean(Horsepower.USA)
```

```
## [1] 147.5208
```

Atividade: Calcular a média da potência dos carros fabricados fora dos EUA.

```
# Escolha de uma variável, na base de dados Cars93
Horsepower.NonUSA <- Cars93$Horsepower[Cars93$Origin == "non-USA"]
# Média aritmética
mean(Horsepower.NonUSA)
```

```
## [1] 139.8889
```

Emprego da função with

Atividade: Calcular a média de uma variável em um data frame existente

```
# Uso da função 'with'
with(Cars93, mean(Horsepower[Origin == "USA"]))
```

Atividade: Qual a potência média dos carros fabricados nos EUA de 4 cilindros?

```
# Outro exemplo com a função 'with'
with(Cars93, mean(Horsepower[Origin=="USA" & Cylinders == 4]))
```

```
## [1] 104.0909
```

Atividade: Crie histogramas da potência dos carros em função da origem. Sugestão: Use o recurso da facetagem (Divide os dados de acordo com uma variável nominal - neste caso *Origin: USA Origin: non-USA*).

PPGEQ/UERJ

```r
# Histograma com ggpltot2
ggplot(Cars93, aes(x = Horsepower)) +
  geom_histogram(color = "black",
                 fill = "white",
                 bindwidth = 10) +
  facet_wrap( ~ Origin)
```

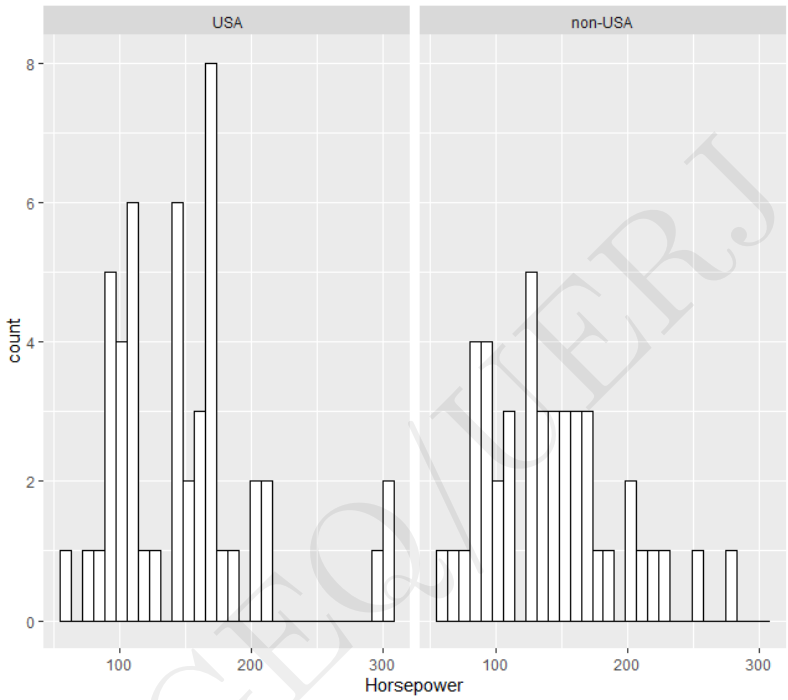


Figura 20: Histograma com ggplot2: Uso da função facet_wrap().

PPGEQ/UERJ

Média aparada ou truncada (trimmed mean).

Excluindo-se 5% dos dados extremos. Utiliza-se a função `mean`, com a habilitação do argumento `trim`.

```
# Média aparada
mean(Horsepower.USA, trim = 0.05)
```

```
## [1] 144.1818
```

Mediana

Lembre-se: é o valor do meio de um grande grupo de números.

Atividade: Considere a velocidade de leitura (em palavras por min) de um grupo de crianças.

```
# Base de dados
reading.speeds <- c(56, 78, 45, 49, 55, 62)
# média
mean(reading.speeds)
```

```
## [1] 57.5
```

Suponha que a criança que lê 78 palavras/min seja substituída por um adulto que lê 180 palavras/min.

```
# substituição:
reading.speeds.new <- replace(reading.speeds,
                              reading.speeds == 78,
                              values = 180)
mean(reading.speeds.new)
```

```
## [1] 74.5
```

```
# Organize os números em ordem ascendente
sort(reading.speeds)
```

```
## [1] 45 49 55 56 62 78
```

```
sort(reading.speeds.new)
```

```
## [1]  45  49  55  56  62 180
```

PPGEQ/UERJ

```r
# Mediana
median(reading.speeds)
```

```
## [1] 55.5
```

```r
median(reading.speeds.new)
```

```
## [1] 55.5
```

Moda

O pacote `stats`, que vem na instalação do R, não possui função para obter a moda.

```r
# Requisição do pacote
library(modeest)
# Criar um vetor dos escores
scores <- c(1, 2, 2, 2, 3, 4, 4, 4, 5, 6)
# Função mfv = "most frequent value" ou valor mais frequente
mfv(scores)
```

```
## [1] 2 4
```

Medidas de Dispersão

Variância Amostral usando a função var()

```r
# Criar um vetor de dados
heights <- c(50, 47, 52, 46, 45)
var(heights)
```

```
## [1] 8.5
```

A variância amostral é estimada com N-1 graus de liberdade no denominador.

$$s^2 = \frac{\sum_{i=1}^{N} (x_i - \bar{x})^2}{N-1} \tag{1}$$

Para calcular a variância populacional é preciso multiplicar pelo fator de correção $(N-1)/N$.

$$\sigma^2 = \frac{\sum_{i=1}^{N} (x_i - \bar{x})^2}{N-1} \times \frac{(N-1)}{N} \tag{2}$$

PPGEQ/UERJ

```r
# Usando length
var(heights)*(length(heights)-1)/length(heights)
```

```
## [1] 6.8
```

```r
# Neste caso, é preferível definir uma função var.p()
var.p = function(x){var(x)*(length(x)-1)/length(x)}
var.p(heights)
```

```
## [1] 6.8
```

Desvio-padrão

$$s = \sqrt{\frac{\sum_{i=1}^{N}(x_i - \bar{x})^2}{N-1}} \tag{3}$$

```r
# Desvio padrão amostral
sd(heights)
```

```
## [1] 2.915476
```

```r
# Desvio padrão populacional
sd(heights)*sqrt((length(heights)-1)/length(heights))
```

```
## [1] 2.607681
```

```r
# De forma similar, podemos criar uma função para o desvio padrão populacional
sd.p=function(x){sd(x)*sqrt((length(x)-1)/length(x))}
sd.p(heights)
```

```
## [1] 2.607681
```

Podemos também obter a estimativa da variância em um objeto da classe `data.frame`. Neste caso, podemos usar a função `with`.

```r
# Estimativa da variância de uma variável no data frame usando a função with
with(Cars93, var(Horsepower[Origin == "USA"]))
```

```
## [1] 2965.319
```

PPGEQ/UERJ

```r
# Número de carros deste grupo
with(Cars93, length(Horsepower[Origin == "USA"]))
```

```
## [1] 48
```

```r
# Para carros estrangeiros, isto é Non-USA
with(Cars93, var(Horsepower[Origin == "non-USA"]))
```

```
## [1] 2537.283
```

```r
with(Cars93, length(Horsepower[Origin == "non-USA"]))
```

```
## [1] 45
```

Padronização

No R, a função `scale()` é usada para padronização.

$$z_i = \frac{x_i - \bar{x}}{s} \tag{4}$$

```r
# Criar um vetor de potências de carros USA de 8 cilindros
Horsepower.USA.Eight <- with(Cars93,
                             Horsepower[Origin == "USA" & Cylinders == 8])
Horsepower.USA.Eight
```

```
## [1] 200 295 170 300 190 210
```

```r
# Padronizar o vetor de potências
scale(Horsepower.USA.Eight)
```

```
##            [,1]
## [1,] -0.4925263
## [2,]  1.2089283
## [3,] -1.0298278
## [4,]  1.2984785
## [5,] -0.6716268
## [6,] -0.3134259
## attr(,"scaled:center")
## [1] 227.5
## attr(,"scaled:scale")
## [1] 55.83458
```

PPGEQ/UERJ

Percentil

Considere o vetor de potências de carros USA de 8 cilindros, da base de dados **Cars93**.

```
# Ordem crescente:
sort(Horsepower.USA.Eight)
```

```
## [1] 170 190 200 210 295 300
```

```
# Obtenha os percentis
quantile(Horsepower.USA.Eight)
```

```
##     0%    25%    50%    75%   100%
## 170.00 192.50 205.00 273.75 300.00
```

Função summary

Retorna o resumo de cinco número de Tukey, junto com a média.

```
summary(Horsepower.USA.Eight)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   170.0   192.5   205.0   227.5   273.8   300.0
```

Quadro resumido

A base de dados **Cars93** contém dados de 27 variávies e 93 carros de 1993. Como visto, é um objeto da classe `data.frame`.

Note que, algumas funções retornam a mesma resposta:

```
# O que ocorre quando se aplica o length() no data frame
length(Cars93)
```

```
## [1] 27
```

```
# A função ncol () faz a mesma coisa.
ncol(Cars93)
```

```
## [1] 27
```

PPGEQ/UERJ

```r
# função nrow () indica o número de linhas.
nrow(Cars93)
```

```
## [1] 93
```

```r
# Para somar o número de carros originados dos EUA, declare a condição.
sum(Cars93$Origin == "USA")
```

```
## [1] 48
```

```r
# Para somar o número de carros estrangeiros (non-USA)
# Operador != (significa diferente de)
sum(Cars93$Origin != "USA")
```

```
## [1] 45
```

```r
# De forma alternativa
sum(Cars93$Origin == "non-USA")
```

```
## [1] 45
```

```r
# Para somar o número de carros dos EUA com 4 cilindros
sum(Cars93$Origin == "USA" & Cars93$Cylinders == 4)
```

```
## [1] 22
```

```r
# De forma alternativa, sem cifrões
with(Cars93, sum(Origin == "USA"& Cylinders == 4))
```

```
## [1] 22
```

```r
# Para achar o número de elementos de um vetor
Horsepower.USA.Four <- Cars93$Horsepower[Cars93$Origin == "USA" &
                                          Cars93$Cylinders == 4]
# Valores máximo e mínimo
max(Horsepower.USA.Four)
```

```
## [1] 155
```

PPGEQ/UERJ

```r
min(Horsepower.USA.Four)
```

```
## [1] 63
```

```r
# Amplitude
range(Horsepower.USA.Four)
```

```
## [1]  63 155
```

Momentos em Estatística Descritiva

Podemos construir uma função para calcular o momento central de um vetor. Nesta função, considere que o argumento x é o vetor de dados e o argumento y é a ordem (segundo, terceiro, ...):

```r
# Criação da função:
cen.mom <- function(x, y){mean(x)^y}
# Considere o seguinte vetor:
Horsepower.USA <- Cars93$Horsepower[Cars93$Origin == "USA"]
# Calcular o primeiro momento central:
cen.mom(Horsepower.USA, 1)
```

```
## [1] 147.5208
```

```r
# Calcular o segundo momento central:
cen.mom(Horsepower.USA, 2)
```

```
## [1] 21762.4
```

```r
# Calcular o terceiro momento central:
cen.mom(Horsepower.USA, 3)
```

```
## [1] 3210407
```

```r
# Calcular o quarto momento central:
cen.mom(Horsepower.USA, 4)
```

```
## [1] 473601891
```

onde:

- Primeiro momento central = média do histograma.
- Segundo momento central = dispersão do histograma.
- Terceiro momento central = associado a assimetria.
- Quarto momento central = associado a curtose.

PPGEQ/UERJ

Definição de assimetria

A assimetria de um conjunto de números é o terceiro momento central dividido pelo segundo momento central elevado à potência de três meios.

```
# Cálculo manual:
cen.mom(Horsepower.USA,3)/cen.mom(Horsepower.USA,2)^1.5
```

```
## [1] 1
```

```
# Usando pacote 'moments'
library(moments)
# Assimetria (skewness):
skewness(Horsepower.USA)
```

```
## [1] 1.133031
```

```
# Comparando com os carros estrangeiros
Horsepower.NonUSA <- Cars93$Horsepower[Cars93$Origin == "non-USA"]
skewness(Horsepower.NonUSA)
```

```
## [1] 0.642995
```

```
# A assimetria é positiva sendo superior para os carros USA
# Verifique nos histogramas gerados
ggplot(Cars93, aes(x = Horsepower)) +
  geom_histogram(color = "black",
                 fill = "white",
                 binwidth = 10) +
  facet_wrap( ~ Origin)
# Para gerar os gráficos de densidade de probabilidade
ggplot(Cars93, aes(x = Horsepower)) +
  geom_density() +
  facet_wrap( ~ Origin)
```

PPGEQ/UERJ

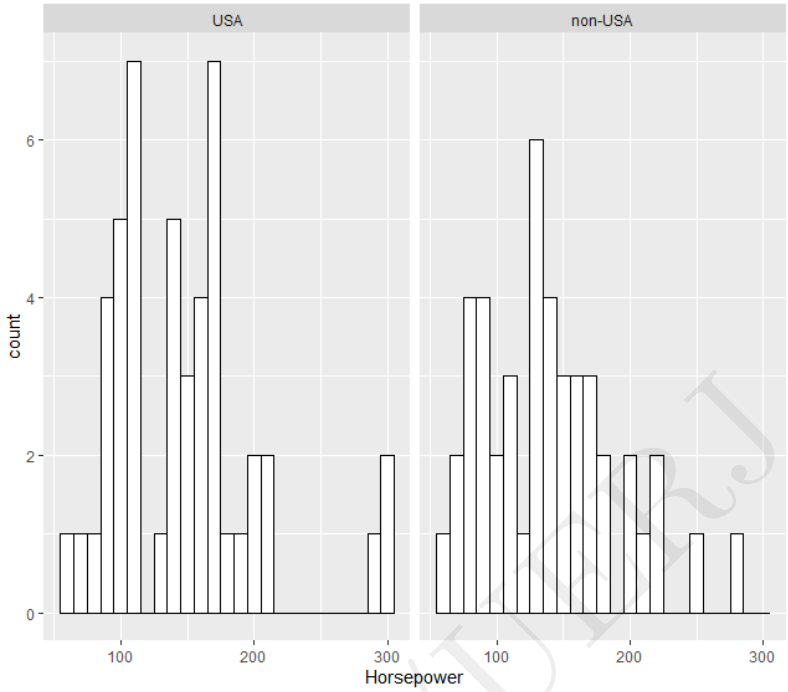


Figura 21: Visualização da assimetria.

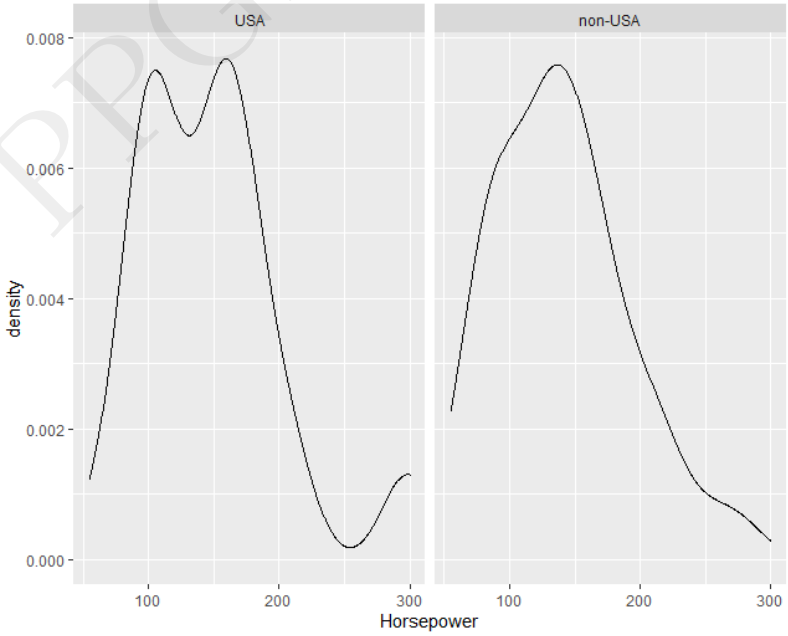


Figura 22: Gráfico de densidade.

PPGEQ/UFRJ

Curtose

A curtose é obtida pela divisão do quarto momento central pelo quadrado do segundo momento centrtal.

Muitos estatísticos subtraem 3 do resultado da fórmula da curtose.

É chamado de excesso de curtose.

```
cen.mom(Horsepower.USA,4)/cen.mom(Horsepower.USA,2)^2
```

```
## [1] 1
```

```
# De forma alternativa, usando a função kurtosis () no pacote moments.
library(moments)
kurtosis(Horsepower.USA)
```

```
## [1] 4.403952
```

```
kurtosis(Horsepower.NonUSA)
```

```
## [1] 3.097339
```

A cauda mais espessa do lado esquerdo do gráfico da densidade de probabilidade sugere que os carros USA têm uma curtose maior do que os carros estrangeiros.

Frequência dos dados

Variáveis nominais

```
# Usar a função table()
car.type <- table(Cars93$Type)
car.type
```

```
##
## Compact   Large Midsize   Small  Sporty     Van
##      16      11      22      21      14       9
```

```
# Somar as frequências
margin.table(car.type)
```

```
## [1] 93
```

PPGEQ/UERJ

```r
# Apresentar as frequências na forma de proporção com duas decimais
round(prop.table(car.type), 2)
```

```
##
## Compact   Large Midsize   Small  Sporty     Van
##    0.17    0.12    0.24    0.23    0.15    0.10
```

Variáveis numéricas

```r
# Para criar uma tabela de frequência, usa-se a função hist()
prices <- hist(Cars93$Price,
               plot = F,
               breaks = 5)
prices
```

```
## $breaks
## [1]  0 10 20 30 40 50 60 70
##
## $counts
## [1] 12 50 19  9  2  0  1
##
## $density
## [1] 0.012903226 0.053763441 0.020430108 0.009677419 0.002150538 0.000000000
## [7] 0.001075269
##
## $mids
## [1]  5 15 25 35 45 55 65
##
## $xname
## [1] "Cars93$Price"
##
## $equidist
## [1] TRUE
##
## attr(,"class")
## [1] "histogram"
```

```r
# Criar uma matriz de fequências
prices.matrix <- matrix(c(prices$mids,
                          prices$counts),
                        ncol = 2)
prices.frame <- data.frame(prices.matrix)
colnames(prices.frame) <- c("Price Midpoint(X$1,000)", "Frequency")
prices.frame
```

PPGEQ/UERJ

```
##   Price Midpoint(X$1,000) Frequency
## 1                      5        12
## 2                     15        50
## 3                     25        19
## 4                     35         9
## 5                     45         2
## 6                     55         0
## 7                     65         1
```

Frequência cumulativa

```r
# Usando a função cumsum()
prices$counts
```

```
## [1] 12 50 19  9  2  0  1
```

```r
cumsum(prices$counts)
```

```
## [1] 12 62 81 90 92 92 93
```

Para obter o histograma de frequência cumulativa, substitua o vetor original pelo vetor de frequências cumulativas

```r
# Soma cumulativa:
prices$counts <- cumsum(prices$counts)
```

```r
# Construção do gráfico (fig. 23)
plot(prices, main = "Cumulative Histogram",
     xlab = "Price",
     ylab = "Cumulative Frequency",
     ylim=c(0,120),
     col = "AliceBlue")
```

```r
# Função de Distribuição Cumulativa Empírica, ecdf (fig. 24)
plot(ecdf(Cars93$Price),
     xlab = "Price",
     ylab = "Fn Price")
```

PPGEQ/UERJ

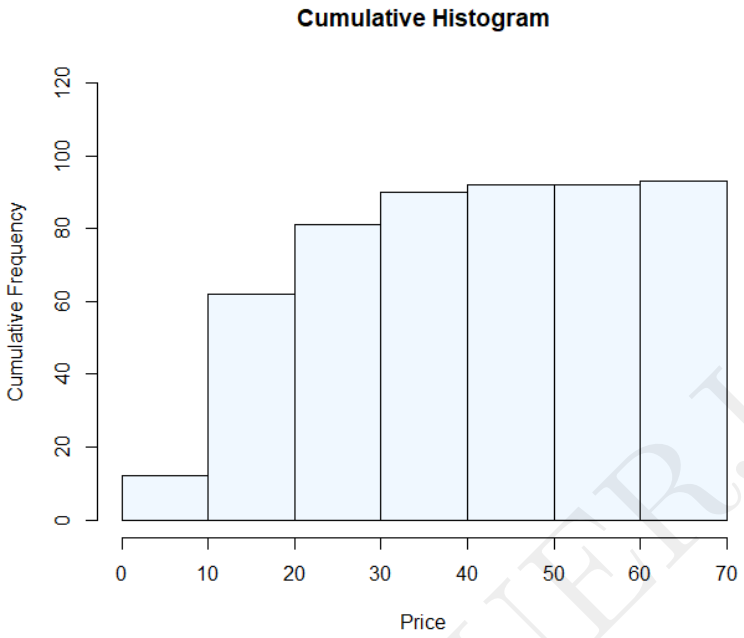


Figura 23: Gráfico de frequência acumulada.

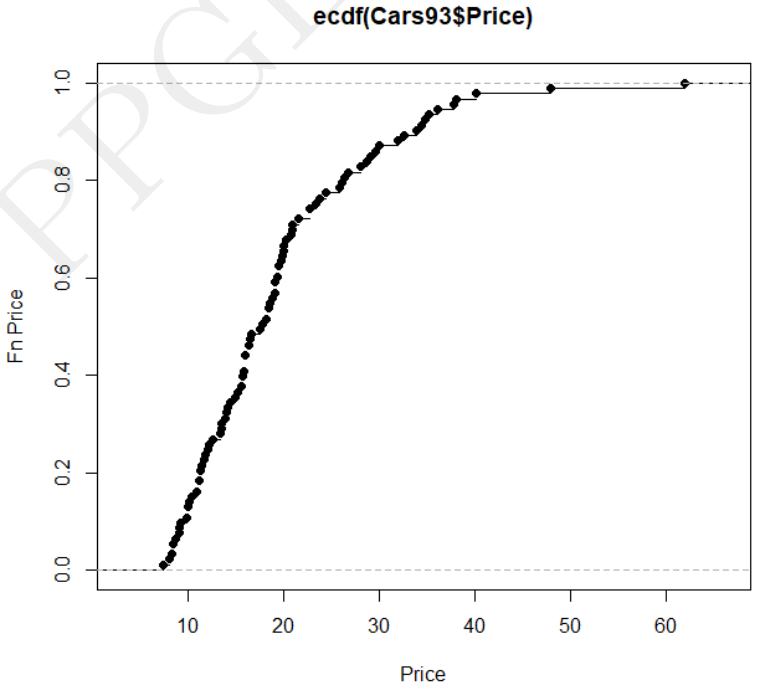


Figura 24: Gráfico de distribuição empírica acumulada.

PPGEQ/UERJ

```r
# Usando ggplot2
library(ggplot2)
ggplot(NULL, aes(x = Cars93$Price)) +
  geom_step(stat = "ecdf") +
  labs(x = "Price X $1,000", y = "Fn (Price)")
```

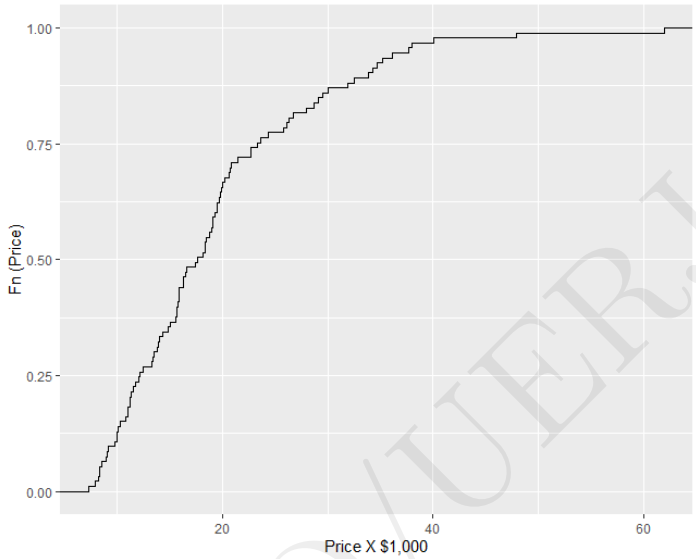


Figura 25: Gráfico de distribuição empírica acumulada, com ggplot2.

PPGEQ/UERJ

```r
# Adicione linhas de cada quartil
price.q <- quantile(Cars93$Price)
# Gráfico
ggplot(NULL, aes(x = Cars93$Price)) +
  geom_step(stat = "ecdf") + labs(x = "Price X $1,000", y = "Fn (Price)") +
  geom_vline(aes(xintercept = price.q), linetype = "dashed")
```

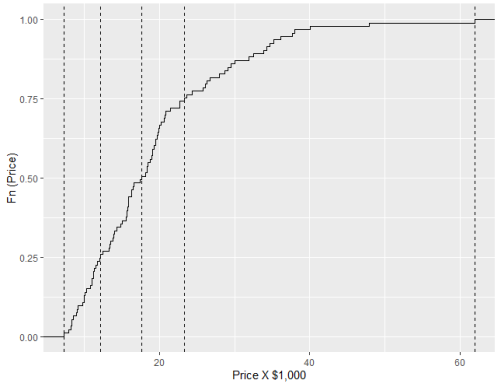


Figura 26: Gráfico de distribuição empírica acumulada, com linhas de cada quartil, no ggplot2.

Para adicionar os valores dos quartis no eixo x, usa-se a função `scale_x_continuous()`

```
# gráfico:
ggplot(NULL, aes(x = Cars93$Price)) +
  geom_step(stat = "ecdf") +
  labs(x = "Price X $1,000", y = "Fn (Price)") +
  geom_vline(aes(xintercept = price.q), linetype = "dashed") +
  scale_x_continuous(breaks = price.q, labels = price.q)
```

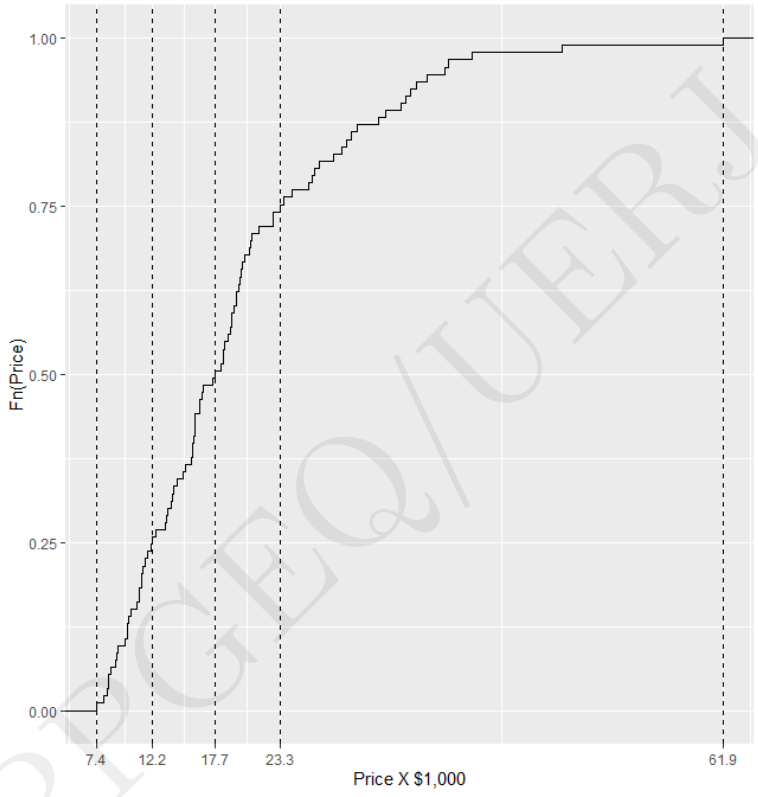


Figura 27: Função distribuição cumulativa empírica.

PPGEQ/UERJ

Variáveis numéricas: stem (diagrama de ramo e folhas)

Exemplo: os preços dos carros em **Cars93**, organizados em ordem ascendente e arredondado para o número inteiro mais próximo.

```
rounded <- (round(sort(Cars93$Price),0))
# Para exibir os valores arredondados, usa-se a função cat().
cat(rounded, fill = 50)
```

```
## 7 8 8 8 8 9 9 9 9 10 10 10 10 10 11 11 11 11 11
## 11 12 12 12 12 12 13 13 14 14 14 14 14 15 15 16
## 16 16 16 16 16 16 16 16 16 17 18 18 18 18 18 18
## 19 19 19 19 19 20 20 20 20 20 20 20 21 21 21 22
## 23 23 23 24 24 26 26 26 27 28 29 29 30 30 32 32
## 34 34 35 35 36 38 38 40 48 62
```

```
# Usando a função stem
stem(Cars93$Price)
```

```
##
##   The decimal point is 1 digit(s) to the right of the |
##
##   0 | 788889999
##   1 | 00000111111222233344444556666666667788888999999
##   2 | 00000001112333446667899
##   3 | 00234455688
##   4 | 08
##   5 |
##   6 | 2
```

Resumo de uma data frame

```
autos <- subset(Cars93,
                select = c(MPG.city, Type, Cylinders, Price, Horsepower))
summary(autos)
```

```
##     MPG.city          Type     Cylinders      Price         Horsepower
##  Min.   :15.00   Compact:16   3     : 3   Min.   : 7.40   Min.   : 55.0
##  1st Qu.:18.00   Large  :11   4     :49   1st Qu.:12.20   1st Qu.:103.0
##  Median :21.00   Midsize:22   5     : 2   Median :17.70   Median :140.0
##  Mean   :22.37   Small  :21   6     :31   Mean   :19.51   Mean   :143.8
##  3rd Qu.:25.00   Sporty :14   8     : 7   3rd Qu.:23.30   3rd Qu.:170.0
##  Max.   :46.00   Van    : 9   rotary: 1   Max.   :61.90   Max.   :300.0
```

PPGEQ/UERJ

Distribuições de probabilidades

Distribuição Normal

A função de densidade de probabilidade (fdp) para a distribuição normal é

$$f\left(X|\mu,\sigma^2\right) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot e^{-(1/2)[(X-\mu)/\sigma]^2} \tag{5}$$

onde:

▷ e = a constante matemática aproximada por 2,71828...

▷ π = a constante matemática aproximada por 3,14159...

▷ μ = a média aritmética

▷ σ = o desvio-padrão

▷ X = qualquer valor correspondente à variável contínua, para o qual $-\infty < X < \infty$.

PPGEQ/UERJ

Diagramando uma curva normal

```
#Criar uma sequência de 40 a 160 com intervalo de 1.
x.values <- seq(40, 160, 1)
# Criar um vetor de nove valores de desvio padrão para o intervalo de 40 a 160.
sd.values <- seq(40, 160, 15)
# Criar um terceiro vetor de zeros
zeros9 <- rep(0, 9)
```

Uma vez com a base de dados criada, podemos gerar o gráfico da função densidade de probabilidade da distribuição normal:

```
# Criar o gráfico da função de densidade de probabilidade
ggplot(NULL, aes(x = x.values,
                 y = dnorm(x.values,
                           m = 100,
                           s = 15))) +
  geom_line() +
  labs(x = "IQ", y = "f(IQ)")
```

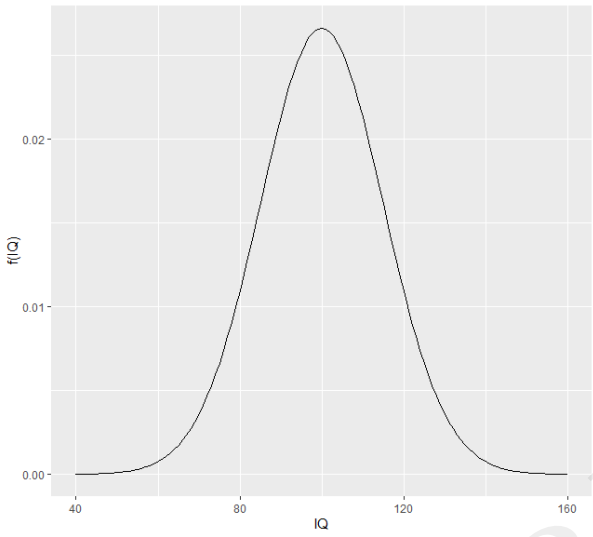


Figura 28: Distribuição normal (exemplo).

```
# Agora, vamos colocar os sd.values no eixo x.
ggplot(NULL, aes(x = x.values,
                 y = dnorm(x.values,
                           m = 100,
                           s = 15))) +
  geom_line() +
  labs(x = "IQ", y = "f(IQ)") +
  scale_x_continuous(breaks = sd.values,
                     labels = sd.values)
```

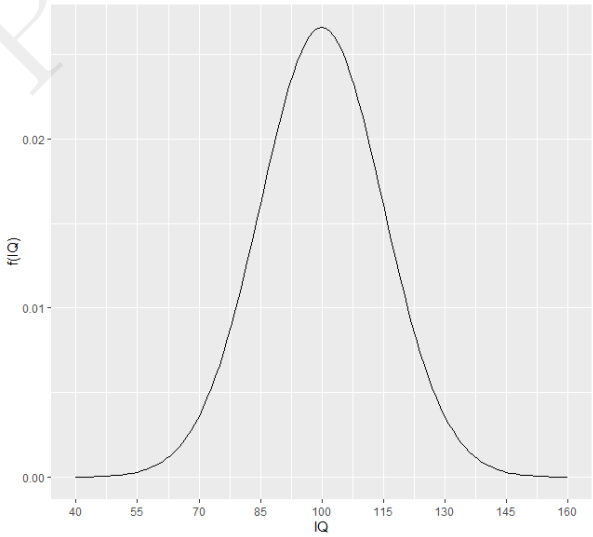


Figura 29: Distribuição normal (com os desvios-padrão).

PPGEQ/UERJ

```r
# Adicionando-se uma declaração sobre as linhas tracejadas
ggplot(NULL, aes(x = x.values,
                 y = dnorm(x.values,
                           m = 100,
                           s = 15))) +
  geom_line() +
  labs(x = "IQ", y = "f(IQ)") +
  scale_x_continuous(breaks = sd.values, labels = sd.values) +
  geom_segment((aes(x = sd.values,
                    y = zeros9,
                    xend = sd.values,
                    yend = dnorm(sd.values,
                                 m = 100,
                                 s = 15))),
               linetype = "dashed")
```

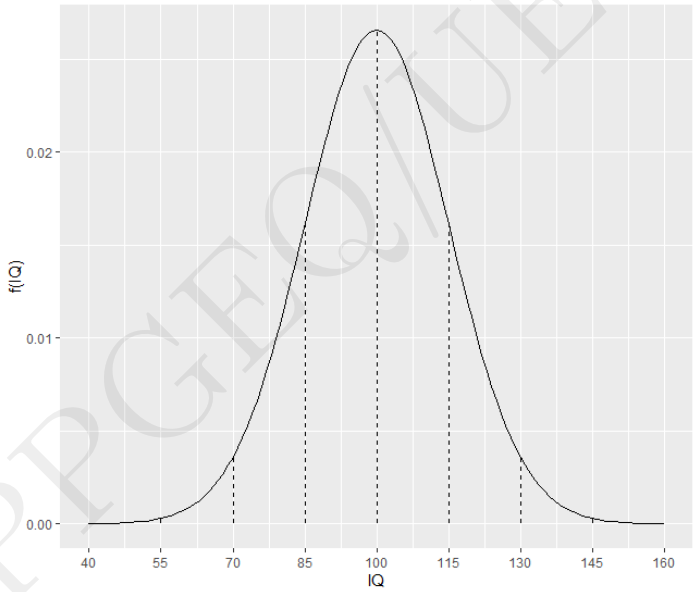


Figura 30: Distribuição normal (com os desvios-padrão e linhas verticais).

```r
# Alterações estéticas no gráfico
ggplot(NULL, aes(x = x.values,
                 y = dnorm(x.values,
                           m = 100,
                           s = 15))) +
  geom_line() +
  labs(x = "IQ", y = "f(IQ)") +
  scale_x_continuous(breaks = sd.values,
                     labels = sd.values) +
  geom_segment((aes(x = sd.values,
                    y = zeros9,
```

PPGEQ/UERJ

```
                    xend = sd.values,
                    yend = dnorm(sd.values,
                                 m = 100,
                                 s = 15))),
               linetype = "dashed") +
  scale_y_continuous(expand = c(0,0))
```

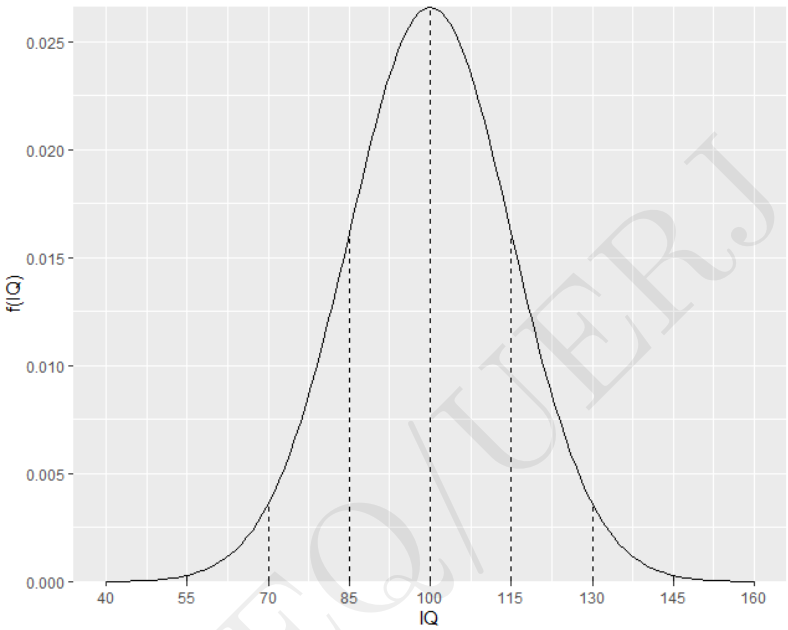


Figura 31: Gráfico de distribuição normal.

PPGEO/UFRJ

Função de densidade cumulativa

A função `pnorm(x, m, s)` retorna a probabilidade de uma pontuação ser menor do que x em distribuição com média m e desvio padrão s.

Atividade: Considere as seguintes questões:

a) Qual a probabilidade do número ser menor que 100 quando a média é igual a 100 e o desvio padrão é 15, ou seja, $P\left(X < 100 | \mu = 100, \sigma = 15\right) =?$

b) Qual a probabilidade do número ser menor que 85, ou seja, $P\left(X < 85 | \mu = 100, \sigma = 15\right) =?$

c) Qual a probabilidade do número ser maior que 85, ou seja, $P\left(X > 85 | \mu = 100, \sigma = 15\right) =?$ Lembre-se que $P\left(X > x\right) = 1 - P\left(X \leq x\right)$.

d) Qual a probalidade do valor estar entre 85 e 100, ou seja, $P\left(85 \leq X \leq 100 | \mu = 100, \sigma = 15\right) = ?$

```r
# letra a) P(X<100), com m = 100 e s = 15?
pnorm(100, 100, 15)
```

```
## [1] 0.5
```

```r
# letra b) P(X<85), com m = 100 e s = 15?
pnorm(85, 100, 15)
```

```
## [1] 0.1586553
```

```r
# letra c) P(X>85), com m = 100 e s = 15?
1-pnorm(85, 100, 15)
```

```
## [1] 0.8413447
```

```r
  # ou então usar o argumento lower.tail=FALSE:
pnorm(85, 100, 15, lower.tail = FALSE)
```

```
## [1] 0.8413447
```

```r
# letra d) P(85<X<100), com m=100 e s=15?
pnorm(100, 100, 15)-pnorm(85, 100, 15)
```

```
## [1] 0.3413447
```

Note que podemos resolver esta última questão de forma elegante, visualmente, com o pacote `tigerstats`:

```r
# Usando o pacote tigerstats
library(tigerstats)
pnormGC(c(85, 100),
        region = "between",
        m = 100,
        s = 15,
        graph = TRUE)
```

PPGEQ/UERJ

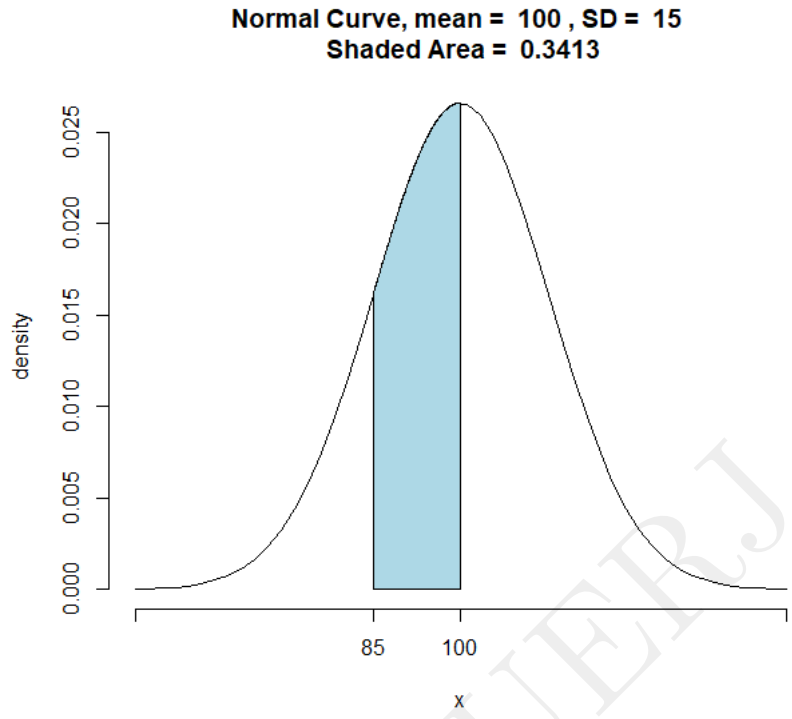


Figura 32: Distribuição Normal, com área sombreada em $P(85 \leq X \leq 100)$.

Diagramando a cdf (função de densidade cumulativa)

```
# Diagramando a cdf (função de densidade cumulativa)
ggplot(NULL, aes(x = x.values, y = pnorm(x.values, m = 100, s = 15))) +
  geom_line() +
  labs(x = "IQ", y = "f(IQ)") +
  scale_x_continuous(breaks = sd.values, labels = sd.values) +
  geom_segment((aes(x = sd.values,
                    y = zeros9,
                    xend = sd.values,
                    yend = dnorm(sd.values,
                                 m = 100,
                                 s = 15))),
               linetype = "dashed") +
  scale_y_continuous(expand = c(0, 0))
```

PPGEQ/UERJ

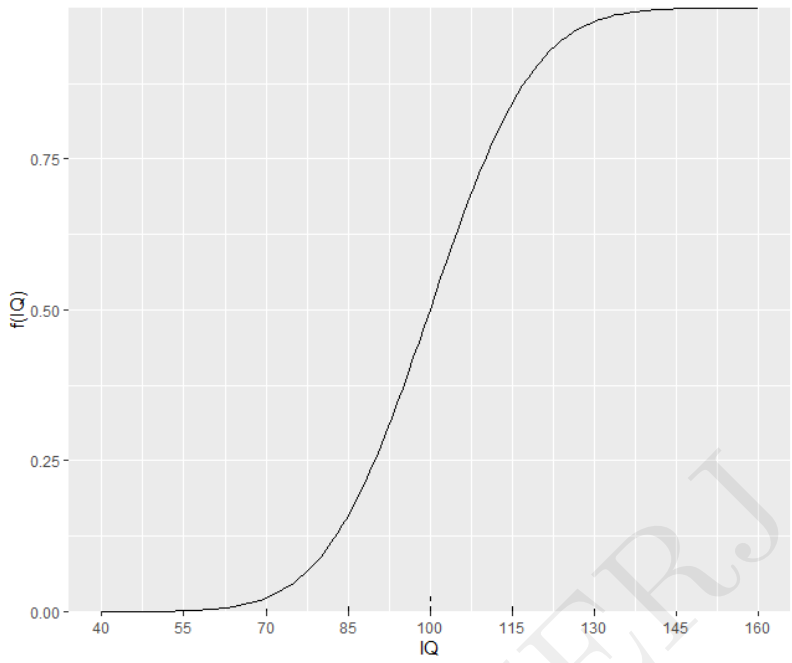


Figura 33: Função densidade cumulativa.

Quantis de distribuição normal

A função `qnorm()` é o inverso da `pnorm()`. Dada uma área, a `qnorm()` retornará a pontuação que corta aquela área (à esquerda) da distribuição normal especificada.

```
qnorm(0.1586553, m = 100, s = 15)
```

```
## [1] 85
```

```
# Para saber a pontuação que corta a área indicada à direita
qnorm(0.1586553, m = 100, s = 15, lower.tail = FALSE)
```

```
## [1] 115
```

Podemos visualizar usando o pacote `tigerstats`, modificando o argumento `region`, ajustando para `below`:

```
# Usando o pacote tigerstats
qnormGC(0.1586553,
        region = "below",
        m = 100,
        s = 15,
        graph = TRUE)
```

PPGEQ/UERJ

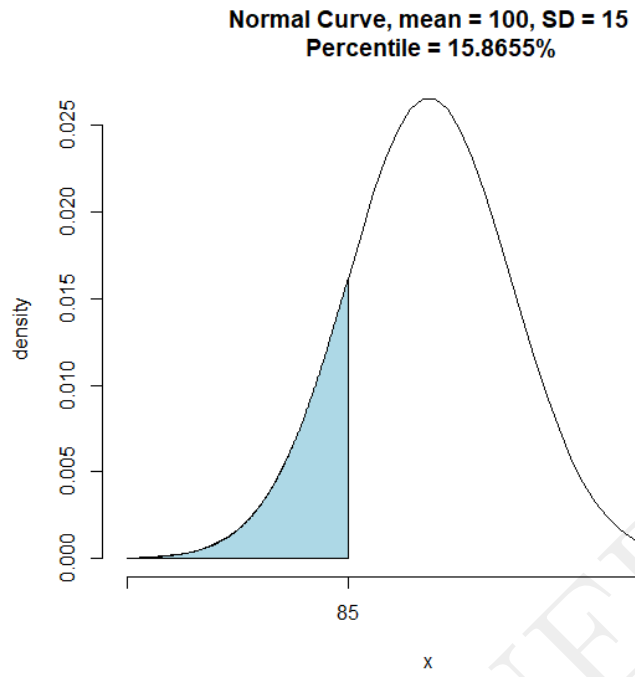


Figura 34: Distribuição Normal, com área sombreada em $P(85 < X)$.

Quantis

```
# Quantis
qnorm(c(0, 0.25, 0.50, 0.75, 1.00), m = 100, s = 15)
```

```
## [1]      -Inf  89.88265 100.00000 110.11735       Inf
```

```
# Fazendo arredondamento
round(qnorm(c(0, 0.25, 0.50, 0.75, 1.00), m = 100, s = 15))
```

```
## [1] -Inf   90  100  110  Inf
```

Diagramando a cdf com quartis

```
# Para substituir os valores do desvio padrão com os valores dos quantis,
# criam-se dois vetores.
q.values <- round(qnorm(c(0.25, 0.50, 0.75), m = 100, s = 15))
zeros3 <- c(0, 0, 0)
```

Agora, inserimos esses vetores no código:

PPGEQ/UERJ

```r
# Insere estes vetores no código do gráfico
ggplot(NULL, aes(x = x.values, y = pnorm(x.values, m = 100, s = 15))) +
  geom_line() +
  labs(x = "IQ", y = "f(IQ)") +
  scale_x_continuous(breaks = q.values, labels = q.values) +
  geom_segment((aes(x = q.values,
                    y = zeros3,
                    xend = q.values,
                    yend = pnorm(q.values,
                                 m = 100,
                                 s = 15))),
               linetype = "dashed") +
  scale_y_continuous(expand = c(0, 0))
```

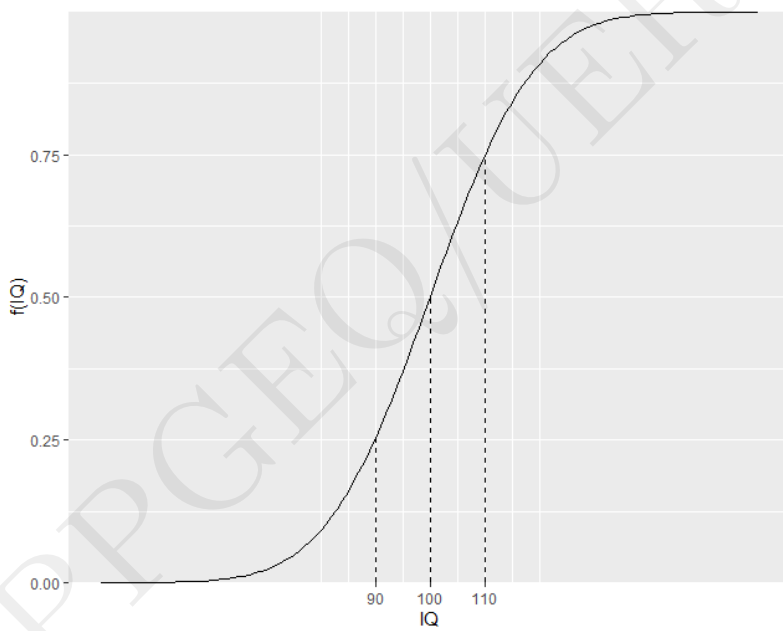


Figura 35: Gráfico da cdf com quartis.

PPGEQ/UERJ

Distribuição Binomial

A seguir, são mostradas as equações da função de probabilidade ($f(x)$), a média (μ), a variância (σ^2), a assimetria (α_3) e a curtose (α_4)[1].

$$f(x|p,n) = P(X = x) = \binom{n}{x} p^x q^{n-x} = \frac{n!}{x!\,(n-x)!} p^x q^{n-x} \sim B(n,p) \tag{6}$$

Média:

$$\mu = np \tag{7}$$

$$\sigma^2 = npq \tag{8}$$

$$\alpha_3 = \frac{q-p}{\sqrt{npq}} \tag{9}$$

$$\alpha_4 = 3 + \frac{1-6pq}{npq} \tag{10}$$

```r
# Tamanho da amostra = 50 e probabilidade = 0,45
success <- 0:50
plot(success, dbinom(success, size = 50, prob = 0.45),
     type = "h",
     main = "Binomial Distribution (n=50, p=0.45)",
     ylab = "Probability",
     xlab = "Successes",
     lwd = 3)
```

PPGEQ/UERJ

[1]Spiegel, m. R.; Schiller, J.; Srinivasan, A. Schaum's outlines: Probability and Statistics, 3rd ed, McGraw-Hill, 2009, p. 109

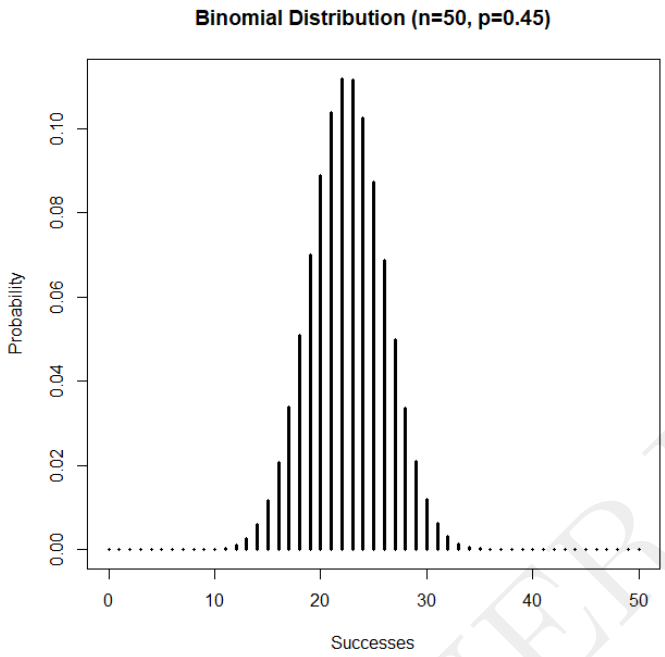


Figura 36: Distribuição Binomial.

Podemos também gerar o gráfico da distribuição Binomial usando a função `barplot`:

```
# Usando o barplot
prob <- c(0.5, 1/6, 0.6)
yval <- 0:1000
Y <- outer(yval, prob, dbinom, size = max(yval))  ## no `t()` now
barplot(Y, names.arg = paste0("p = ", format(prob, digits = 2)), beside = TRUE)
```

PPGEQ/UERJ

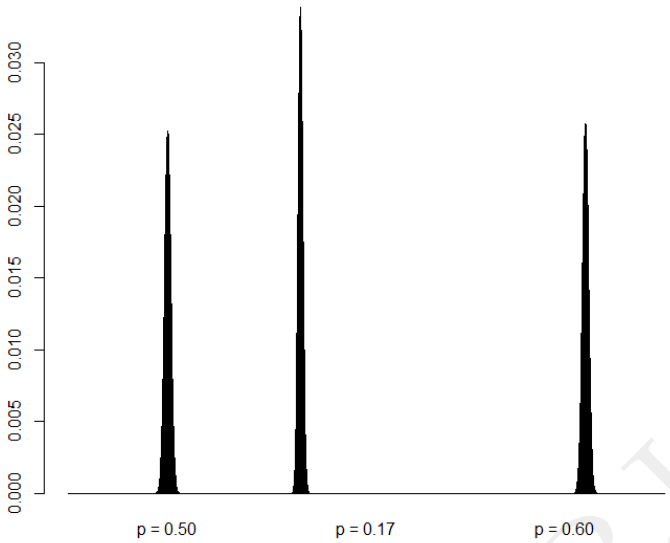


Figura 37: Distribuição Binomial.

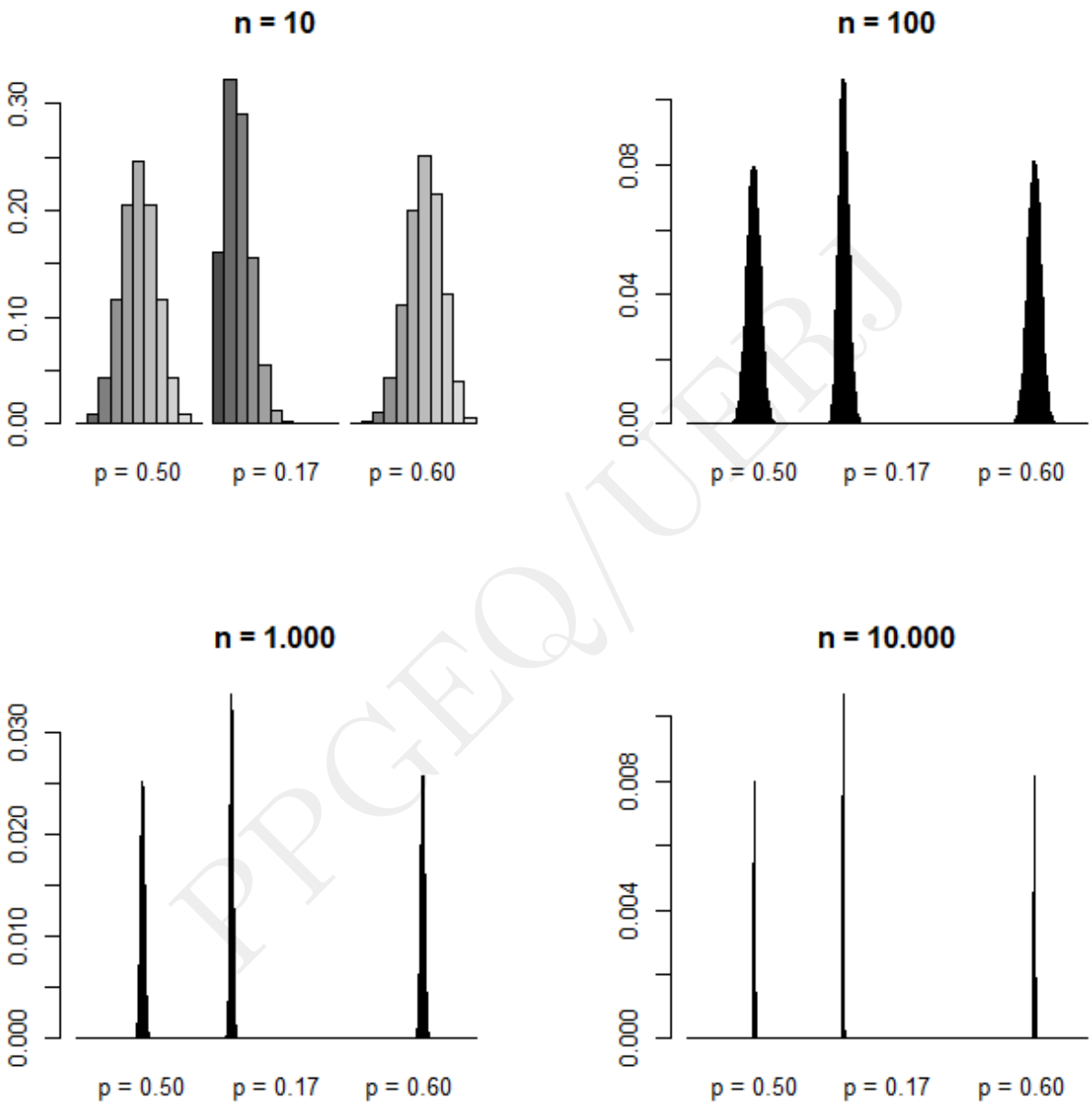
Influência do tamanho da amostra na Dist. Binomial

```
# Tamanho igual a 10
prob.1 <- c(0.5, 1/6, 0.6)
yval.1 <- 0:10
Y1 <- outer(yval.1, prob.1, dbinom, size = max(yval.1))
# Tamanho igual a 100
prob.2 <- c(0.5, 1/6, 0.6)
yval.2 <- 0:100
Y2 <- outer(yval.2, prob.2, dbinom, size = max(yval.2))  ## no `t()` now
# Tamanho igual a 1000
prob.3 <- c(0.5, 1/6, 0.6)
yval.3 <- 0:1000
Y3 <- outer(yval.3, prob.3, dbinom, size = max(yval.3))
# Tamanho igual a 10000
prob.4 <- c(0.5, 1/6, 0.6)
yval.4 <- 0:10000
Y4 <- outer(yval.4, prob.4, dbinom, size = max(yval.4))
```

```
# Gráficos
par(mfrow = c(2, 2))
barplot(Y1, names.arg = paste0("p = ", format(prob.1, digits = 2)),
        beside = TRUE, main = "n = 10")
barplot(Y2, names.arg = paste0("p = ", format(prob.2, digits = 2)),
        beside = TRUE, main = "n = 100")
barplot(Y3, names.arg = paste0("p = ", format(prob.3, digits = 2)),
        beside = TRUE, main = "n = 1.000")
```

PPGEQ/UERJ

```r
barplot(Y4, names.arg = paste0("p = ", format(prob.4, digits = 2)),
        beside = TRUE, main = "n = 10.000")
```



PPGEQ/UFRJ

Figura 38: Distribuição Binomial.

Distribuição de Poisson

A seguir, são mostradas as equações da função probabilidade, da média, da variância, assimetria e curtose [2]

$$f(x) = P(X = x) = \frac{\lambda^x e^{-\lambda}}{x!} \tag{11}$$

$$\mu = \lambda \tag{12}$$

$$\sigma^2 = \lambda \tag{13}$$

$$\alpha_3 = \frac{1}{\sqrt{\lambda}} \tag{14}$$

$$\alpha_4 = 3 + \left(\frac{1}{\lambda}\right) \tag{15}$$

```
# Tamanho da amostra = 20 e lambda = 1
plot(dpois(x = 0:20, lambda = 1), type = "b")
```

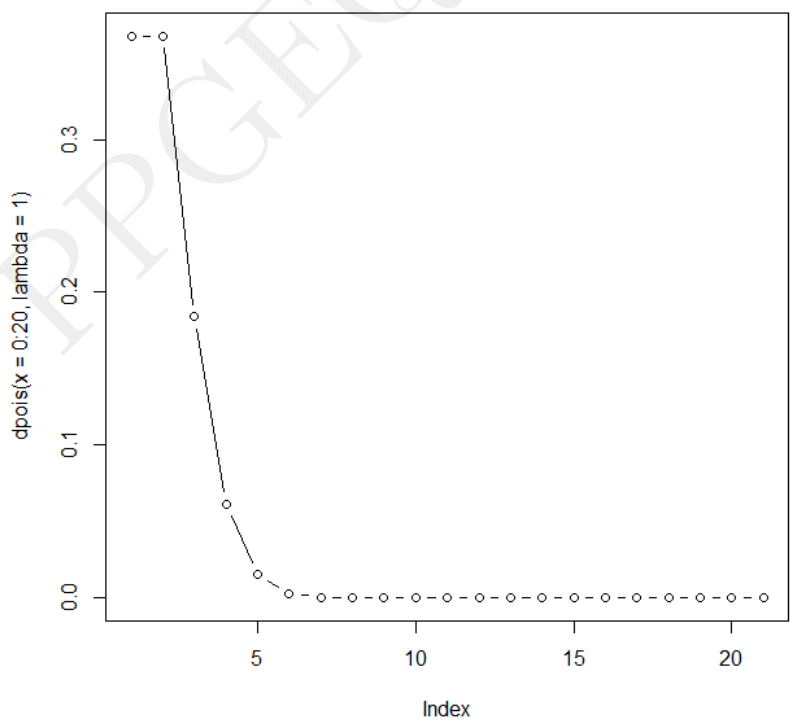


Figura 39: Distribuição de Poisson.

[2] Spiegel, m. R.; Schiller, J.; Srinivasan, A. Schaum's outlines: Probability and Statistics, 3rd ed, McGraw-Hill, 2009, p. 111

PPGEQ/UERJ

```r
# Qual a influência de lambda no formato da curva?
plot(dpois( x=0:20, lambda=2 ), type="b")
```

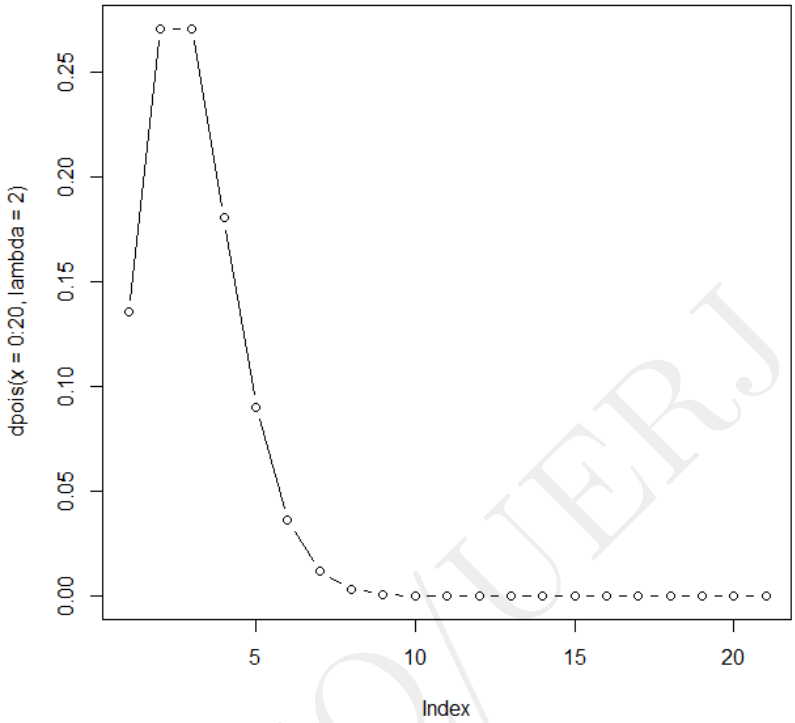


Figura 40: Distribuição de Poisson.

```r
# Colocando os gráficos na mesma janela
par(mfrow=c(1,2))
plot( dpois( x=0:20, lambda=1 ), type="b")
plot( dpois( x=0:20, lambda=2 ), type="b")
dev.off()
```

PPGEQ/UERJ

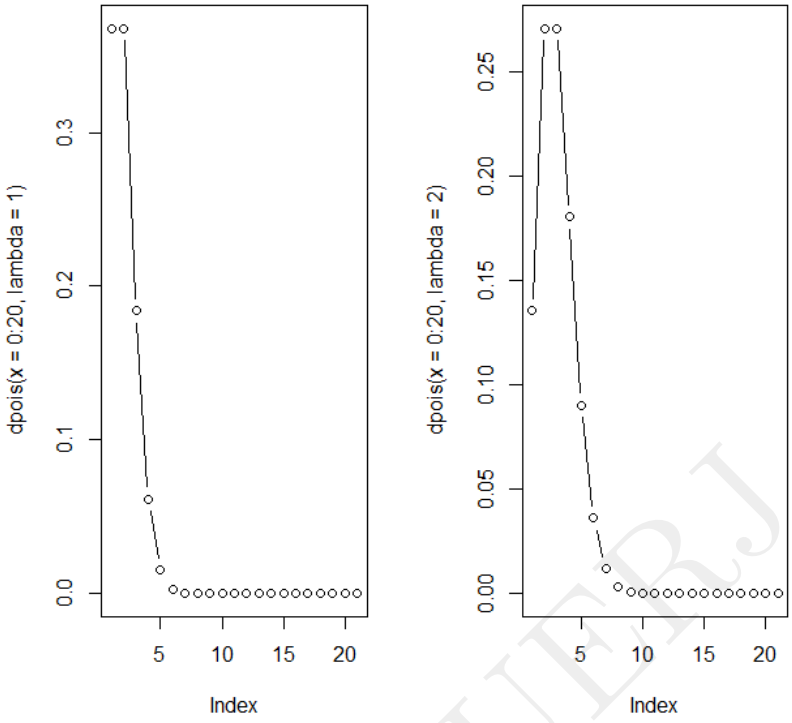


Figura 41: Distribuição de Poisson.

Distribuição Exponencial

A seguir, são mostradas as equações da função densidade de probabilidade, da média e da variância[3].

$$f(x) = \begin{cases} \alpha e^{-\alpha x} & \text{para} \quad x > 0 \\ 0 & \text{para} \quad x \leq 0 \end{cases} \tag{16}$$

$$\mu = \frac{1}{\alpha} \tag{17}$$

$$\sigma^2 = \frac{1}{\alpha^2} \tag{18}$$

Influência do tamanho da amostra

Considere a função `rexp(n, rate=1)`. O prefixo `r` retorna valores aleatórios, enquanto o sufixo `exp` indica que esses valores terão aderência à distribuição exponencial. Esta função tem dois argumentos: `n`, que é o tamanho amostral, e `rate`, a taxa. Considere as simulações a seguir, com n aumentando.

[3]Spiegel, m. R.; Schiller, J.; Srinivasan, A. Schaum's outlines: Probability and Statistics, 3rd ed, McGraw-Hill, 2009, p. 118

PPGEQ/UERJ

```r
# Reprodução dos resultados
set.seed(1)
# Histograma da distribuição exponencial
par(mfrow=c(2, 2))
hist(rexp(10), col="turquoise3")
hist(rexp(100), col="turquoise3")
hist(rexp(1000), col="turquoise3")
hist(rexp(10000), col="turquoise3")
dev.off()
```

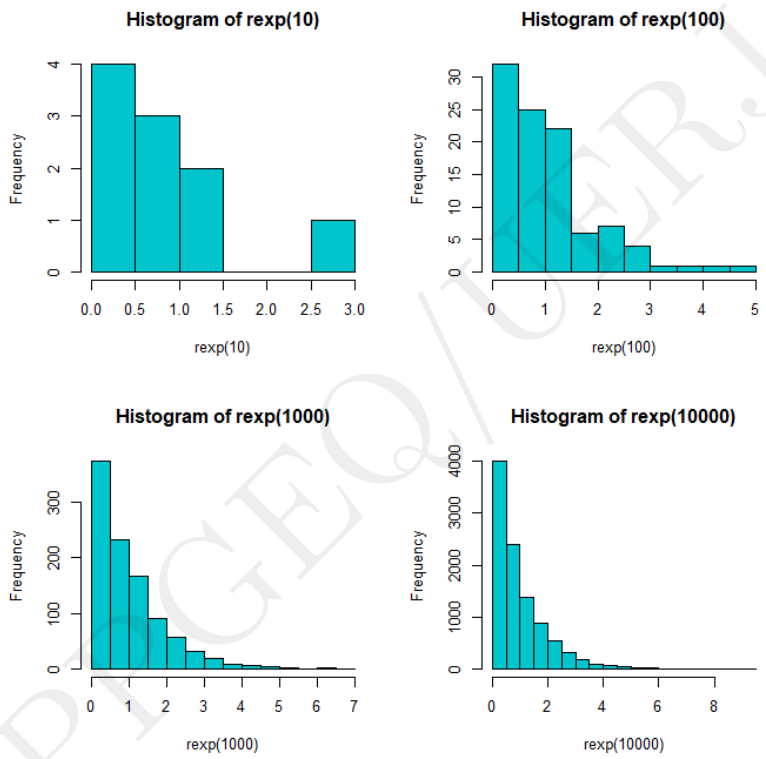


Figura 42: Distribuição Exponencial

PPGEQ/UERJ

Amostragem Aleatória

A função `rnorm(n, mean, sd)` gera números aleatórios a partir de uma distribuição normal. Caso nenhum valor seja atribuído aos argumentos `mean` e `sd`, a função interpretará como `mean=0` e `sd=1`.

```
# Valores aleatórios: n = 5, média = 100 e desvio-padrão = 15.
rnorm(5, m = 100, s = 15)
```

```
## [1] 121.81286  79.76891 112.24590 104.22209 108.17701
```

```
rnorm(5, m = 100, s = 15)
```

```
## [1]  79.91043  80.26800 111.85729  90.88871  82.22616
```

Para que ocorra repetição é necessário colocar uma semente

```
# Use a função set.seed()
set.seed(7637060)
rnorm(5, m = 100, s = 15)
```

```
## [1]  71.99120  98.67231  92.68848 103.42207  99.61904
```

```
set.seed(7637060)
rnorm(5, m = 100, s = 15)
```

```
## [1]  71.99120  98.67231  92.68848 103.42207  99.61904
```

Funções da distribuição normal no R

```
dnorm(0)
```

```
## [1] 0.3989423
```

```
pnorm(0)
```

```
## [1] 0.5
```

```
qnorm(c(0.25, 0.50, 0.75))
```

```
## [1] -0.6744898  0.0000000  0.6744898
```

PPGEQ/UERJ

```r
rnorm(5)
```

```
## [1]  0.9155883 -2.4263919 -0.9018797  1.4056493  0.5340863
```

```r
# Usando o pacote tigerstats
library(tigerstats)
pnormGC(c(-1,0), region = "between")
```

```
## [1] 0.3413447
```

```r
qnormGC(0.50, region = "below")
```

```
## [1] 0
```

Diagramando a distribuição normal padrão

```r
# Criam-se os vetores
z.values <- seq(-4, 4, 0.01)
z.sd.values <- seq(-4, 4, 1)
ggplot(NULL, aes(x = z.values, y = dnorm(z.values))) +
  geom_line() +
  labs(x = "z", y = "f(z)") +
  scale_x_continuous(breaks = z.sd.values,
                     labels = z.sd.values) +
  geom_segment((aes(x = z.sd.values,
                    y = zeros9,
                    xend = z.sd.values,
                    yend = dnorm(z.sd.values))),
               linetype = "dashed") +
  scale_y_continuous(expand = c(0,0))
```

PPGEQ/UERJ

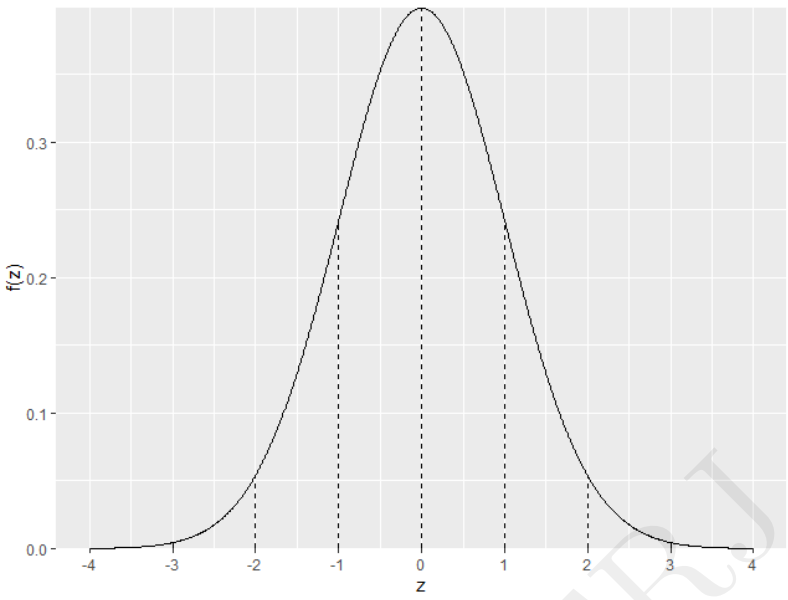


Figura 43: Distribuição normal padronizada.

;

PPGEQ/UERJ

Estimação e Intervalo de Confiança

Simulando o teorema do limite central

Como uma população que não é normalmente distribuída produz uma distribuição amostral normalmente distribuída?

Imagine uma população enorme que consiste em apenas três pontuações (1, 2 e 3). Cada pontuação tem a mesma probabilidade de aparecer na amostra.

```
# Criam-se os vetores para as pontuações e outro para as probabilidades.
values <- c(1, 2, 3)
probabilities <- c(1/3, 1/3, 1/3)
# Cria-se outro vetor para as 600 médias amostrais
smpl.means <- NULL
# Para retirar uma amostra, use a função sample()
smpl <- sample(x = values,
             prob = probabilities,
             size = 3,
             replace = TRUE)
# Calcula-se a média da amostra retirada
smpl.means <- append(smpl.means, mean(smpl))
smpl.means
```

```
## [1] 2
```

Imagine repetir este processo 600 vezes! O R pode nos ajudar usando o *loop* `for` (laço `for`).

```
# Fazendo a simulação
values <- c(1, 2, 3)
probabilities <- c(1/3, 1/3, 1/3)
smpl.means <- NULL
# Adição da função set.seed() para fazer a mesma simulação
set.seed(7637060)
for(i in 1:600){
  smpl <- sample(x = values, prob = probabilities,
               size = 3, replace = TRUE)
  smpl.means <- append(smpl.means, mean(smpl))
}
```

Uma vez feita a simulação, vamos verificar!

```
# Requisição do pacote:
library(ggplot2)
# Visualização:
ggplot(NULL, aes(x = smpl.means))+ geom_histogram()
```

PPGEQ/UERJ

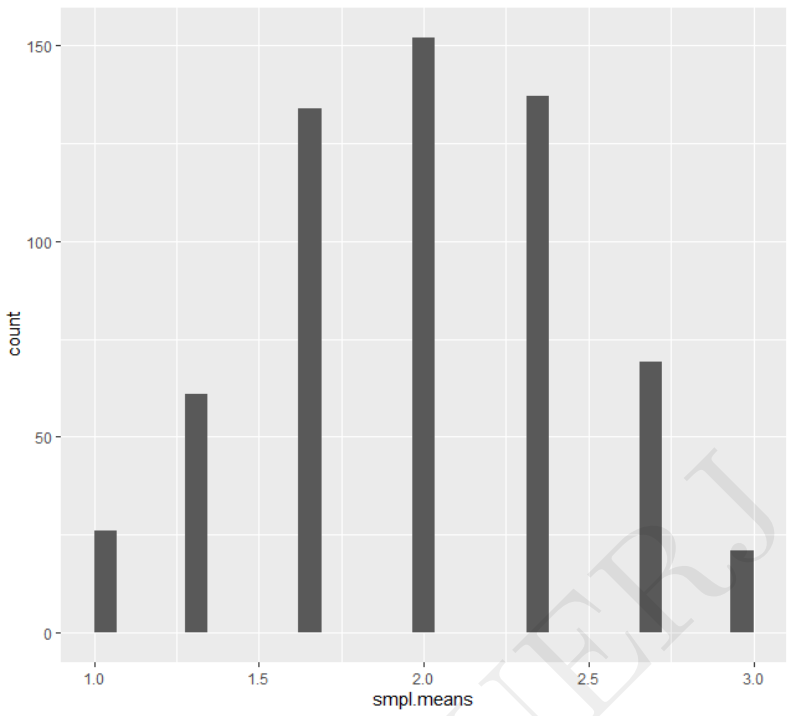


Figura 44: Distribuições de médias amostrais.

Torne o gráfico mais informativo! Vamos buscar os valores únicos!

```
# valores únicos:
unique(smpl.means)
# Arredondando para duas decimais
round(unique(smpl.means),2)
# Criar um vetor para armazenar estes dados
m.values <- round(unique(smpl.means), 2)
# Juntando tudo no único código
ggplot(NULL, aes(x = smpl.means)) +
  geom_histogram() +
  scale_x_continuous(breaks = m.values, labels = m.values) +
  scale_y_continuous(expand = c(0,0)) +
  labs(x = expression (bar(X)),
       y = expression(frequency(bar(X))))
```

FEGEO/UERJ

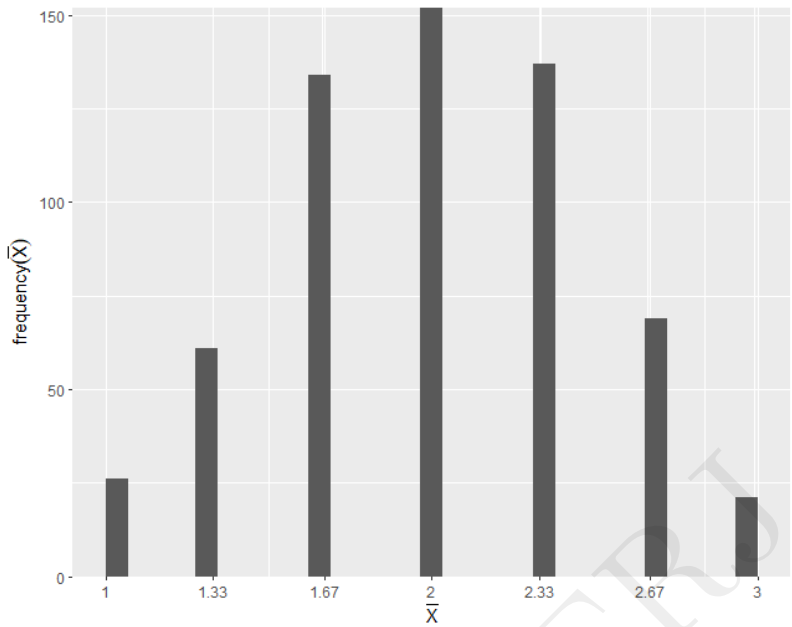


Figura 45: Distribuições de médias amostrais.

Previsões usando o teorema do limite central

A média de uma variável aleatória discreta é chamada de valor esperado ou esperança, $E(X)$.

$$E(X) = \sum_{i=1}^{n} x_i P(X = x_i) \tag{19}$$

```
# Valor esperado:
E.values <- sum(values*probabilities)
E.values
# A variância da variável aleatória discreta
var.values <- sum((values-E.values)^2*probabilities)
var.values
# O desvio padrão da variável aleatória discreta
sd.values <- sqrt(var.values)
sd.values
```

A população tem média 2 e desvio padrão 0,82. De acordo com o teorema do limite central, a média amostral deve ser 2 e o desvio padrão será (0,82/raiz de 3)= 0,47.

```
mean(smpl.means)
```

```
## [1] 2.002222
```

PPGEQ/UERJ

```r
sd(smpl.means)
```

```
## [1] 0.4745368
```

```r
# Valores concordantes?? Muito bem!!
```

Intervalo de confiança para a média usando a estatística z

$$\bar{x} \pm N\left(1 - \frac{\alpha}{2}; \mu = 0, \sigma = 1\right) \times \frac{s}{\sqrt{n}} \tag{20}$$

```r
# IC para média usando a estatística z
mean.battery <- 60
sd.battery <- 20
N <- 100
error <- qnorm(0.025, lower.tail = FALSE)*sd.battery/sqrt(N)
lower <- mean.battery-error
upper <- mean.battery+error
lower; upper
```

```
## [1] 56.08007
```

```
## [1] 63.91993
```

Intervalo de confiança para a média usando a estatística t

$$\bar{x} \pm t\left(1 - \frac{\alpha}{2}; \nu : n - 1\right) \times \frac{s}{\sqrt{n}} \tag{21}$$

```r
# IC para a média usando a estatística t
mean.battery <- 60
sd.battery <- 20
N <- 25
error <- qt(0.025, N-1, lower.tail = FALSE)*sd.battery/sqrt(N)
lower <- mean.battery-error
upper <- mean.battery+error
lower; upper
```

```
## [1] 51.74441
```

```
## [1] 68.25559
```

O pacote nativo `stats` tem a função `t.test()`, que permite obter a estimativa do intervalo de confiança da média, ajustando o nível de nível de significância.

Qual é o intervalo de confiança da média aritmética usando os dados brutos, com 90% de confiança?

PPGEQ/UERJ

```r
# Base de dados:
battery.data <- c(82, 64, 68, 44, 54, 47, 50, 85, 51, 41, 61, 84, 53, 83, 91, 43,
                  35, 36, 33, 87, 90, 86, 49, 37, 48)
# Estimativa do intervalo de confiança
t.test(battery.data, conf.level = 0.90)
```

```
##
##  One Sample t-test
##
## data:  battery.data
## t = 15, df = 24, p-value = 1.086e-13
## alternative hypothesis: true mean is not equal to 0
## 90 percent confidence interval:
##  53.22727 66.93273
## sample estimates:
## mean of x
##     60.08
```

PPGEQ/UERJ

Teste de hipóteses para uma amostra

Teste de hipóteses usando a estatística Z

$$Z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{N}} \sim N\left(\mu = 0, \sigma = 1\right) \tag{22}$$

Infelizmente, não existe a `z.test()` na base do R. É preciso instalar e habilitar o pacote BSDA.

```
# Requisição do pacote:
library(BSDA)
# Base de dados:
IQ.data <- c(100, 101, 104, 109, 125, 116, 105, 108, 110)
# Teste z
z.test(IQ.data,
       mu = 100,
       sigma.x = 15,
       conf.level = 0.95)
```

```
##
##  One-sample z-Test
##
## data:  IQ.data
## z = 1.7333, p-value = 0.08304
## alternative hypothesis: true mean is not equal to 100
## 95 percent confidence interval:
##   98.86685 118.46649
## sample estimates:
## mean of x
##  108.6667
```

PPGEQ/UERJ

Teste de hipóteses usando a estatística t

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} \sim t\left(\alpha, \nu : n - 1\right) \tag{23}$$

```
# Cria-se o vetor
FarKlempt.data <- c(3, 6, 9, 9, 4, 10, 6, 4, 12)
# teste t, usando a função t.test()
t.test(FarKlempt.data,
       mu = 4,
       alternative = "greater")
```

```
##
##  One Sample t-test
```

```
##
## data:  FarKlempt.data
## t = 2.8823, df = 8, p-value = 0.01022
## alternative hypothesis: true mean is greater than 4
## 95 percent confidence interval:
##  5.064521      Inf
## sample estimates:
## mean of x
##         7
```

O argumento `alternative` permite habilitar três opções:

- ▷ `greater`, quando $H_1 : \mu > \mu_0$.
- ▷ `less`, quando $H_1 : \mu < \mu_0$.
- ▷ `two.sided`, quando $H_1 : \mu \neq \mu_0$.

Trabalhando com a Distribuição t

A seguir, são mostradas as equações da função densidade de probabilidade, da média e da variância.

$$f(t;\nu) = \frac{\Gamma\left[(\nu+1)/2\right]}{\Gamma(\nu/2)\sqrt{\pi\nu}}\left(1+\frac{t^2}{\nu}\right)^{-(\nu+1)/2}, \ -\infty < t < +\infty \tag{24}$$

$$\mu = 0 \tag{25}$$

$$\sigma^2 = \frac{\nu}{\nu-2}, \ com\ \nu > 2 \tag{26}$$

Tabela 5: Funções relacionadas à distribuição t no R.

Função `R`	Resultado
`dt()`	densidade de probabilidade
`pt()`	probabilidade cumulativa
`qt()`	quantil
`rt()`	geração de números aleatórios para a distibuição t

```r
# Cria-se um vetor sequencial com graus de liberdade (gl) = 12
t.values <- seq(-4, 4, 1)
round(dt(t.values, 12), 2)
```

```
## [1] 0.00 0.01 0.06 0.23 0.39 0.23 0.06 0.01 0.00
```

PPGEO/UERJ

```r
round(pt(t.values, 12), 2)
```

```
## [1] 0.00 0.01 0.03 0.17 0.50 0.83 0.97 0.99 1.00
```

```r
# Para obter os quantis da distribuição t com gl = 12
quantis <- c(0.25, 0.50, 0.75)
qt(quantis, 12)
```

```
## [1] -0.6954829  0.0000000  0.6954829
```

```r
# Geração de 8 números aleatórios (arredondados) da distribuição t com df = 12
# Para reprodução dos resultados:
set.seed(1234)
round(rt(8, 12), 2)
```

```
## [1] -1.19  2.27  0.04  1.23 -0.77 -1.25  2.23 -0.66
```

Diagramando t em gráficos de base R

```r
# Diagramando t em gráficos de base R
t.values <- seq(-4, 4, 0.1)
# Função t com gl = 3
plot(x = t.values, y = dt(t.values, 3), type = "l", lty = "dotted",
     ylim = c(0, 0.4), xlab = "t", ylab = "f(t)", lwd = 2)
# Adiciona-se duas curvas para a distribuição t com gl = 10 e para a
# normal padrão (gl = infinito)
lines(t.values, dt(t.values, 10), lty = "dashed", lwd = 2)
lines(t.values, dnorm(t.values), lwd = 2)
# Adicionando a legenda
legend("topright", title = "gl", bty = "n", lwd = 2,
       legend = c(expression(infinity), "10", "3"),
       lty = c("solid", "dashed", "dotted"), cex = 1.5)
```

PPGEQ/UERJ

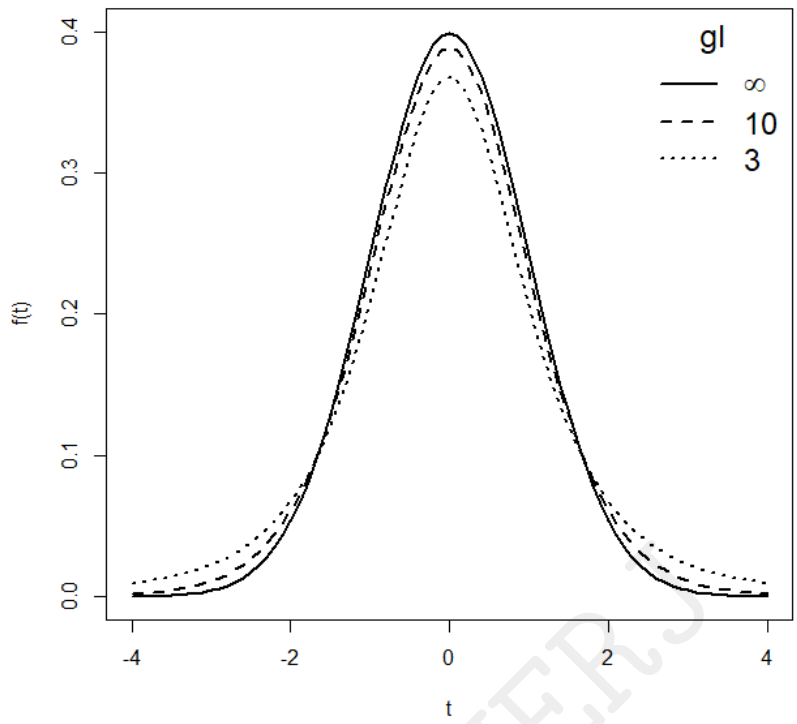


Figura 46: Distribuição t.

Diagramando t no ggplot2

```
# Cria-se um data frame usando os dados vetor anterior
t.frame <- data.frame(t.values,
                      df3 = dt(t.values,3),
                      df10 = dt(t.values, 10),
                      std_normal = dnorm(t.values))
head(t.frame)
```

```
##   t.values         df3        df10   std_normal
## 1       -4 0.009163361 0.002031034 0.0001338302
## 2       -3 0.022972037 0.011400549 0.0044318484
## 3       -2 0.067509661 0.061145766 0.0539909665
## 4       -1 0.206748336 0.230361989 0.2419707245
## 5        0 0.367552597 0.389108384 0.3989422804
## 6        1 0.206748336 0.230361989 0.2419707245
```

```
# Para fazer a remodelagem dos dados do formato largo para o longo
# Habilite o pacote reshape2
library(reshape2)
t.frame.melt <- melt(t.frame, id = "t.values")
head(t.frame.melt)
```

```
##   t.values variable       value
```

PPGEQ/UERJ

```
## 1       -4      df3 0.009163361
## 2       -3      df3 0.022972037
## 3       -2      df3 0.067509661
## 4       -1      df3 0.206748336
## 5        0      df3 0.367552597
## 6        1      df3 0.206748336
```

```r
# Renomeando-se as colunas
colnames(t.frame.melt) = c("t", "df", "density")
head(t.frame.melt)
```

```
##    t  df     density
## 1 -4 df3 0.009163361
## 2 -3 df3 0.022972037
## 3 -2 df3 0.067509661
## 4 -1 df3 0.206748336
## 5  0 df3 0.367552597
## 6  1 df3 0.206748336
```

```r
# Cria-se um vetor para organizar o eixo x.
x.axis.values <- seq(-4, 4, 2)
# Construção do gráfico
ggplot(t.frame.melt, aes(x = t, y = density, group = df)) +
  geom_line(aes(linetype = df), color = "blue", lwd = 1.2) +
  scale_x_continuous(breaks = x.axis.values,
                     labels = x.axis.values) +
  scale_linetype_manual(values = c("dotted", "dashed", "solid"),
                        labels = c("3", "10", expression(infinity)))+
  guides(linetype=guide_legend(reverse = TRUE)) +
  theme(axis.title.x = element_text(size = rel(1.5)),
        legend.text = element_text(size = rel(1.5)),
        legend.title = element_text(size = rel(1.5)))
```

PEGEQ/UERJ

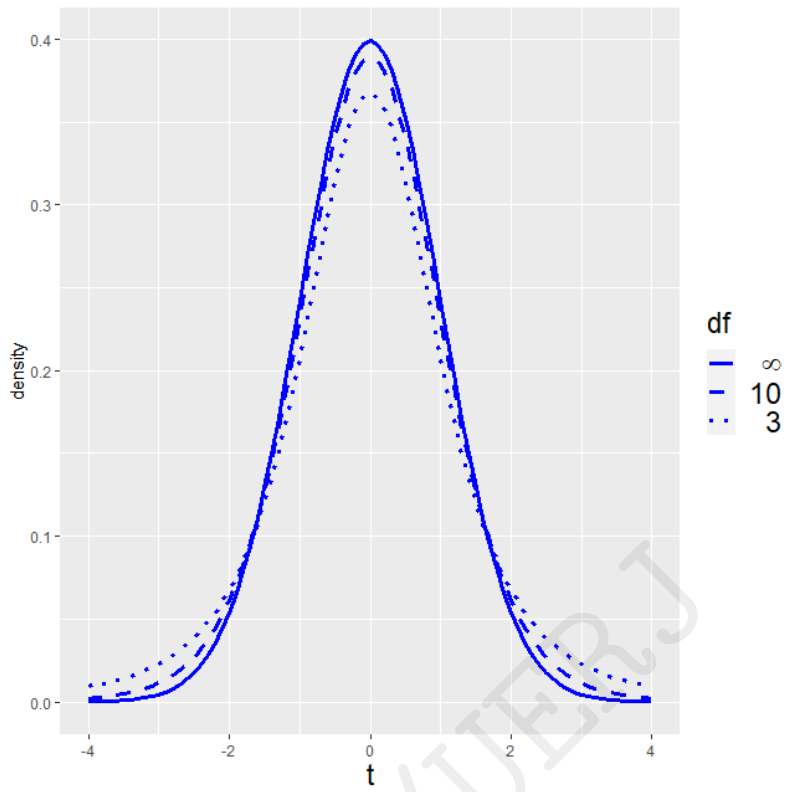


Figura 47: Distribuição t no ggplot2.

Teste da Variância

$$\chi^2 = \frac{(n-1)s^2}{\sigma^2} \sim \chi^2(\alpha, \nu : n-1) \tag{27}$$

```
# Requisição do pacote:
library(EnvStats)
# Cria-se um vetor de dados e supondo que a variância hipotética seja 2.25
FarKlempt.data <- c(12.43, 11.71, 14.41, 11.05, 9.53, 11.66, 9.33, 11.71, 14.35,
                    13.81)
varTest(FarKlempt.data,
        alternative = "greater",
        conf.level = 0.95,
        sigma.squared = 2.25)
```

```
## $statistic
## Chi-Squared
##     12.9812
##
## $parameters
## df
##  9
##
```

PPGEQ/UERJ

```
## $p.value
## [1] 0.163459
##
## $estimate
## variance
## 3.245299
##
## $null.value
## variance
##     2.25
##
## $alternative
## [1] "greater"
##
## $method
## [1] "Chi-Squared Test on Variance"
##
## $data.name
## [1] "FarKlempt.data"
##
## $conf.int
##      LCL      UCL
## 1.726327      Inf
## attr(,"conf.level")
## [1] 0.95
##
## attr(,"class")
## [1] "htestEnvStats"
```

PPGEQ/UERJ

Trabalhando com Distribuição qui-quadrado

A seguir, são mostradas as equações da função densidade de probabilidade, da média e da variância[4].

$$f\left(x;\nu\right)=\begin{cases}\frac{1}{2^{\nu/2}\Gamma(\nu/2)}x^{\nu/2-1}e^{-x/2} & \text{para} \quad x>0\\ 0 & \text{para} \quad x\leq 0\end{cases} \tag{28}$$

onde $\Gamma\left(.\right)$ é a **função gama**, dada por:

$$\Gamma\left(\alpha\right)=\int_0^\infty x^{\alpha-1}e^{-x}dx,\ para\ \alpha>0 \tag{29}$$

$$\mu=\nu \tag{30}$$

[5]Spiegel, m. R.; Schiller, J.; Srinivasan, A. Schaum's outlines: Probability and Statistics, 3rd ed, McGraw-Hill, 2009, p. 113

$$\sigma^2 = 2\nu \tag{31}$$

Note que a distribuição qui-quadrado, χ^2, é um caso especial da Distribuição Gama, quando $\alpha = \nu/2$ e $\beta = 2$. A tabela a seguir mostra as funções relacionadas à distribuição qui-quadrado, no R, oriundas do pacote `stats`.

Tabela 6: Funções relacionadas à distribuição qui-quadrado, χ^2, no R.

Função `R`	Resultado
`dchisq()`	densidade de probabilidade
`pchisq()`	probabilidade cumulativa
`qchisq()`	quantil
`rchisq()`	geração de números aleatórios para a distibuição qui-quadrado

```r
# Qual o valor do quantil (valor crítico) do exercício anterior
qchisq(0.05, df = 9, lower.tail = FALSE)
```

```
## [1] 16.91898
```

```r
# Considere o conjunto de valores
chisq.values <- seq(0, 16, 2)
# A densidade de probabilidade é
round(dchisq(chisq.values, 9), 3)
```

```
## [1] 0.000 0.016 0.066 0.100 0.101 0.081 0.056 0.036 0.021
```

```r
# A probabilidade cumulativa é
round(pchisq(chisq.values, 9), 3)
```

```
## [1] 0.000 0.009 0.089 0.260 0.466 0.650 0.787 0.878 0.933
```

```r
# Selecione 6 números aleatórios com distribuição qui-quadrada
round(rchisq(n = 6, df = 9), 3)
```

```
## [1] 5.621 8.545 6.158 8.020 7.850 3.256
```

PPGEQ/UERJ

Visualizando a distribuição qui-quadrado

```
# Criação do vetor de dados
chi.values <- seq(0, 25, 0.1)
plot(x = chi.values,
     y = dchisq(chi.values, df = 4),
     type = "l",
     xlab = expression(chi^2), cex.axis = 1.5, cex.lab = 1.5,
     ylab = "", lwd = 2, col = "blue")
# Para evitar cortar o rótulo de y
mtext(side = 2, text = expression(f(chi^2)), line = 2.5, cex = 1.5)
# Adicione a curva do qui-quadrado com df = 10
lines(x = chi.values, y = dchisq(chi.values, df = 10), lwd = 2, col = "red")
# Acrescente a identificação de cada curva
text(x = 6, y = 0.15, label = "df=4", cex = 1.5)
text(x = 16, y = 0.07, label = "df=10", cex = 1.5)
abline(h=0)
```

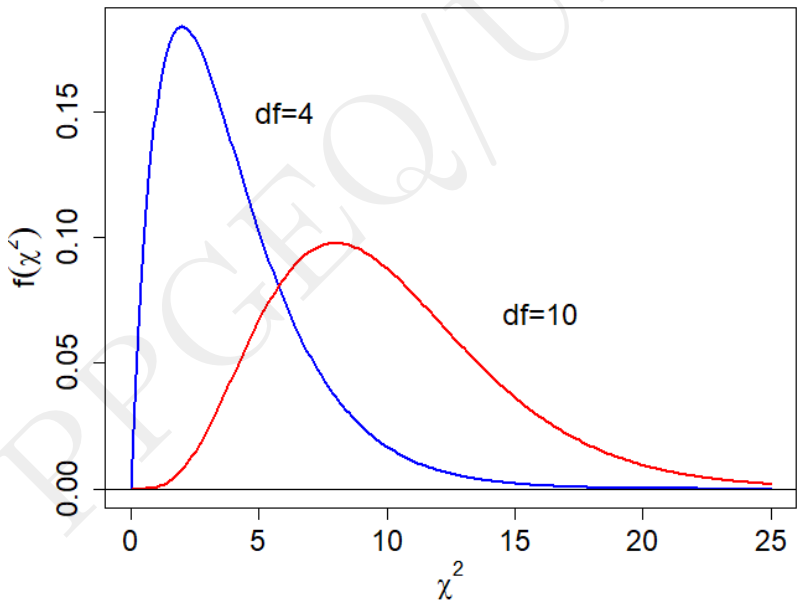


Figura 48: Distribuição qui-quadrado.

PPGEQ/UERJ

Diagramando a distribuição qui-quadrado com ggplot2

```
# Distribuição qui-quadrado
ggplot(NULL, aes(x = chi.values)) +
  geom_line(aes(y = dchisq(chi.values, 4)), color ="blue", lwd=1.5) +
  geom_line(aes(y = dchisq(chi.values, 10)), color ="red", lwd=1.5) +
  labs(x = expression(chi^2), y = expression(f(chi^2))) +
  annotate(geom = "text", x = 6, y = 0.15, label = "df=4") +
  annotate(geom = "text", x = 16, y = 0.07, label = "df=10")
```

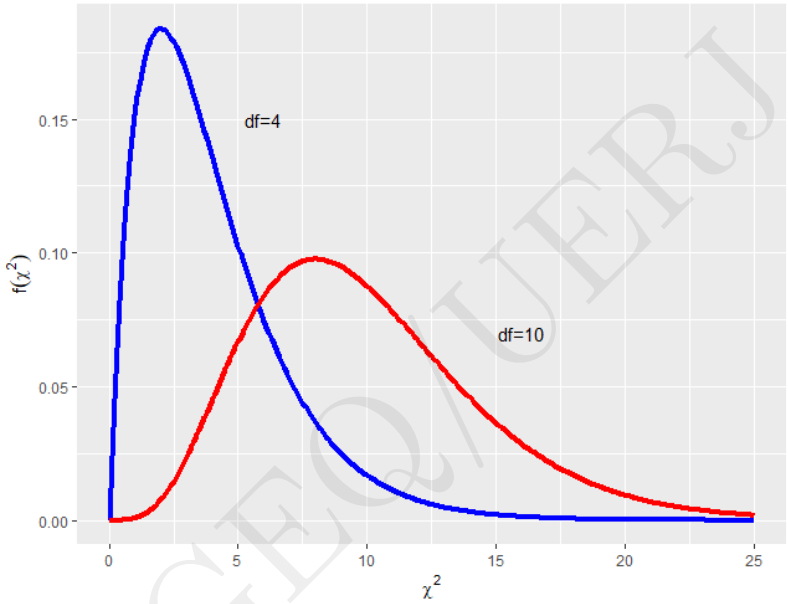


Figura 49: Distribuição qui-quadrado com ggplot2.

PPGEQ/UERJ

Teste de hipóteses para duas amostras

Teste Z

$$Z = \frac{(\bar{x}_1 - \bar{x}_2) - (\mu_1 - \mu_2)}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}} \tag{32}$$

```r
# Criam-se dois vetores
x <- c(100, 118, 97, 92, 118, 125, 136, 95, 111)
y <- c(91, 109, 83, 88, 115, 108, 127, 102, 86)
# Desvio-padrão conhecido
sigma.x = 15
sigma.y = 15
# Requisição do pacote
library(BSDA)
z.test(x, y, alternative = "two.sided", mu = 0,
       sigma.x = 15,
       sigma.y = 15,
       conf.level = 0.95)
```

```
##
##  Two-sample z-Test
##
## data:  x and y
## z = 1.3042, p-value = 0.1922
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  -4.636816 23.081260
## sample estimates:
## mean of x mean of y
##  110.2222  101.0000
```

PPGEQ/UERJ

Teste de hipóteses para duas amostras com a estatística t

Variâncias iguais

$$t = \frac{(\bar{x}_1 - \bar{x}_2) - (\mu_1 - \mu_2)}{\sqrt{s_p^2 \left(\frac{1}{n_1} + \frac{1}{n_2}\right)}} \sim t(\nu : n_1 + n_2 - 2) \tag{33}$$

onde

$$S_p^2 = \frac{(n_1 - 1)\, s_1^2 + (n_2 - 1)\, s_2^2}{(n_1 - 1) + (n_2 - 1)}$$

Variâncias diferentes

$$t = \frac{(\bar{x}_1 - \bar{x}_2) - (\mu_1 - \mu_2)}{\sqrt{\frac{s_p^2}{n_1} + \frac{s_2^2}{n_2}}} \sim t(\nu') \tag{34}$$

onde

$$\nu' = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{\left(\frac{s_1^2}{n_1}\right)^2}{n_1 - 1} + \frac{\left(\frac{s_2^2}{n_2}\right)^2}{n_2 - 1}}$$

Teste t para amostras dependentes ou pareadas

$$t = \frac{\bar{d} - \mu_d}{s_d / \sqrt{n}} \sim t(n - 1) \tag{35}$$

onde $\bar{d}$ é a média da diferença e s_d é o desvio-padrão da diferença.

Teste de igualdade de variâncias

$$F = \frac{s_1^2}{s_2^2} \sim F(1 - \alpha, \nu_1 : n_1 - 1, \nu_2 : n_2 - 1) \tag{36}$$

No `R`, a função `t.test()` permite obter uma série de análises entre duas amostras:

▷ Argumentos `x, y = NULL`: é realizado apenas o teste t para a média de uma amostra. Se são colocas os dois vetores, `x` e `y`, é calculado então um teste t para amostras independentes.

▷ Argumento `alternative`: permite escolher se o teste é bilateral (`two.sided`), unilateral superior (`greater`) ou unilateral inferior (`less`).

▷ Argumento `mu`: quando `mu=0`, então a diferença teórica entre as duas médias populacionais é zero.

PPGEQ/UERJ

▷ Argumento `paired`: se `paired=TRUE`, é realizado o cálculo para um teste t para amostras depentes ou pareadas.

▷ Argumento `var.equal`: indica lógico que indica se as variâncias são iguais (`var.equal=TRUE`) ou não.

▷ Argumento `conf.level`: especifica o nível de confiança.

Pode-se, ainda, utilizar a função `var.test(A, B)`, para verificar se as variâncias são iguais ou não. Neste caso, `A` é o vetor com a maior variância e `B`, a menor.

```
# -------------------------------
# Teste t, variâncias iguais
# -------------------------------
# Criam-se dois vetores
machine1 <- c(24.58, 22.09, 23.70, 18.89, 22.02, 28.71, 24.44, 20.91, 23.83, 20.83)
machine2 <- c(21.61, 19.06, 20.72, 15.77, 19, 25.88, 21.48, 17.85, 20.86, 17.77)
# Supondo-se que os conjuntos de dados têm variância constante
t.test(machine1, machine2, var.equal = TRUE, alternative = "two.sided", mu = 0)
```

```
##
##  Two Sample t-test
##
## data:  machine1 and machine2
## t = 2.4396, df = 18, p-value = 0.02528
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  0.4164695 5.5835305
## sample estimates:
## mean of x mean of y
##        23        20
```

```
# Representação gráfica através do boxplot
data <- data.frame(machine1, machine2)
boxplot(data, col=c("deepsky1", "khaki1"))
```

PPGEQ/UEPI

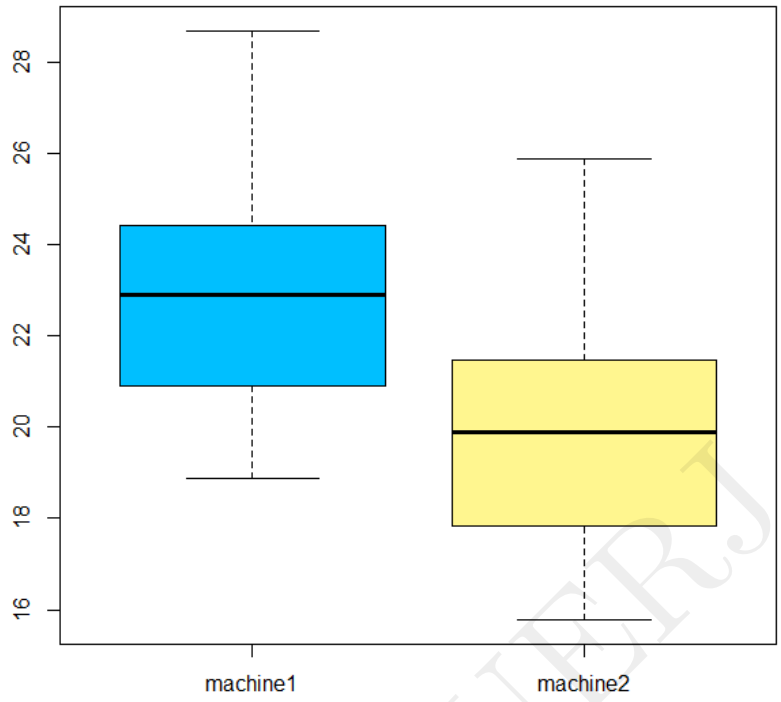


Figura 50: Gráfico de caixas dos dados machine1 e machine2.

```
# -------------------------------
# Teste t, variâncias diferentes
# -------------------------------
# Supondo-se que as variâncias sejam diferentes
t.test(machine1, machine2, var.equal = FALSE, alternative = "two.sided", mu = 0)
```

```
##
##  Welch Two Sample t-test
##
## data:  machine1 and machine2
## t = 2.4396, df = 17.985, p-value = 0.02529
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  0.4163193 5.5836807
## sample estimates:
## mean of x mean of y
##        23        20
```

```
# -------------------------------
# Teste F, de proprocionalidde de variâncias
# -------------------------------
# Criam-se dois vetores
machine1 <- c(24.58, 22.09, 23.70, 18.89, 22.02, 28.71, 24.44, 20.91, 23.83, 20.83)
machine2 <- c(21.61, 19.06, 20.72, 15.77, 19, 25.88, 21.48, 17.85, 20.86, 17.77)
```

PPGEQ/UERJ

```r
# A variância da amostra 1 é maior ?
var(machine1) > var(machine2)
```

```
## [1] FALSE
```

```r
# Teste de proporcionalidade de variâncias:
var.test(machine2, machine1,
         ratio = 1,
         alternative = "two.sided",
         conf.level = 0.95)
```

```
##
##  F test to compare two variances
##
## data:  machine2 and machine1
## F = 1.0586, num df = 9, denom df = 9, p-value = 0.9338
## alternative hypothesis: true ratio of variances is not equal to 1
## 95 percent confidence interval:
##  0.2629492 4.2620464
## sample estimates:
## ratio of variances
##           1.058632
```

```r
# -------------------------------
# Teste t, amostras pareadas
# -------------------------------
# Criam-se dois vetores de dados
before <- c(198, 201, 210, 185, 204, 156, 167, 197, 220, 186)
after <- c(194, 203, 200, 183, 200, 153, 166, 197, 215, 184)
t.test(before, after, alternative = "greater", paired = TRUE)
```

```
##
##  Paired t-test
##
## data:  before and after
## t = 2.8241, df = 9, p-value = 0.009956
## alternative hypothesis: true difference in means is greater than 0
## 95 percent confidence interval:
##  1.017647      Inf
## sample estimates:
## mean of the differences
##                     2.9
```

PPGEQ/UERJ

Trabalhando com a distribuição F

A função densidade de probabilidade da distribuição F é dada por:

$$f(x) = \begin{cases} f\left(u|\nu_1,\nu_2\right) = \frac{\Gamma\left(\frac{(\nu_1+\nu_2)}{2}\right)}{\Gamma\left(\frac{\nu_1}{2}\right)\Gamma\left(\frac{\nu_2}{2}\right)}\nu_1^{\nu_1/2}\nu_2^{\nu_2/2}u^{(\nu_1/2)^{-1}}\left(\nu_2+\nu_1 u\right)^{-(\nu_1+\nu_2)/2} & \text{para} \quad x>0 \\ 0 \quad \text{para} \quad x \leq 0 \end{cases} \tag{37}$$

A média da distribuição F de Snedcor é dada por:

$$\mu = E\left(u\right) = \frac{\nu_2}{\nu_2 - 2} \; com \; \nu_2 > 2 \tag{38}$$

A variância da distribuição F de Snedcor é dada por:

$$Var\left(u\right) = \sigma^2 = \frac{2\nu_2^2\left(\nu_1+\nu_2-2\right)}{\nu_1\left(\nu_2-2\right)^2\left(\nu_2-4\right)} \tag{39}$$

Tabela 7: Funções relacionadas à distribuição F no R.

Função R	Resultado
`df()`	densidade de probabilidade
`pf()`	probabilidade cumulativa
`qf()`	quantil
`rf()`	geração de números aleatórios para a distibuição F

Qual o valor crítico para um teste F bicaudal com 9 e 14 graus de liberdade (alfa = 0,05)?

```
# Neste caso, df1 = 9 e df2 = 14
qf(0.025, 9, 14, lower.tail = FALSE)
```

```
## [1] 3.2093
```

```
# Considere agora o seguinte vetor de dados:
F.scores<-seq(0, 5, 1)
# Função densidade de probabilidade
round(df(F.scores, 9, 14), 3)
```

```
## [1] 0.000 0.645 0.164 0.039 0.011 0.004
```

```
# Função densidade cumulativa
round(pf(F.scores, 9, 14), 3)
```

PPCIO/UERJ

```
## [1] 0.000 0.518 0.882 0.968 0.990 0.996
```

```r
# Para gerar 5 números aleatórios
set.seed(1234)
rf(5, 9, 14)
```

```
## [1] 0.4140651 1.5489699 1.5430493 1.9277394 1.7219407
```

Visualizando a distribuição F

Considere duas distribuições: uma $F(\nu_1 = 5, \nu_2 = 15)$ e outra $F(\nu_1 = 10, \nu_2 = 20)$.

```r
# Requisição dos pacotes:
library(dplyr)
library(ggplot2)
library(tidyr)

data.frame(f = 0:1000 / 100) %>%
  mutate(df_10_20 = df(x = f, df1 = 10, df2 = 20),
         df_05_15 = df(x = f, df1 = 5, df2 = 15)) %>%
  gather(key = "df", value = "density", -f) %>%
  ggplot() +
  geom_line(aes(x = f, y = density, color = df), lwd = 2) +
  labs(title = "F at Various Degrees of Freedom",
       x = "F",
       y = "Density")
```

PPGEO/UERJ

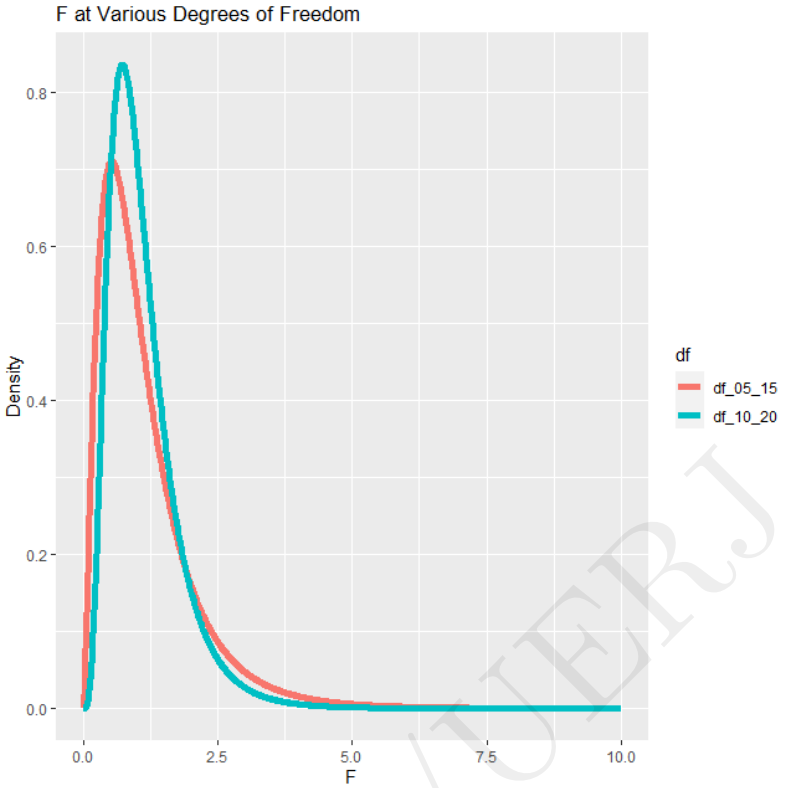


Figura 51: Duas distribuições F, com diferentes graus de liberdade nos dois parâmetros.

;

PPGEQ/UERJ

Parte II

Análise da Variância (ANOVA One-way)

Considere o seguinte problema:

Periodicamente, o The Wall Street Journal conduz um concurso de escolha de ações. O mais recente foi conduzido em março de 2001. Nesse experimento, três diferentes métodos foram utilizados para selecionar ações que se esperava apresentassem um bom desempenho nos cinco meses subsequentes.Quatro profissionais de Wall Street, considerados especialistas em escolha de ações, selecionaram, cada um deles, uma ação. Quatro leitores do The Wall Street Journal, escolhidos aleatoriamente, selecionaram cada um uma ação. Por fim, quatro ações foram selecionadas por arremesso de dardos em um alvo contendo uma lista de ações.

Os rendimentos das ações selecionadas, no período entre 20 de março de 2001 até 31 de agosto de 2001 (em termos do percentual de rendimento) estão apresentados no arquivo CONCURSO2001.xls (livro-texto do Levine).

Observe que durante este período a Média Industrial Dow Jones cresceu 2,4%. (extraído de G. Jansen, "In picking stocks, dartboard beats the pros", The Wall Street Journal, de 27 de setembro de 2001, p. C1, C10).

Especialistas	**Leitores**	**Dardos**
39,5	-31,0	39,0
-1,1	-20,7	31,9
-4,5	-45,0	14,1
-8,0	-73,3	5,4

Existem evidências de uma diferença significativa na média do rendimento para as três categorias? Utilize alfa=0,05.

Testes de hipóteses podem ser feitos, com alinhamento:

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_c$$
$$H_1 : Nem\ todos\ as\ \mu_j\ são\ iguais$$

Fonte de variação	G.L.	S.Q.	M.Q.	Fcalc	Fcrit
Entre	$c-1$	SQE	MQE	$\frac{MQE}{MQD}$	$F(1-\alpha, c-1, n-c)$
Dentro	$n-c$	SQD	MQD		
Total	$n-1$	SQT	MQT		$=$

PPGEQ/UERJ

onde:

- Soma quadrática entre grupos, SQE:

$$SQE = \sum_{j=1}^{c} n_j \left(\bar{X}_j - \bar{\bar{X}}\right)^2 \tag{40}$$

- Soma quadrática *dentro dos* grupos, SQD:

$$SQD = \sum_{j=1}^{c} \sum_{i=1}^{n_j} \left(X_{ij} - \bar{X}_j\right)^2 \tag{41}$$

- Soma quadrática entre grupos, SQT:

$$SQT = \sum_{j=1}^{c} \sum_{i=1}^{n_j} \left(X_{ij} - \bar{\bar{X}}\right)^2 \tag{42}$$

onde $\bar{\bar{X}}$ é a *média global*, obtida como:

$$\bar{\bar{X}} = \frac{\sum_{j=1}^{c} \sum_{i=1}^{n_j} X_{ij}}{n} \tag{43}$$

Existem duas abordagens, na hora de estruturar a base de dados. A primeira considera que todos os tratamentos tem o mesmo número de amostras. A segunda abordagem, permite lidar com tratamentos desbalanceados. A seguir, serão mostradas as duas formas.

► Primeira forma de estruturar os dados e calcular a ANOVA de fator único:

```r
# ------------------
# Primeira forma:
# ------------------
# Base de dados:
dados <- c(39.5, -31.0, 39.0, -1.1, -20.7, 31.9, -4.5, -45.0, 14.1, -8.0,
           -73.3, 5.4)
# rep(x, each, times)
rotulo <- factor(rep(c("Especialista", "Leitores", "Dardos"), 1:4, 12),
                 ordered = TRUE)
# ANOVA
resultado <- aov(dados~rotulo)
# Resultado da ANOVA
summary(resultado)
```

```
##             Df Sum Sq Mean Sq F value  Pr(>F)
## rotulo       2   9195    4598   10.99 0.00383 **
## Residuals    9   3764     418
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

PPGEQ/UERJ

► Segunda forma de estruturar os dados e calcular a ANOVA de fator único. Aplicável também a dados desbalanceados.

```
# ------------------
# Segunda forma
# ------------------
# Base de dados:
esp <- c(39.5, -1.1, -4.5, -8.0)
lei <- c(-31.0, -20.7, -45.0, -73.3)
dar <- c(39.0, 31.9, 14.1, 5.4)
# Data.frame
X <- data.frame(
    y = c(esp, lei, dar),
    Tratamentos = factor(c(rep("Especialista", length(esp)),
            rep("Leitores", length(lei)),
            rep("Dardos", length(dar)) )))
# ANOVA
modelo <- aov(y ~ Tratamentos, data = X)
# Resultado
summary(modelo)
```

```
##             Df Sum Sq Mean Sq F value  Pr(>F)
## Tratamentos  2   9195    4598   10.99 0.00383 **
## Residuals    9   3764     418
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Gráficos múltiplos na mesma janela
par(mfrow=c(2,2))
# Gráfico para análise de resíduos
plot(resultado)
```

PPGEO/UERJ

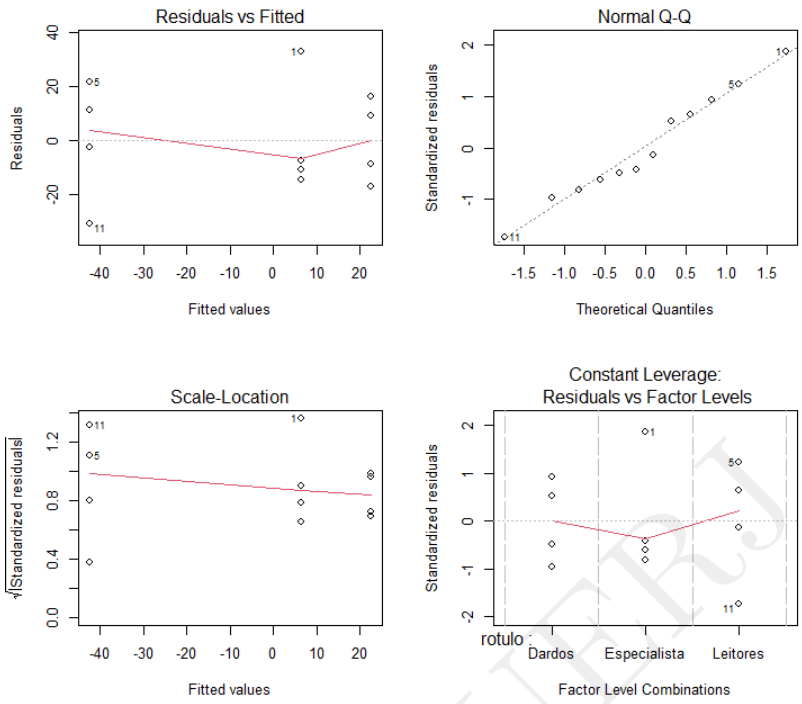


Figura 52: ANOVA fator único.

Procedimento de Tukey-Kramer de comparações múltiplas

Caso seja apropriado, determine quais categorias diferem na média aritmética do rendimento.

$$|\bar{x}_j - \bar{x}_{j'}| > Q_\alpha \times \sqrt{\frac{MQDentro}{2}\left(\frac{1}{n_j} + \frac{1}{n_{j'}}\right)} \tag{44}$$

onde Q_α é o valor crítico da distribuição de intervalos de Student, com c graus de liberdade no numerador e $N - c$ graus de liberdade no denominador.

```
# Teste de comparações múltiplas HSD de Tukey
tcm <- TukeyHSD(resultado, "rotulo", ordered = TRUE, conf.level=0.95)
tcm
```

```
##   Tukey multiple comparisons of means
##     95% family-wise confidence level
##     factor levels have been ordered
##
## Fit: aov(formula = dados ~ rotulo)
##
## $rotulo
##                          diff        lwr      upr     p adj
## Especialista-Leitores 48.975   8.601104  89.3489 0.0198472
## Dardos-Leitores       65.100  24.726104 105.4739 0.0038035
## Dardos-Especialista   16.125 -24.248896  56.4989 0.5291573
```

PPGEQ/UERJ

```r
# Valor crítico Q_alfa: qtukey(1-alfa, c, N-c)
qtukey(1-0.05, 3, 12-3)
```

```
## [1] 3.948492
```

```r
# Visualização
plot(TukeyHSD(resultado,
              which = "rotulo",
              ordered = TRUE,
              conf.level=0.95))
```

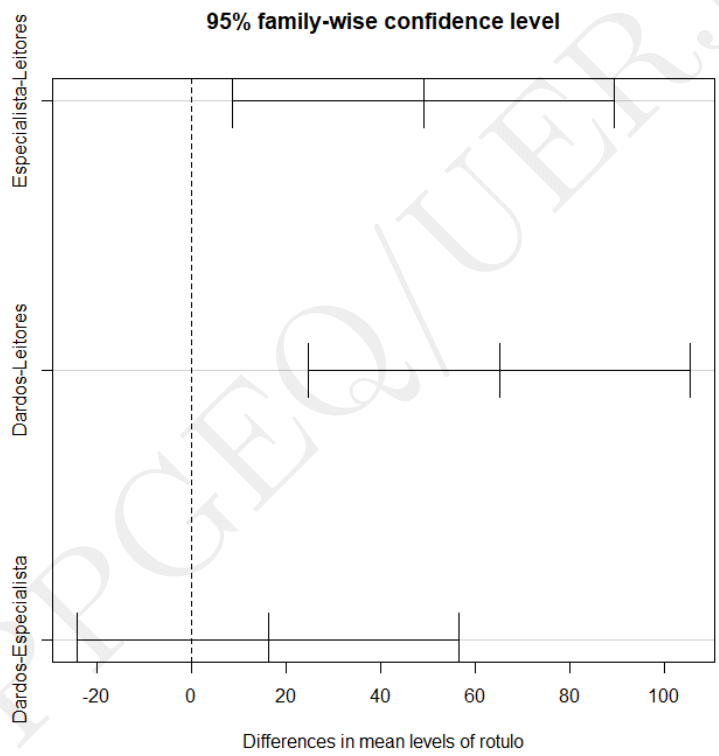


Figura 53: Procedimento de Tukey-Kramer.

PPGEQ/UERJ

Homogeneidade da variância

Existem evidências de uma diferença significativa na variação nos rendimentos para as três categorias? (utilize alfa=0,05)

Teste de Levene

O teste de Levene é feito da seguinte forma: toma-se a diferença, em módulo, de cada i-ésimo valor em relação à mediana do seu respectivo tratamento. A seguir, calcula-se a ANOVA e utiliza-se o mesmo valor crítico de F.

```r
# Resolução
tratamento <- data.frame(rotulo=rotulo, dados=dados)
# Requisição do pacote:
library(car)
# Teste de Levene modificado (diferença absoluta da mediana)
leveneTest(dados, rotulo)
```

```
## Levene's Test for Homogeneity of Variance (center = median)
##       Df F value Pr(>F)
## group  2  0.1011 0.9049
##        9
```

```r
# Teste de Levene original (diferença absoluta da média)
leveneTest(dados, rotulo, center = mean)
```

```
## Levene's Test for Homogeneity of Variance (center = mean)
##       Df F value Pr(>F)
## group  2   0.185 0.8342
##        9
```

Teste de Bartlett

Lembre-se de que este teste é sensível à desvios da normalidade e retorna resultado exato quando os tamanhos amostrais são iguais.

$$b = \frac{\left[(s_1^2)^{n_1-1}(s_2^2)^{n_2-1}\cdots(s_k^2)^{n_k-1}\right]^{1/(N-k)}}{s_p^2} \tag{45}$$

onde $N = \sum_{i=1}^{k} n_k$ e s_p^2 é a variância combinada, dada por:

$$s_p^2 = \frac{1}{N-k}\sum_{i=1}^{k}(n_i-1)\,s_i^2 \tag{46}$$

A H_0 será rejeitada se $b < b_k(\alpha, n)$, cujos valores críticos são oriundos da distribuição de Bartlett.

PPGEQ/UERJ

```r
bartlett.test(dados, rotulo)
```

```
##
##  Bartlett test of homogeneity of variances
##
## data:  dados and rotulo
## Bartlett's K-squared = 0.437, df = 2, p-value = 0.8037
```

PPGEQ/UERJ

ANOVA de dois fatores (ANOVA Two-way)

Considere o seguinte problema:

Você supervisiona a produção da Perfect Parachutes Company. Os paraquedas são de uma fibra sintética adquirida de um dentre quatro diferentes fornecedores.

A resistência dessas fibras é uma característica importante que garante paraquedas de qualidade.

Você precisa decidir se as fibras sintéticas de cada um de seus quatro fornecedores resultam em paraquedas de igual resistência.

Além disso, sua fábrica utiliza dois tipos de teares para produzir paraquedas: Jetta e Turk.

Avalie se quaisquer diferenças entre os quatro fornecedores, em termos da resistência dos paraquedas, são dependentes do tipo de tear utilizado. Os dados estão no arquivo paraquedas2.cls (livro-texto do Levine).

	Fornecedor			
Tear	Forn1	Forn2	Forn3	Forn4
Jetta	20,6	22,6	27,7	21,5
Jetta	18,0	24,6	18,6	20,0
Jetta	19,0	19,6	20,8	21,1
Jetta	21,3	23,8	25,1	23,9
Jetta	13,2	27,1	17,7	16,0
Turk	18,5	26,3	20,6	25,4
Turk	24,0	25,3	25,2	19,9
Turk	17,2	24,0	20,8	22,6
Turk	19,9	21,2	24,7	17,5
Turk	18,0	24,5	22,9	20,4

☞Também conhecida como **Delineamento em Blocos Casualizados (DBC)**.

Tabela 10: ANOVA de 2 fatores.

Fonte de variação	graus de liberdade	Soma quadrática	Média quadrática	F
Fator A	$r-1$	SQA	MQA	$F = \frac{MQA}{MQE}$
Fator A	$c-1$	SQB	MQB	$F = \frac{MQB}{MQE}$
Interação AB	$(r-1)(c-1)$	$SQAB$	$MQAB$	$F = \frac{MQAB}{MQE}$
Aleatória	$rc(n'-1)$	MQE		
Total	$n-1$	SQT		

onde:

▷ r = número de níveis do fator A.

▷ c = número de níveis do fator B.

PPGEC/UFRJ

▷ n' = número de valores (repetições) para cada célula (combinação de um determinado nível do fator A com um determinado nível do fator B).

▷ n = número de valores em todo o experimento (em que $n = rcn'$).

▷ x_{ijk} = valor da k-ésima observação do nível i do fator A e para o nível j do fator B.

$$\bar{\bar{x}} = \frac{\sum_{i=1}^{r}\sum_{j=1}^{c}\sum_{k=1}^{n'} x_{ijk}}{rcn'} \tag{47}$$

$$\bar{x}_{i..} = \frac{\sum_{j=1}^{c}\sum_{k=1}^{n'} x_{ijk}}{cn'} \tag{48}$$

$$\bar{x}_{.j.} = \frac{\sum_{i=1}^{r}\sum_{k=1}^{n'} x_{ijk}}{rn'} \tag{49}$$

$$\bar{x}_{ij.} = \frac{\sum_{k=1}^{n'} x_{ijk}}{n'} \tag{50}$$

$$SQT = SQA + SQB + SQAB + SQE \tag{51}$$

Variação total em ANOVA de dois fatores

$$SQT = \sum_{i=1}^{r}\sum_{j=1}^{c}\sum_{k=1}^{n'}\left(x_{ijk} - \bar{\bar{x}}\right)^2 \tag{52}$$

Variação decorrente do fator A

$$SQA = cn'\sum_{i=1}^{r}\left(\bar{x}_{i..} - \bar{\bar{x}}\right)^2 \tag{53}$$

Variação decorrente do fator B

$$SQB = rn'\sum_{j=1}^{c}\left(\bar{x}_{.j.} - \bar{\bar{x}}\right)^2 \tag{54}$$

Variação decorrente da interação

$$SQAB = n'\sum_{i=1}^{r}\sum_{j=1}^{c}\left(\bar{x}_{ij.} - \bar{x}_{i..} - \bar{x}_{.j.} + \bar{\bar{x}}\right)^2 \tag{55}$$

Variação aleatória em ANOVA de dois fatores

$$SQE = \sum_{i=1}^{r}\sum_{j=1}^{c}\sum_{k=1}^{n'}\left(x_{ijk} - \bar{x}_{ij.}\right)^2 \tag{56}$$

PPGEQ/UERJ

Média dos quadrados em ANOVA de dois fatores

$$MQA = \frac{SQA}{r-1} \tag{57}$$

$$MQB = \frac{SQB}{c-1} \tag{58}$$

$$MQAB = \frac{SQAB}{(r-1)(c-1)} \tag{59}$$

$$MQE = \frac{SQE}{rc(n'-1)} \tag{60}$$

Teste F

$$F = \frac{MQA}{MQE} \sim F(1-\alpha, r-1, rc(n'-1)) \tag{61}$$

$$F = \frac{MQB}{MQE} \sim F(1-\alpha, c-1, rc(n'-1)) \tag{62}$$

$$F = \frac{MQA}{MQE} \sim F(1-\alpha, (r-1)(c-1), rc(n'-1)) \tag{63}$$

```r
# Resolução
tear.fornec <- c(20.6, 22.6, 27.7, 21.5, 18.0, 24.6, 18.6, 20.0, 19.0, 19.6, 20.8,
                 21.1, 21.3, 23.8, 25.1, 23.9, 13.2, 27.1, 17.7, 16.0, 18.5, 26.3,
                 20.6, 25.4, 24.0, 25.3, 25.2, 19.9, 17.2, 24.0, 20.8, 22.6, 19.9,
                 21.2, 24.7, 17.5, 18.0, 24.5, 22.9, 20.4)
f <- rep(gl(4, 1, label = c(paste("Fornecedor", 1:4))), 1)
t <- gl(2, 20, label = c("Jetta", "Turk"))
tabela <- data.frame(tear = t, fornecedor = f, dados = tear.fornec)
tabela
```

```
##     tear   fornecedor dados
## 1  Jetta Fornecedor 1  20.6
## 2  Jetta Fornecedor 2  22.6
## 3  Jetta Fornecedor 3  27.7
## 4  Jetta Fornecedor 4  21.5
## 5  Jetta Fornecedor 1  18.0
## 6  Jetta Fornecedor 2  24.6
## 7  Jetta Fornecedor 3  18.6
## 8  Jetta Fornecedor 4  20.0
## 9  Jetta Fornecedor 1  19.0
## 10 Jetta Fornecedor 2  19.6
## 11 Jetta Fornecedor 3  20.8
```

PPGEQ/UERJ

```
## 12 Jetta Fornecedor 4  21.1
## 13 Jetta Fornecedor 1  21.3
## 14 Jetta Fornecedor 2  23.8
## 15 Jetta Fornecedor 3  25.1
## 16 Jetta Fornecedor 4  23.9
## 17 Jetta Fornecedor 1  13.2
## 18 Jetta Fornecedor 2  27.1
## 19 Jetta Fornecedor 3  17.7
## 20 Jetta Fornecedor 4  16.0
## 21  Turk Fornecedor 1  18.5
## 22  Turk Fornecedor 2  26.3
## 23  Turk Fornecedor 3  20.6
## 24  Turk Fornecedor 4  25.4
## 25  Turk Fornecedor 1  24.0
## 26  Turk Fornecedor 2  25.3
## 27  Turk Fornecedor 3  25.2
## 28  Turk Fornecedor 4  19.9
## 29  Turk Fornecedor 1  17.2
## 30  Turk Fornecedor 2  24.0
## 31  Turk Fornecedor 3  20.8
## 32  Turk Fornecedor 4  22.6
## 33  Turk Fornecedor 1  19.9
## 34  Turk Fornecedor 2  21.2
## 35  Turk Fornecedor 3  24.7
## 36  Turk Fornecedor 4  17.5
## 37  Turk Fornecedor 1  18.0
## 38  Turk Fornecedor 2  24.5
## 39  Turk Fornecedor 3  22.9
## 40  Turk Fornecedor 4  20.4
```

```
resultado.anova <- anova(aov(dados~fornecedor + tear + fornecedor:tear,
                             data = tabela))
resultado.anova
```

```
## Analysis of Variance Table
##
## Response: dados
##                 Df  Sum Sq Mean Sq F value   Pr(>F)
## fornecedor       3 134.349  44.783  5.1999 0.004866 **
## tear             1   6.972   6.972  0.8096 0.374968
## fornecedor:tear  3   0.287   0.096  0.0111 0.998365
## Residuals       32 275.592   8.612
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

PPGEQ/UERJ

```r
# Para o fator B (fornecedores), o teste de Tukey-Kramer
resultado <- aov(dados~fornecedor + tear + fornecedor:tear, data = tabela)
tcm <- TukeyHSD(resultado, "fornecedor", ordered = TRUE, conf.level = 0.95)
tcm
```

```
##   Tukey multiple comparisons of means
##     95% family-wise confidence level
##     factor levels have been ordered
##
## Fit: aov(formula = dados ~ fornecedor + tear + fornecedor:tear, data = tabela)
##
## $fornecedor
##                            diff        lwr      upr     p adj
## Fornecedor 4-Fornecedor 1 1.86 -1.6958243 5.415824 0.4981796
## Fornecedor 3-Fornecedor 1 3.44 -0.1158243 6.995824 0.0608501
## Fornecedor 2-Fornecedor 1 4.93  1.3741757 8.485824 0.0036713
## Fornecedor 3-Fornecedor 4 1.58 -1.9758243 5.135824 0.6289186
## Fornecedor 2-Fornecedor 4 3.07 -0.4858243 6.625824 0.1102811
## Fornecedor 2-Fornecedor 3 1.49 -2.0658243 5.045824 0.6708279
```

```r
# Visualização
# Definição de espaçamento da área externa
par(oma = c(0, 9, 0, 0)) # baixo, esquerda, acima, direita
plot(tcm, las = 1) # rótulos na horizontal
# encerramento da configuração:
dev.off()
```

PPGEQ/UFRJ

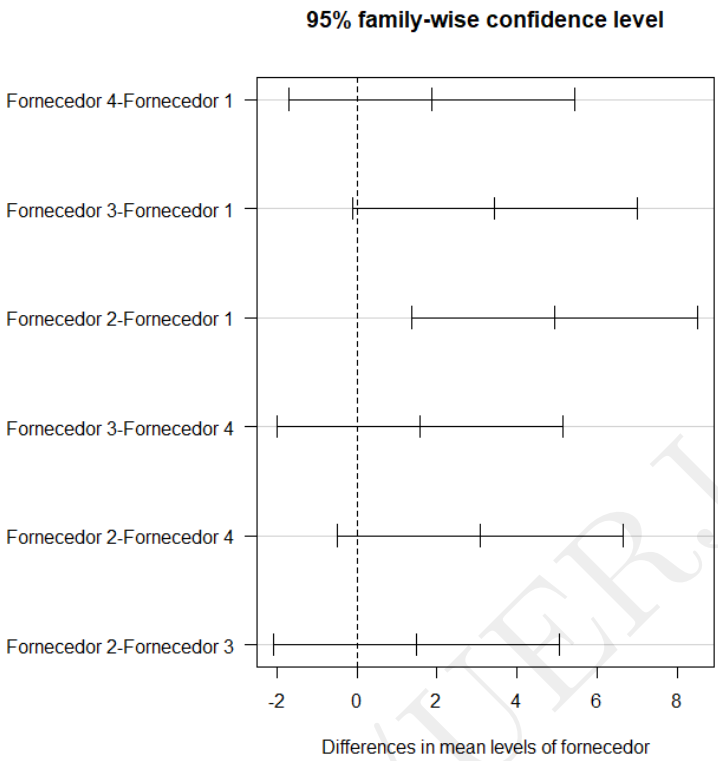


Figura 54: Procedimento de Tukey-Kramer.

;

PPGEQ/UERJ

Modelos Empíricos: Regressão linear simples

A regressão linear simples também é conhecida como regressão por mínimos quadrados ordinários (OLS) e é baseada no método de estimação por mínimos quadrados.

A seguir, são dadas pequenas revisões, sobre as saídas das funções no `R`. **Não deixe de consultar o material teórico!**

Modelo de regressão linear simples

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon \tag{64}$$

Equação da linha de previsão

A equação da linha de previsão é dada por:

$$\hat{y}_i = b_0 + b_1 x_i \tag{65}$$

onde

$$b_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{66}$$

$$b_0 = \bar{y} - \bar{x} b_1 \tag{67}$$

Coeficiente de determinação

Uma métrica de qualidade é dada pelo coeficiente de determinação, r^2:

$$r^2 = \frac{SQReg}{SQT} \tag{68}$$

onde

$$\begin{gathered} SQT = SQReg + SQRes \\ \sum_{i=1}^{n}(y_i - \bar{y})^2 = \sum_{i=1}^{n}(\hat{y}_i - \bar{y}_i)^2 + \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \end{gathered} \tag{69}$$

Coeficiente de determinação ajustado

$$r_{aj}^2 = 1 - \left[\left(1 - r^2\right) \times \frac{n-1}{n-k-1}\right] \tag{70}$$

PPGEQ/UERJ

Erro-padrão dos coeficientes angular e linear

$$s_{b_1} = \frac{\sqrt{\frac{\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}{n-2}}}{\sqrt{\sum_{i=1}^{n}\left(x_i-\bar{x}\right)^2}} \tag{71}$$

$$s_{b_0} = \frac{\sqrt{\frac{\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}{n-2}} \times \sqrt{\sum_{i=1}^{n} x_i^2}}{\sqrt{n\sum_{i=1}^{n}\left(x_i-\bar{x}\right)^2}} \tag{72}$$

Erro-padrão residual

$$S_{YX} = S_e = \hat{\sigma^2} = \sqrt{\frac{\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}{n-2}} \tag{73}$$

A função `lm()` é usada para criar o objeto resultante da regressão linea. A função `summary()` desse objeto recém-criado fornece diversas informações, tais como os coeficientes angular e linear, o teste de significância e seus respectivos valores-p, o erro-padrão da estimativa dos coeficientes, o erro-padrão residual, o coeficente de determinação, o coeficiente de determinação ajustado e o teste F de significância.

Análise dos resíduos

O resíduo de um modelo é dado por:

$$e_i = y_i - \hat{y}_i \tag{74}$$

Existem outras formas de avaliar os resíduos. Entre elas, temos:

Resíduo padronizado

O resíduo padronizado permite uma abordagem similar à do escore-z, para identificação de *outliers*. Também chamado de resíduo estudentizado internamente.

O resíduo é padronizado subtraindo o valor médio (zero) e dividido pelo desvio-padrão estimado (Charnet et al., p. 119, 2008; Devore, p. 484, 2006):

$$e_i^* = \frac{y_i - \hat{y}_i}{S_{YX}\sqrt{1-\frac{1}{n}-\frac{(x_i-\bar{x})^2}{\sum_{i=1}^{n}(x_i-\bar{x})^2}}} \tag{75}$$

Os resíduos padronizados, idealmente, devem cair no intervalo de ± 2 unidades. Se um determinado resíduo padronizado é 1,7, então o próprio resíduo é 1,7 (estimado) desvios-padrão maiores do que o que seria esperado do ajuste do modelo correto.

PPGEQ/UERJ

Resíduos estudentizados

Calculado sem a i-ésima observação. A variável aleatória, z_i^*, tem distribuição t de Student, com $(n-1-2)$ graus de liberdade.

Os resíduos estudentizados são chamados, também, de *resíduos Jackknife* e de *resíduos estudentizados externamente* (Charnet et al., p. 119, 2008).

$$z_i^* = \frac{y_i - \hat{y}_i}{S_{YX_{(i)}}\sqrt{1 - \frac{1}{n} - \frac{(x_i-\bar{x})^2}{\sum_{i=1}^{n}(x_i-\bar{x})^2}}} \tag{76}$$

Um valor acima de ± 3 é considerado um *outlier*.

Pontos de alavancagem

A matriz chapéu (*hat*) é usada para detectar pontos de alavancagem ou valores extremos na variável explicativa:

$$H = X\left(X'X\right)^{-1}X' \tag{77}$$

h_{ii} é o i-ésimo da diagonal principal da matriz chapéu H, que é chamado de *leverage*, ou ponto de alavanca, da observação i.

$$h_{ii} = x_i^{'}\left(X'X\right)^{-1}x_i = \frac{1}{n} + \frac{\left(x_i - \bar{x}\right)^2}{\sum_{i=1}^{n}\left(x_i - \bar{x}\right)^2} \tag{78}$$

Nem toda observação com h_{ii} alto é influente na análise da regressão.

Análise de resíduos: Teste de Breusch-Pagan (heterocedasticidade)

Aplicado apenas quando os resíduos seguem a distribuição normal:

$$e_i \sim N\left(0; \sigma^2\right)$$

Após a construção do modelo OLS, faz-se outro modelo de regressão por OLS, usando os resíduos ao quadrado, como variável dependente (regressão auxiliar):

$$\delta_0 e_i^2 = \delta_0 + \delta_1 \hat{y}_i \tag{79}$$

onde δ_0 e δ_1 são, respectivamente, os coeficientes linear e angular da regressão auxiliar. A partir do novo modelo estimado, obtém-se o coeficiente de determinação, r_{aux}^2, que será usado. A estatística do teste é então obtida:

$$\chi_{calc}^2 = n \times r_{aux}^2 \sim \chi^2\left(1-\alpha, 1\right) \tag{80}$$

PPGEQ/UERJ

Exemplos

Exemplo 1

```
# Criar os objetos
X  <- c(18, 16, 25, 22, 20, 21, 23, 19, 17)
Y <- c(5, 3, 10, 8, 6, 7, 9, 6, 5)
X; Y
```

```
## [1] 18 16 25 22 20 21 23 19 17
```

```
## [1]  5  3 10  8  6  7  9  6  5
```

```
# Criar gráfico de dispersão
plot(X, Y, pch = 16, col = "blue")
```

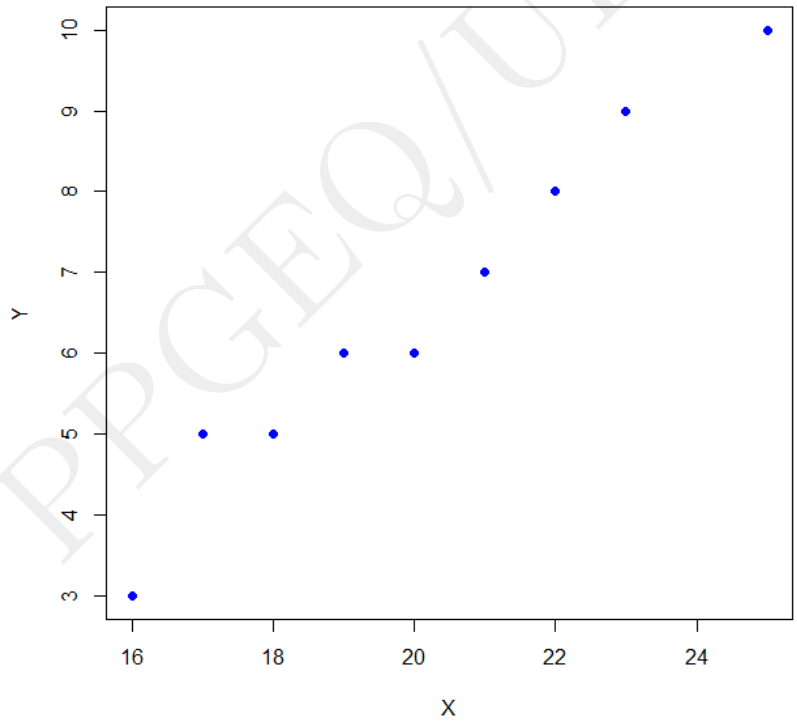


Figura 55: Diagrama de dispersão.

PPGEQ/UERJ

```r
# ------------------------------
# Definição do modelo e estimação de parâmetros
# ------------------------------
mod <- lm(Y~X)
mod
```

```
##
## Call:
## lm(formula = Y ~ X)
##
## Coefficients:
## (Intercept)            X
##     -8.1710       0.7323
```

```r
# ------------------------------
# Análise da variância e sumário do modelo
# ------------------------------
anova(mod)
```

```
## Analysis of Variance Table
##
## Response: Y
##           Df Sum Sq Mean Sq F value    Pr(>F)
## X          1 36.938  36.938   201.4 2.048e-06 ***
## Residuals  7  1.284   0.183
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
summary.lm(mod)
```

```
##
## Call:
## lm(formula = Y ~ X)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.54516 -0.20645 -0.00968  0.25806  0.72258
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -8.1710     1.0475  -7.801 0.000107 ***
## X             0.7323     0.0516  14.191 2.05e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

PPGEQ/UERJ

```
## Residual standard error: 0.4283 on 7 degrees of freedom
## Multiple R-squared:  0.9664, Adjusted R-squared:  0.9616
## F-statistic: 201.4 on 1 and 7 DF,  p-value: 2.048e-06
```

```r
# ------------------------------
# Diagnóstico para análise de modelos lineares
# ------------------------------
# Resíduo ordinário
res <- residuals(mod); res
```

```
##            1            2            3            4            5            6
## -0.009677419 -0.545161290 -0.135483871  0.061290323 -0.474193548 -0.206451613
##            7            8            9
##  0.329032258  0.258064516  0.722580645
```

```r
# Resíduo padronizado internamente (Studentized residual)
rp <- rstandard(mod); rp
```

```
##           1           2           3           4           5           6
## -0.02489048 -1.58680200 -0.42973639  0.15642004 -1.17452974 -0.51463937
##           7           8           9
##  0.87683763  0.64567741  1.95034531
```

```r
# Resíduo padronizado externamente (Jacknife residual, Rstudent)
rs <- rstudent(mod); rs
```

```
##           1           2           3           4           5           6
## -0.02304513 -1.83594454 -0.40321290  0.14507058 -1.21353678 -0.48574139
##           7           8           9
##  0.86042000  0.61641893  2.67222697
```

```r
# Gráficos de diagnósticos (histograma, gráfico de caixa (boxplot) e normalidade)
par(mfrow=c(1,3))
hist(rs, col = "darkgoldenrod1");
boxplot(rs, col = "greenyellow");
qqnorm(rs, pch = 16, col = "red");
qqline(rs, col = "blue")
```

PPGEQ/UERJ

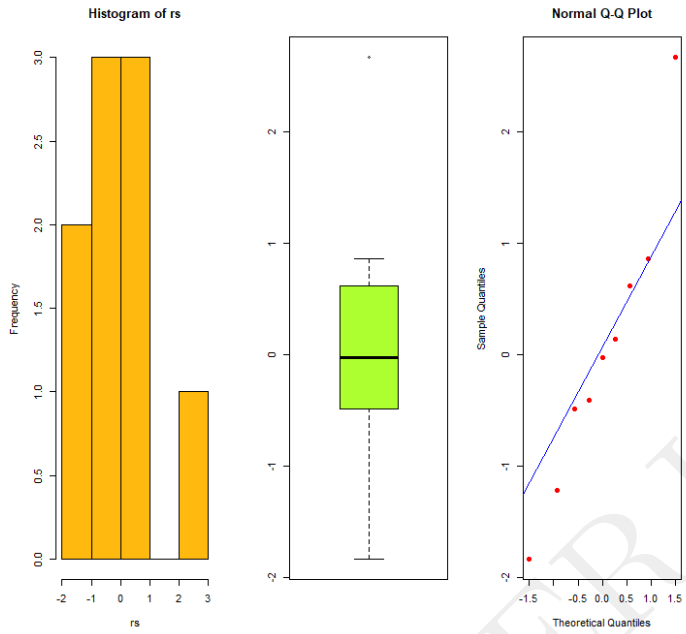


Figura 56: Análise de resíduos.

Exemplo 2

```
# Criar os objetos
X <- c(17:25, 35)
Y <- c(75, 78, 77, 81, 86, 89, 91, 90, 93, 82)
X; Y
```

```
##  [1] 17 18 19 20 21 22 23 24 25 35
```

```
##  [1] 75 78 77 81 86 89 91 90 93 82
```

```
mod <- lm(Y~X)
# Obter os valores da influência
h <- hatvalues(mod); h
```

```
##         1         2         3         4         5         6         7         8
## 0.2233503 0.1818951 0.1489002 0.1243655 0.1082910 0.1006768 0.1015228 0.1108291
##         9        10
## 0.1285956 0.7715736
```

```
# Obter o valor crítico da influência
P = length(mod$coefficient);
N = length(Y);
P; N
```

PPGEQ/UERJ

```
## [1] 2
```

```
## [1] 10
```

```r
hc <- 3*P/N;
hc
```

```
## [1] 0.6
```

```r
# Resíduo padronizado externamente (Jacknife residual, Rstudent)
rs <- rstudent(mod); rs
```

```
##          1          2          3          4          5          6          7
## -1.2242857 -0.6924974 -0.9500225 -0.3269109  0.3915100  0.8153897  1.1018505
##          8          9         10
##  0.8281077  1.3438401 -9.5828098
```

```r
# Diagnóstico de outliers
par(mfrow = c(1, 1))
plot(h, rs, xlim = c(0, 1.2), ylim = c(-3, 3), pch = 16,
     ylab = "Resíduos padronizados externamente", col = "blue" )
abline(v = hc, col = "red")
abline(h = -2, col = "red")
abline(h = 2, col = "red")
```

PPGEQ/UERJ

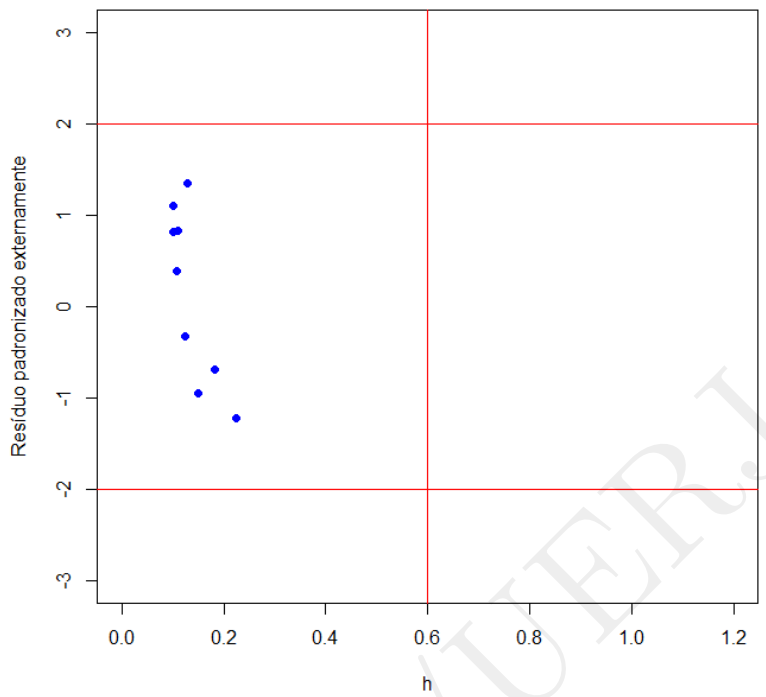


Figura 57: Resíduos padronizados externamente versus influência

PPGEQ/UERJ

Modelos Empíricos: Regressão Linear Múltipla (MLR)

Assim como no conteúdo sobre Regressão Linear Simples, a seguir são dadas algumas equações de onde são originadas as saídas das funções usadas no `R`. **Não deixe de utilizar o material teórico de referência para maiores detalhes! Assista às aulas!**

Modelo de regressão com k variáveis independentes

$$Y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \beta_3 X_{3i} + \cdots + \beta_k X_{ki} + e_i \tag{81}$$

Equação da linha de previsão

$$\hat{y}_i = b_0 + b_1 X_{1i} + b_2 X_{2i} + b_3 X_{3i} + \cdots + b_k X_{ki} \tag{82}$$

Métricas

Raiz do Erro Quadrático Médio (RMSE)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{83}$$

Erro Quadrático Médio (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{84}$$

Erro Absoluto Médio (MAE)

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{85}$$

Multicolineridade: Tolerância e Fator de Inflação da Variância

$$VIF(\beta_j) = \frac{1}{1 - r_j^2} \tag{86}$$

$$Tol = \frac{1}{VIF} \tag{87}$$

PPGEO/UERJ

Seleção de variáveis

Cp de Mallows

$$C_p = \frac{(1 - r_k^2)(n - T)}{1 - r_T^2} - [n - 2(k + 1)] \tag{88}$$

onde k é o número de variáveis independentes incluídas no modelo, T é o número total de parâmetros (incluindo o intercepto), r_k^2 é o coeficiente de determinação múltipla para o modelo com k variáveis independentes e r_T^2 é o coeficiente de determinação com todos os parâmetros estimados.

Critério de Informação de Akaike

$$AIC = -2 \log L + k n_{par} \tag{89}$$

onde L é a estimativa de máxima verossimilhança, k usualmente assume o valor de 2 para AIC e $\log(n)$ para BIC, e n_{par} é o número de parâmetros no modelo estimado.

Note que o primeiro valor de saída (`Full Model`) da função `ols_step_backward_aic()` é o mesmo daquele indicado pela função `AIC()` do pacote `stats`. A vantagem do pacote `olsrr` é que ele fornece os demais valores de AIC sem uma determinada variável no modelo.

Análise de resíduos: pontos influentes

Difference in Fits, DFFITS

Mede a influência que a i-ésima observação tem sobre seu próprio valor ajustado[5].

$$DFFITS_i = \frac{\hat{y}_i - \hat{y}_{(i)}}{\sqrt{\hat{\sigma}_{(i)}^2 h_{ii}}} \tag{90}$$

Uma observação é influente (usando a métrica DFFITS), se:

$$DFFITS_i > 2\sqrt{\frac{k + 2}{n - k - 2}} \tag{91}$$

Onde k é o número de parâmetros da regressão, excluindo-se o intercepto.

Difference in Betas, DFBETAS

Mede a influência da i-ésima observação sobre o coeficiente de X_j.

$$DFBETAS_i = \frac{\hat{\beta}_j - \hat{\beta}_{j(i)}}{\sqrt{\hat{\sigma}_i^2 c_{jj}}} \tag{92}$$

[1]Chatterjee, S.; Hadi, A. S. Regression analysis by example, 5th ed. New Jersey: John Wiley & Sons, 2012, p. 112

PPGEQ/UERJ

Onde c_{jj} é o j-ésimo elemento da diagonal de $(X'X)^{-1}$.

Distância de Cook

Mede a mudança nos coeficientes de regressão se a i-ésima observação é removida dos dados[6].

$$D^2_{(1)} = \frac{\sum_{j=1}^{n} \left(\hat{y}_j - \hat{y}_{j(i)}\right)^2}{pS_e^2} \tag{93}$$

onde S_e é o erro-padrão residual.

Exemplo 3

```
# Criação dos vetores
y <- c(8.0, 8.3, 8.5, 8.8, 9.0, 9.3, 9.3, 9.5,9.8, 10.0, 10.3, 10.5, 10.8, 11.0,
       11.3, 11.5, 11.8, 12.3, 12.5)
x1 <- c(5.2, 5.2, 5.8, 6.4, 5.8, 5.2, 5.6, 6.0, 5.2, 5.8, 6.4, 6.0, 6.2, 6.2,
        6.2, 5.6, 6.0, 5.8, 5.6)
x2 <- c(19.6, 19.8, 19.6, 19.4, 18.6, 18.8, 20.4, 19.0, 20.8, 19.9, 18.0, 20.6,
        20.2, 20.2, 19.2, 17.0, 19.8, 18.8, 18.6)
x3 <- c(29.6, 32.4, 31.0,32.4, 28.6, 30.6, 32.4, 32.6, 32.2, 31.8, 32.6, 33.4,
        31.8, 32.4, 31.4, 33.2, 35.4, 34.0, 34.2)
x4 <- c(94.9, 89.7, 96.2, 95.6, 86.5, 84.5, 88.8, 85.7, 93.6, 86.0, 87.1, 93.1,
        83.4, 94.5, 83.4, 85.2, 84.1, 86.9, 83.0)
x5 <- c(2.1, 2.1, 2.0, 2.2, 2.1, 2.0, 2.2, 2.1, 2.3, 2.1, 2.0, 2.1, 2.2, 2.0,
        1.9, 2.1, 2.0, 2.1, 1.9)
x6 <- c(2.3, 1.8, 2.0, 2.1, 1.8, 2.1, 1.9, 1.9, 2.1, 1.8, 1.6, 2.1, 2.1, 1.9,
        1.8, 2.1, 1.8, 1.8, 2.0)
```

```
# Requisição do pacote:
library(psych)
# Visualização
pairs.panels(cbind(y, x1, x2, x3, x4, x5, x6),
             method = "pearson",
             hist.col = "#00AFBB",
             density = TRUE,
             ellipses = TRUE)
```

[1]Massart et al, 1997, p. 203

PPGEO/UERJ

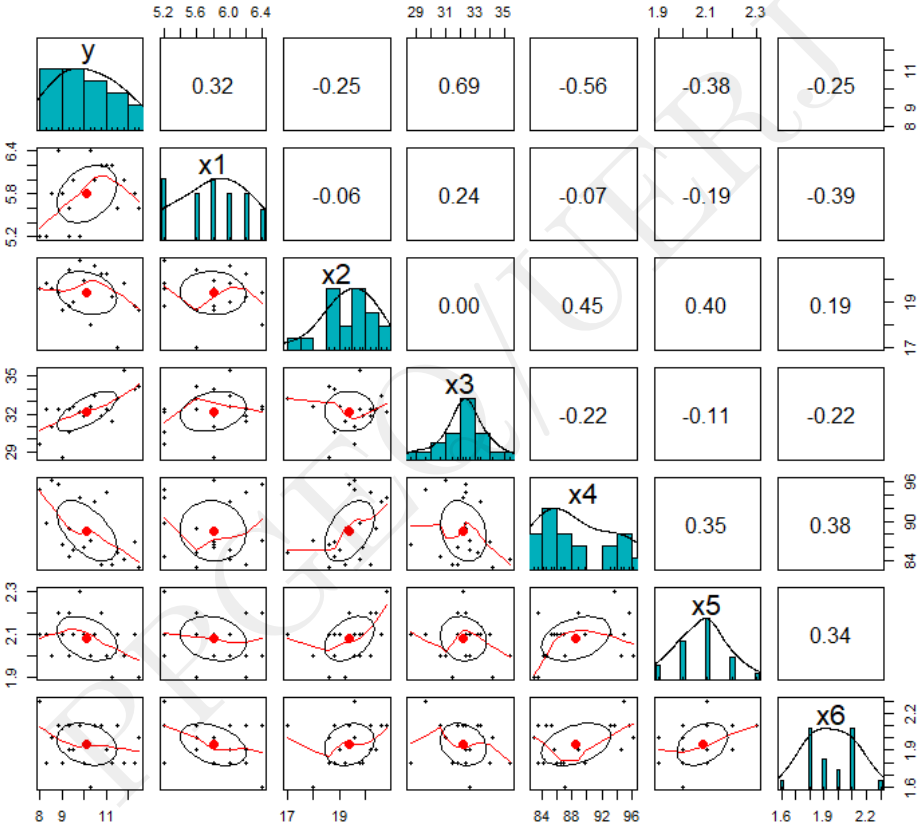


Figura 58: EDA usando psych.

```r
# Requisição do pacote:
library(corrgram)
corrgram(cbind(y, x1, x2, x3, x4, x5, x6),
         lower.panel = panel.pts,
         upper.panel = panel.conf,
         diag.panel = panel.density)
```

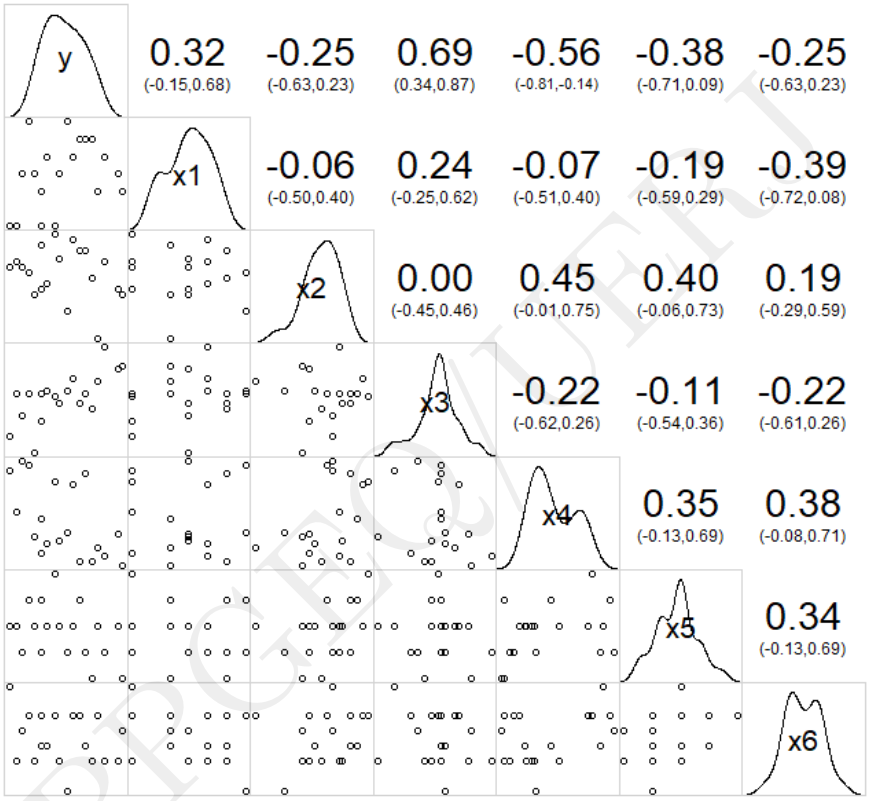


Figura 59: EDA usando corrgram.

PPGEC/UFRJ

```r
# Requisição do pacote:
library(olsrr)
data <- data.frame(cbind(y, x1, x2, x3, x4, x5, x6))
data
```

```
##       y  x1   x2   x3   x4  x5  x6
## 1   8.0 5.2 19.6 29.6 94.9 2.1 2.3
## 2   8.3 5.2 19.8 32.4 89.7 2.1 1.8
## 3   8.5 5.8 19.6 31.0 96.2 2.0 2.0
## 4   8.8 6.4 19.4 32.4 95.6 2.2 2.1
## 5   9.0 5.8 18.6 28.6 86.5 2.1 1.8
## 6   9.3 5.2 18.8 30.6 84.5 2.0 2.1
## 7   9.3 5.6 20.4 32.4 88.8 2.2 1.9
## 8   9.5 6.0 19.0 32.6 85.7 2.1 1.9
## 9   9.8 5.2 20.8 32.2 93.6 2.3 2.1
## 10 10.0 5.8 19.9 31.8 86.0 2.1 1.8
## 11 10.3 6.4 18.0 32.6 87.1 2.0 1.6
## 12 10.5 6.0 20.6 33.4 93.1 2.1 2.1
## 13 10.8 6.2 20.2 31.8 83.4 2.2 2.1
## 14 11.0 6.2 20.2 32.4 94.5 2.0 1.9
## 15 11.3 6.2 19.2 31.4 83.4 1.9 1.8
## 16 11.5 5.6 17.0 33.2 85.2 2.1 2.1
## 17 11.8 6.0 19.8 35.4 84.1 2.0 1.8
## 18 12.3 5.8 18.8 34.0 86.9 2.1 1.8
## 19 12.5 5.6 18.6 34.2 83.0 1.9 2.0
```

```r
# Modelo estatístico
model <- ols_regress(y~x1 + x2 + x3 + x4 + x5 + x6, data = data)
model
```

PPGEQ/UERJ

```
##                          Model Summary
## ------------------------------------------------------------
## R                        0.846       RMSE                0.888
## R-Squared                0.715       Coef. Var           8.766
## Adj. R-Squared           0.573       MSE                 0.789
## Pred R-Squared           0.319       MAE                 0.578
## ------------------------------------------------------------
##  RMSE: Root Mean Square Error
##  MSE: Mean Square Error
##  MAE: Mean Absolute Error
##
##                               ANOVA
## -------------------------------------------------------------------
##                Sum of
##               Squares        DF    Mean Square      F        Sig.
## -------------------------------------------------------------------
## Regression     23.757         6          3.959     5.02    0.0086
## Residual        9.464        12          0.789
## Total          33.221        18
## -------------------------------------------------------------------
##
##                                   Parameter Estimates
## ----------------------------------------------------------------------------------------
##       model      Beta    Std. Error    Std. Beta      t        Sig      lower     upper
## ----------------------------------------------------------------------------------------
## (Intercept)     4.053         8.260                  0.491    0.632    -13.944    22.051
##          x1     0.655         0.584        0.193     1.122    0.284     -0.616     1.927
##          x2    -0.020         0.261       -0.014    -0.075    0.941     -0.587     0.548
##          x3     0.489         0.141        0.569     3.468    0.005      0.182     0.796
##          x4    -0.121         0.056       -0.413    -2.177    0.050     -0.242     0.000
##          x5    -2.431         2.347       -0.185    -1.036    0.321     -7.546     2.684
##          x6     1.382         1.472        0.174     0.939    0.366     -1.826     4.589
## ----------------------------------------------------------------------------------------
```

PPGEQ/UFRJ

```r
# Gráfico dos resíduos versus valores preditos
model <- lm(y ~ x1 + x2 + x3 + x4 + x5 + x6, data = data)
# Requisição do pacote:
library(olsrr)
ols_plot_resid_fit(model)
```

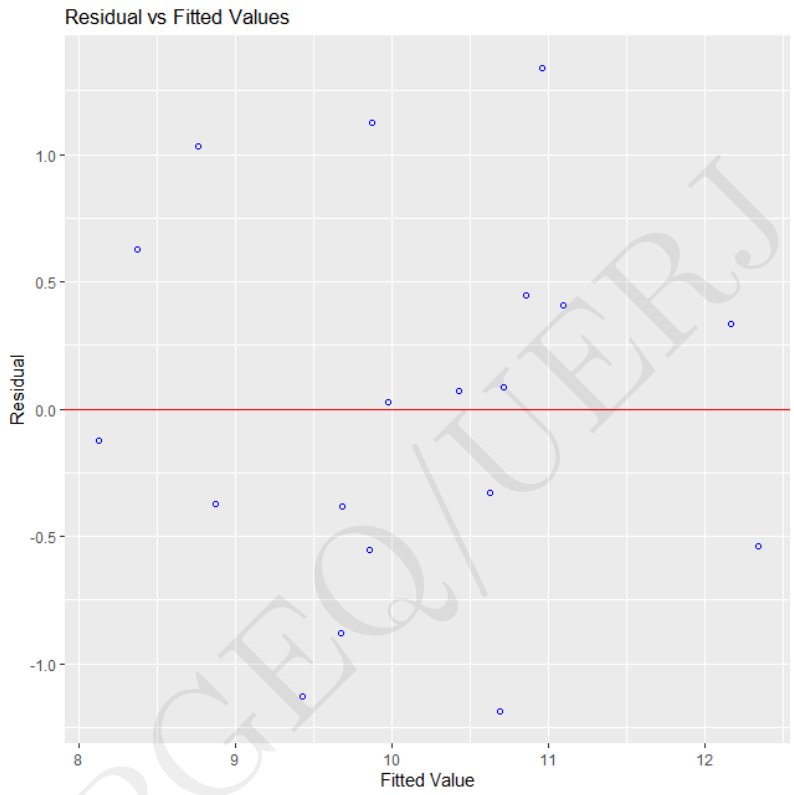


Figura 60: Gráfico dos resíduos versus valores preditos.

PPGEQ/UERJ

```r
# Requisição do pacote:
library(olsrr)
# Histograma dos resíduos
ols_plot_resid_hist(model)
```

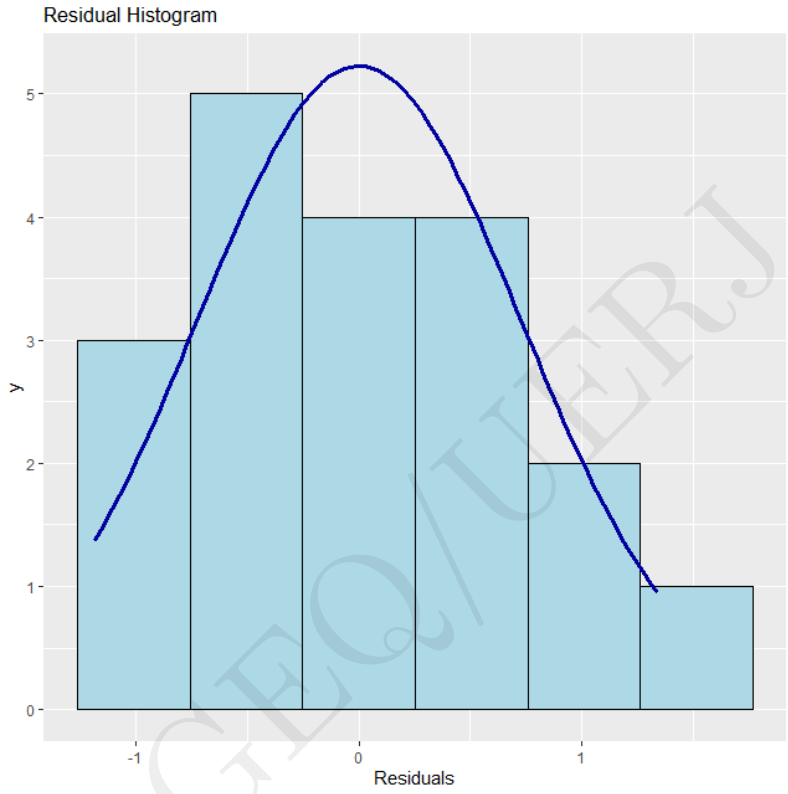


Figura 61: Histograma dos resíduos.

PPGEQ/UERJ

```r
# Requisição do pacote:
library(olsrr)
# Medida de influência: DFBetas
ols_plot_dfbetas(model)
```

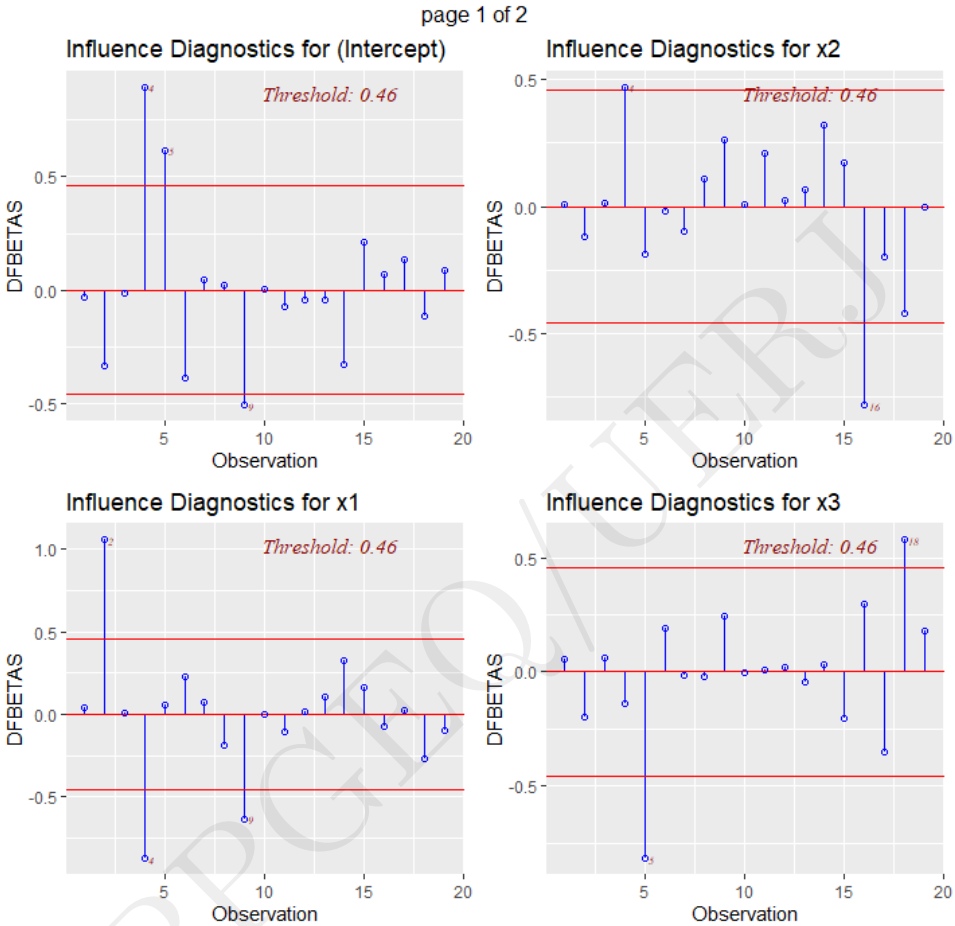


Figura 62: DFBetas (parte 1).

PPGEQ/UFRJ

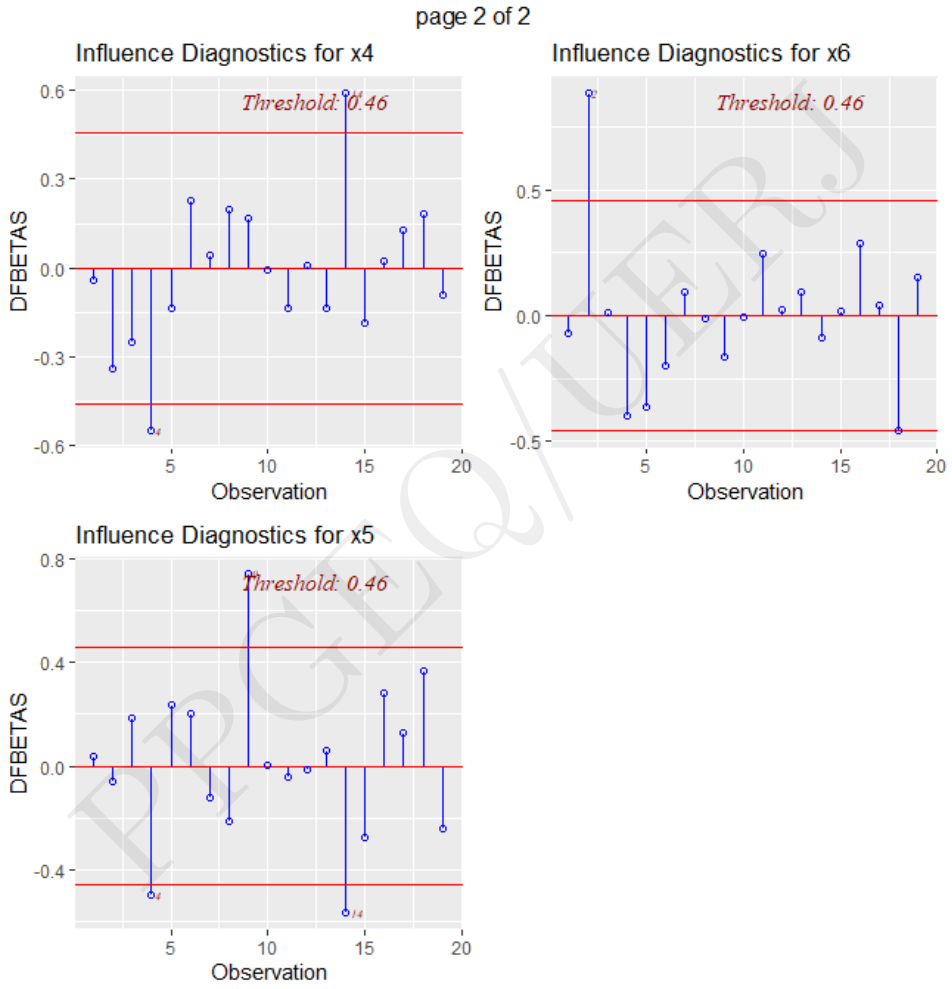


Figura 63: DFBetas (parte 2).

PPGEQ/UFRJ

```r
# Requisição do pacote:
library(olsrr)
# Medida de influência: DFFits
ols_plot_dffits(model)
```

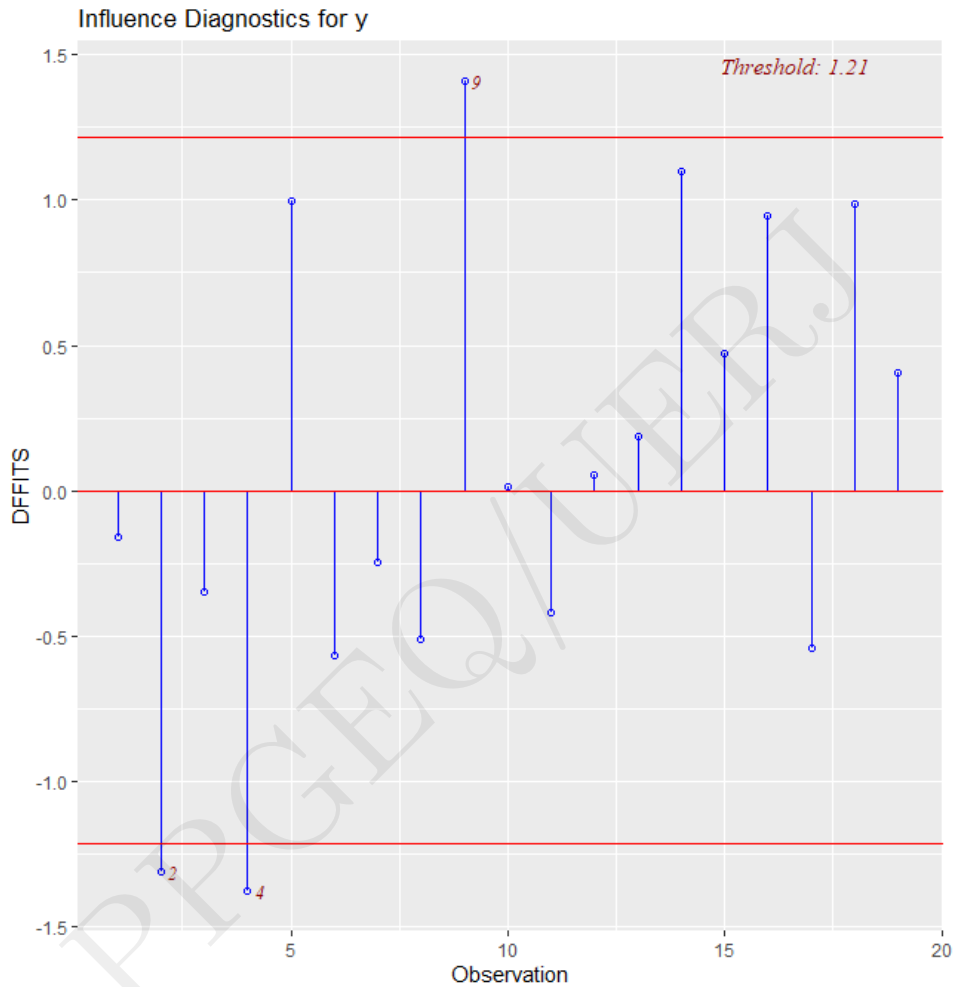


Figura 64: DFFits.

PPGEQ/UERJ

```r
# Teste de Breusch-Pagan:
model <- lm(y ~ x1 + x2 + x3 + x4 + x5 + x6, data = data)
ols_test_breusch_pagan(model)
```

```
##
##  Breusch Pagan Test for Heteroskedasticity
##  -----------------------------------------
##  Ho: the variance is constant
##  Ha: the variance is not constant
##
##             Data
##  -----------------------------
##  Response : y
##  Variables: fitted values of y
##
##         Test Summary
##  -----------------------------
##  DF            =    1
##  Chi2          =    0.02805524
##  Prob > Chi2   =    0.866979
```

```r
# Requisição do pacote
library(olsrr)
# Diagnóstico de Colinearidade
model <- lm(y~x1 + x2 + x3 + x4 + x5 + x6, data = data)
ols_coll_diag(model)
```

```
## Tolerance and Variance Inflation Factor
## ---------------------------------------
##   Variables Tolerance      VIF
## 1        x1 0.8040604 1.243688
## 2        x2 0.7152740 1.398066
## 3        x3 0.8814260 1.134525
## 4        x4 0.6602393 1.514602
## 5        x5 0.7470970 1.338514
## 6        x6 0.6901237 1.449016
##
##
## Eigenvalue and Condition Index
## ------------------------------
##     Eigenvalue Condition Index    intercept           x1           x2
## 1 6.9825309436         1.00000 1.247149e-05 7.364649e-05 3.325994e-05
## 2 0.0089695655        27.90106 5.239874e-04 1.462584e-01 8.564368e-05
## 3 0.0028592285        49.41764 3.344767e-05 2.087981e-01 1.382711e-01
## 4 0.0025051104        52.79502 3.519878e-03 3.708869e-01 3.479462e-04
```

PPGEQ/UERJ

```
## 5 0.0014964474        68.30865 2.612591e-05 6.819063e-02 5.953957e-02
## 6 0.0011345520        78.45023 8.837690e-03 7.733757e-02 7.953618e-01
## 7 0.0005041525       117.68618 9.870464e-01 1.284547e-01 6.360691e-03
##             x3           x4           x5           x6
## 1 4.114583e-05 3.499225e-05 0.0000356106 0.0001022816
## 2 1.944604e-02 3.232813e-03 0.0027811266 0.2851316935
## 3 2.779560e-02 5.812378e-02 0.0844225940 0.5529225182
## 4 4.387230e-01 1.166511e-01 0.0051774576 0.0047455778
## 5 8.003889e-02 2.745654e-01 0.7166307075 0.0001766351
## 6 5.104797e-02 4.603169e-01 0.0670523188 0.1119466921
## 7 3.829073e-01 8.707501e-02 0.1239001849 0.0449746015
```

PPGEQ/UERJ

Regressão por passos (Stepwise)

```r
# Requisição do pacote:
library(olsrr)
# Regressão por passos (Stepwise)
ols_step_both_p(model)
```

```
##
##                              Stepwise Selection Summary
## --------------------------------------------------------------------------------
##                       Added/                    Adj.
## Step    Variable    Removed     R-Square    R-Square     C(p)       AIC       RMSE
## --------------------------------------------------------------------------------
##    1       x3       addition       0.477       0.446    7.0190    58.2116    1.0107
##    2       x4       addition       0.648       0.604    1.8440    52.7201    0.8554
## --------------------------------------------------------------------------------
```

```r
# Critérios de avaliação do modelo
k <- ols_step_both_p(model)
```

```r
# Visualização
plot(k)
```

PPGEQ/UERJ

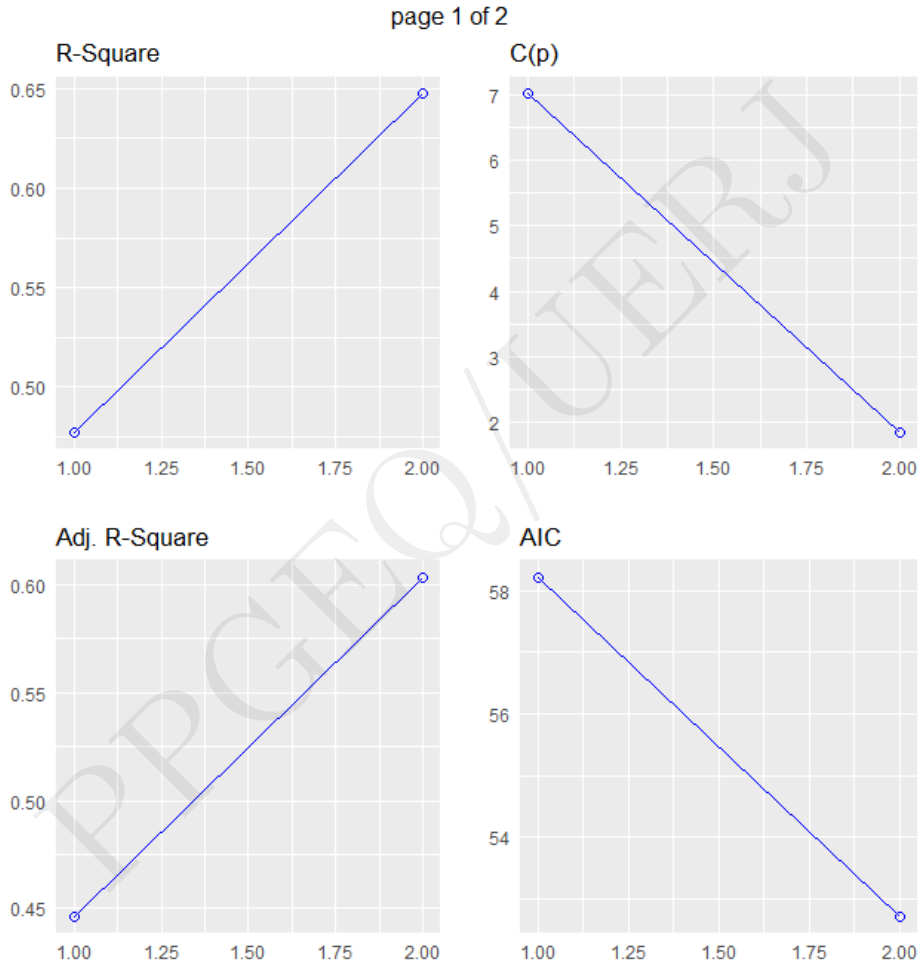


Figura 65: Regressão stepwise (parte 1).

PPGEQ/UERJ

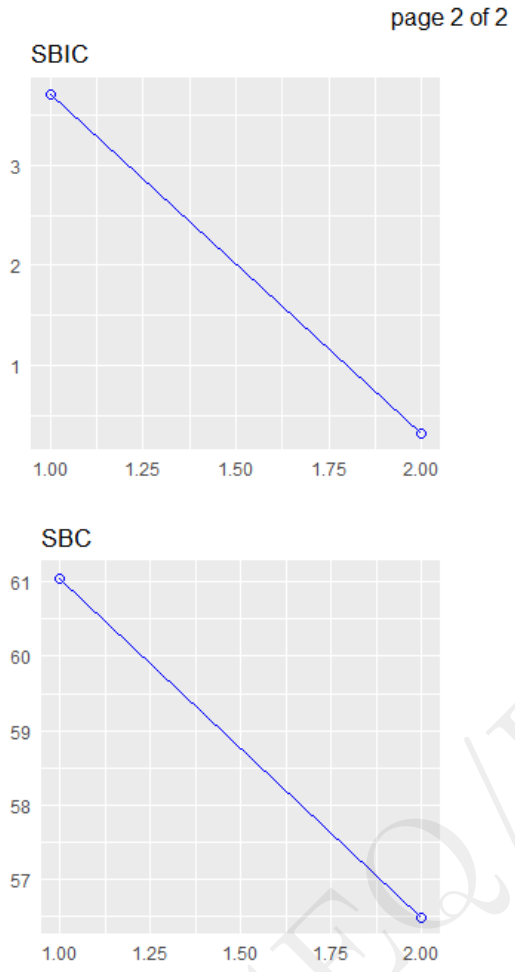


Figura 66: Regressão stepwise (parte 2).

PPGEQ/UERJ

Regressão por passos para trás (Backward Regressrion)

```r
# Regressão por passos para trás usando o critério de Akaike
k <- ols_step_backward_aic(model)
k
```

```
##
##
##                    Backward Elimination Summary
## ----------------------------------------------------------------
## Variable       AIC       RSS      Sum Sq     R-Sq      Adj. R-Sq
## ----------------------------------------------------------------
## Full Model    56.678     9.464    23.757    0.71511      0.57266
## x2            54.687     9.469    23.752    0.71497      0.60535
## x6            54.043    10.169    23.052    0.69390      0.60644
## x1            53.077    10.738    22.483    0.67677      0.61213
## x5            52.720    11.708    21.513    0.64758      0.60353
## ----------------------------------------------------------------
```

```r
# Visualização
plot(k)
```

PPGEQ/UFRJ

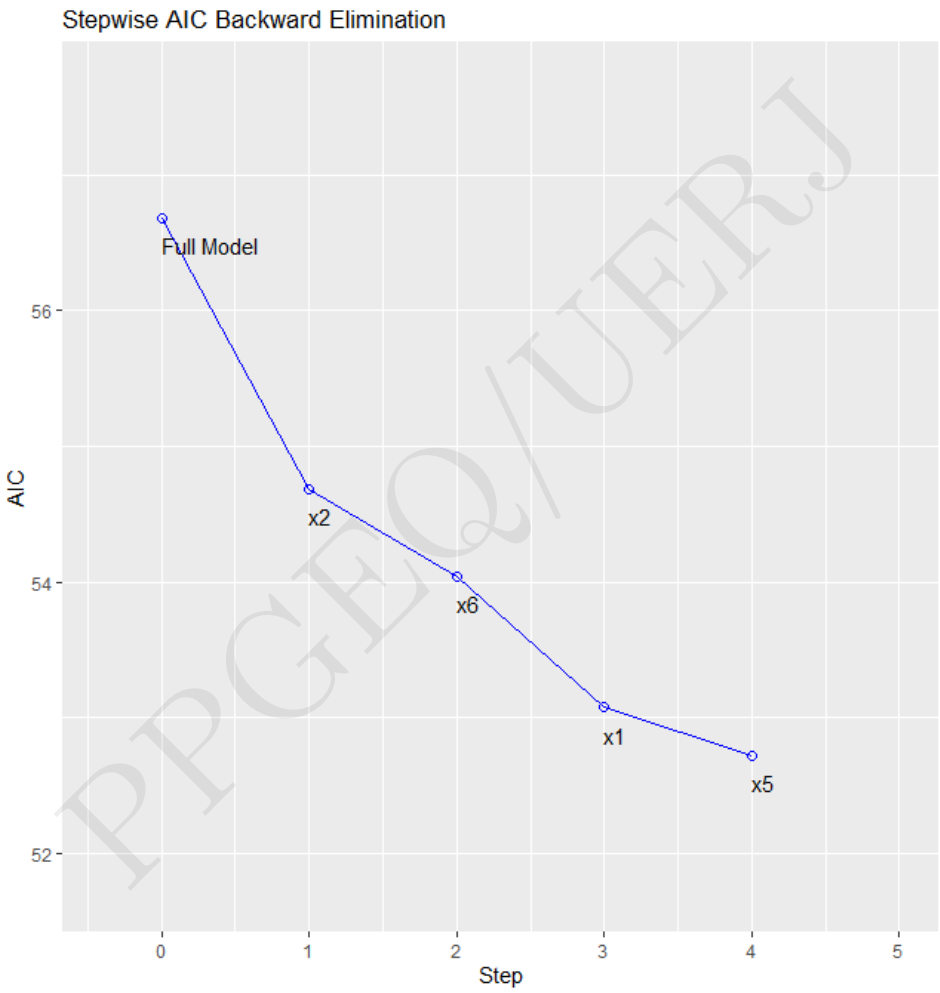


Figura 67: Regressão backward com AIC.

PPGEQ/UERJ

```r
# Gráfico de normalidade dos resíduos
ols_plot_resid_qq(model)
```

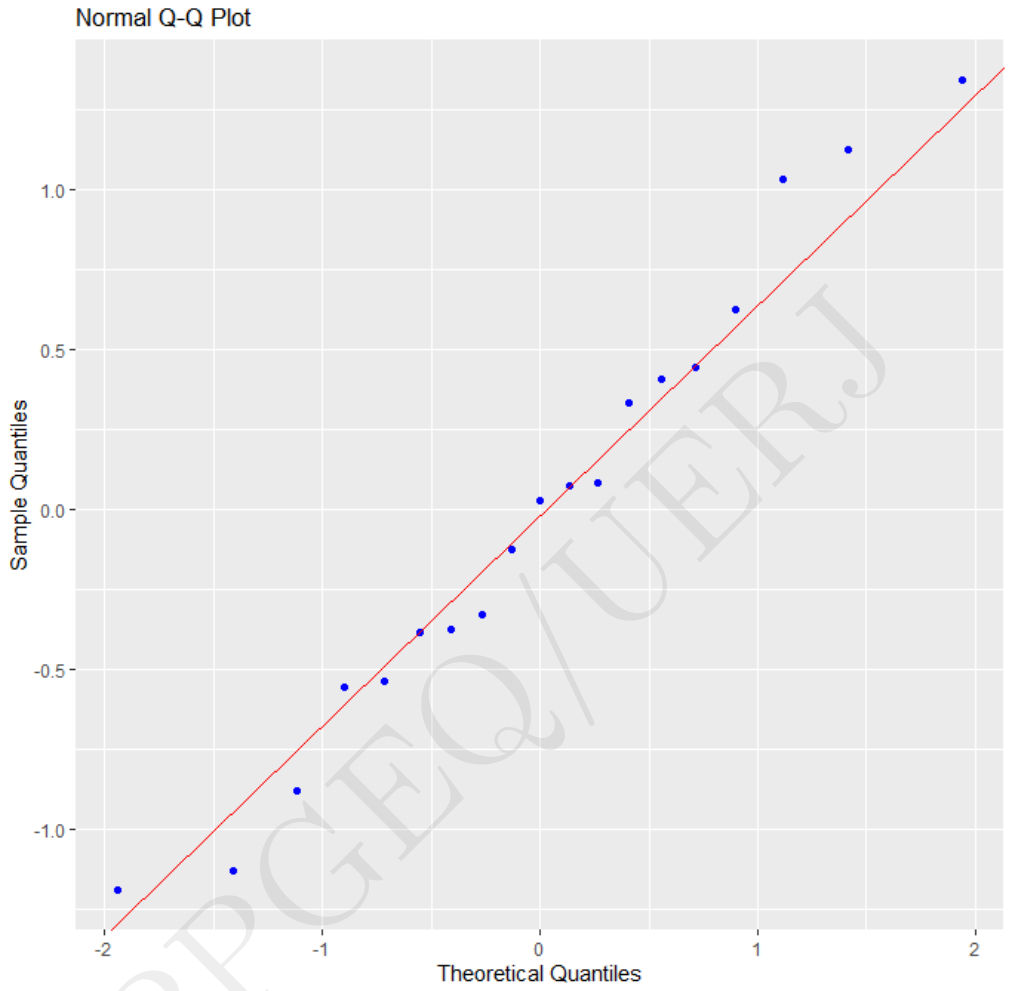


Figura 68: Gráfico de normalidade dos resíduos (pacote olsrr).

```r
# Teste de normalidade dos resíduos
ols_test_normality(model)
```

```
## -----------------------------------------------
##        Test              Statistic       pvalue
## -----------------------------------------------
## Shapiro-Wilk              0.9725         0.8255
## Kolmogorov-Smirnov        0.0955         0.9883
## Cramer-von Mises          1.7596         0.0000
## Anderson-Darling          0.1741         0.9126
## -----------------------------------------------
```

PPGEQ/UERJ

```r
# Correlação dos resíduos observados e os resíduos
# esperados sob a condição de normalidade
ols_test_correlation(model)
```

```
## [1] 0.9912222
```

PPGEQ/UERJ